FLEXSched – The Complete Project Guide

Intelligent, fatigue-aware task scheduler & planner — Python · C++ · scikit-learn · SQLite · Catch2

Author: Adit Bissa

How to use this guide

This document has two jobs:

1. **Teach you the project from zero.** Every technology, every file, every important line of code and every algorithm is explained. If you only know Python basics, ML basics and SQL basics, read it front to back once; you will then understand every design decision well enough to defend it.
2. **Let you present the project.** Part 1 is a self-contained presentation (pitch, architecture, results, demo script). Part 7 is an interview drill with model answers.

Part	What it gives you	When to read it
1. Overview	pitch, architecture, data flow, results, demo script	before any presentation
2. Tech-stack primers	Python, C++, CMake, SQLite/SQL, scikit-learn, testing concepts used here	first study pass
3. C++ engine code walkthrough	every engine file, fully listed and explained	second study pass
4. Python code walkthrough	every Python file, fully listed and explained	second study pass
5. Algorithms & design deep dive	worked examples, complexity, design patterns, the story of the deadline guard	before technical interviews
6. Results, limitations, how to run	honest numbers, assumptions, setup and troubleshooting	before demos
7. Interview drill	what → why → how → trade-offs → advanced, for every resume line + tech-stack justification	the night before

> **Golden rule for interviews:** never claim more than the code does. Everything this guide says about FLEXSched is backed by a file you can open, and Part 6 lists the limitations so that you can name them before the interviewer does.

Part 1 — The project in ten minutes

1.1 Elevator pitches

30 seconds. "FLEXSched is a task scheduler that knows you get tired. It learns from your work history how long tasks *really* take, including how much slower you are when fatigued, using scikit-learn. A C++ engine then simulates your energy through the day and builds a timetable that does hard work when you're fresh, easy work when you're tired, and inserts breaks, while a lookahead guard makes sure it never causes more deadline misses than the classic Earliest-Deadline-First rule. Data lives in SQLite, and the engine is covered by Catch2 unit tests."

2 minutes (add these to the 30-second version):

- *The problem:* people underestimate tasks (the "planning fallacy") and slow down when tired, but calendar tools treat durations as fixed numbers.
- *The key idea:* a task's duration is not a number but a **curve**: minutes as a function of fatigue. The ML model produces the curve; the engine reads it at the fatigue level it simulates.
- *The architecture:* Python handles data, ML and orchestration. The C++ engine is a standalone executable that takes JSON on stdin and writes JSON on stdout, so each half can be tested and replaced on its own.
- *The evidence:* on a temporally held-out test set the model cuts duration error by 62% versus trusting the user's estimate. Over 200 random scheduling scenarios the fatigue-aware policy lowers average fatigue at task start by 17% and peak fatigue by 7%, with the same total lateness as EDF.

1.2 The problem

Two well-documented human facts:

1. **Planning fallacy.** People systematically underestimate how long tasks take, and the error depends on the kind of task (coding is notoriously underestimated; admin often overestimated).
2. **Fatigue.** Mental work tires you; tired people are slower and make more mistakes. Short breaks restore a lot of capacity; a night's sleep restores nearly all of it.

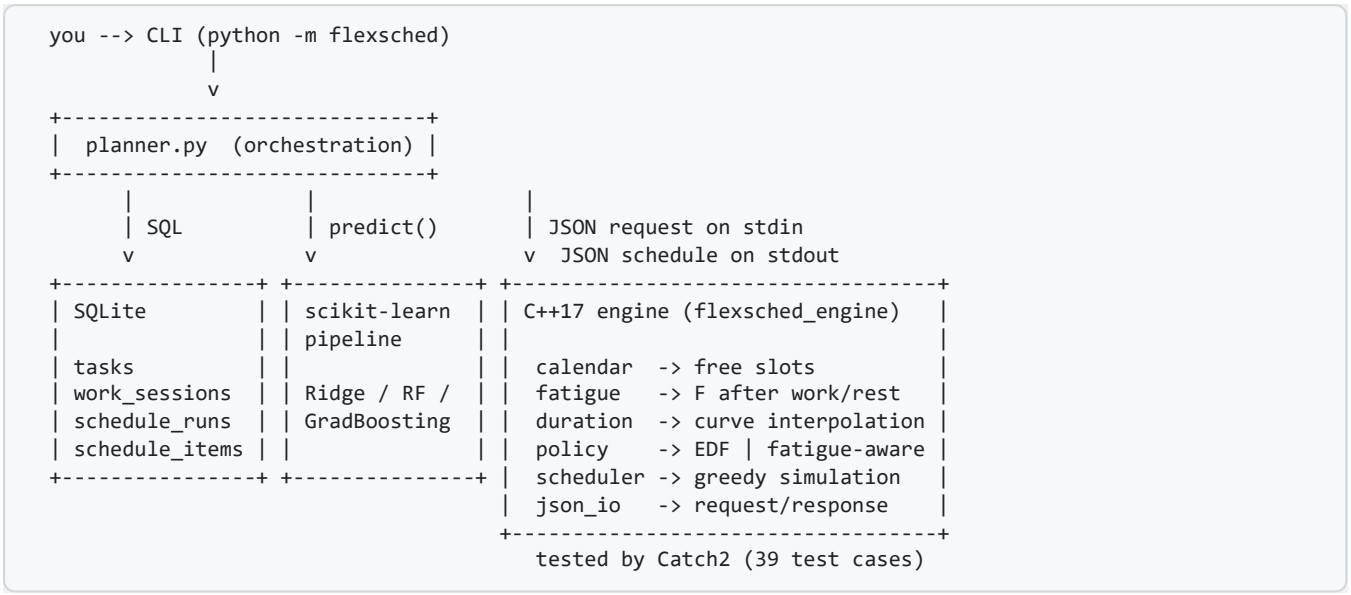
Ordinary to-do and calendar apps ignore both. A schedule built on raw guesses and a fixed duration per task falls apart by mid-afternoon.

1.3 What FLEXSched does

- Stores tasks (title, category, difficulty 1–5, priority 1–5, your estimate, optional deadline) and a **history of real work sessions** in SQLite.
- Reconstructs **how tired you were** at the start of every past session by replaying the day through a fatigue model.
- Trains and compares five regression models (two baselines, Ridge, Random Forest, Gradient Boosting), picks the best by **time-series cross-validation**, and predicts each task's duration at five fatigue levels → a **duration curve**.

- Computes **free time** (working hours minus meetings/lectures) and runs a **greedy, fatigue-simulating scheduler** in C++ with two pluggable strategies:
  - `edf`: Earliest Deadline First, the fatigue-blind textbook baseline;
  - `fatigue_aware`: priority + urgency + *energy match*, breaks, and a **rollout deadline guard**.
- Saves every generated plan, compares strategies, benchmarks them on hundreds of random scenarios, and shows estimation-bias statistics, all from a command-line interface.

1.4 Architecture



Responsibilities

Layer	Language	Files	Owns
Interface	Python	`cli.py`	parsing commands, printing tables
Service / orchestration	Python	`planner.py`, `benchmark.py`	DB → ML → engine → DB
ML	Python	`predictor.py`, `synthetic.py`, `fatigue.py`	training, evaluation, curves
Persistence	Python + SQL	`schema.sql`, `db.py`, `repository.py`	all SQL lives here
Scheduling core	C++17	`engine/`	free time, fatigue, policies, scheduling

1.5 What happens when you run `python -m flexsched plan`

1. `cli.py` parses the arguments (`--start`, `--days`, `--block`, ...) and opens SQLite.
2. `planner.plan()` loads all `pending` / `scheduled` tasks through `TaskRepository`.
3. It loads the trained model (`DurationPredictor.load`). For every task it predicts minutes at fatigue 0, 0.25, 0.5, 0.75 and 1.0 in **one vectorised call**.

4. It converts real datetimes into **integer minute offsets** from the first planned day's midnight (the engine never deals with dates or time zones).
5. It sends one JSON request to `flexsched_engine` through `subprocess.run`.
6. The engine merges the working windows, subtracts blocked intervals (free slots), then simulates the day: rest during gaps, breaks when tired, ask the policy what to do next, place the task, update fatigue, repeat.
7. The engine prints a JSON response: blocks, unscheduled tasks with reasons, metrics.
8. Python converts offsets back to datetimes, attaches titles, saves the run and its items **in one transaction**, updates task statuses and prints the timetable.

1.6 Results at a glance

Claim	Number	Where it comes from
Duration error vs trusting your estimate	**17.47 → 6.68 min MAE (−62%)**	`flexsched train` on 445 sessions, temporal 80/20 split
Variance explained	R^2 0.61 → 0.94	same
Avg fatigue at task start, fatigue-aware vs EDF	**0.321 → 0.265 (−17%)**	`flexsched benchmark --scenarios 200`
Peak fatigue	0.842 → 0.783 (−7%)	same
Total lateness	90.9 → 90.4 min (equal)	same
Engine tests	39 Catch2 test cases, ~340 assertions	`ctest`
Python tests	49 pytest tests	`pytest`

All data is synthetic (Part 6 explains why, and what that means).

1.7 Demo script (5 minutes)

```
python -m flexsched demo
```

What to say while it runs:

1. "*First it simulates 120 days of a person's work history. The simulated person has hidden habits: they underestimate coding by about 35%, they slow down when tired, and there's a post-lunch dip.*"
2. "*Then it trains five models. Note the two baselines: trusting your own estimate gives about 17.5 minutes of error; the chosen model gives about 6.7. Model selection uses time-series cross-validation, so the hold-out set is never used to choose.*"
3. "*It adds ten realistic tasks with deadlines and a couple of meetings, and compares strategies on identical input.*"
4. "*Here's the fatigue-aware plan. Look at the fatigue column: the 3-hour 'Implement auth module' task, the hardest one, starts at 0.01 fatigue right after lunch. Breaks appear when fatigue crosses 0.65, and nothing is late.*"

Then optionally show the engine standing alone:

```
build/flexsched_engine < engine/examples/request.json
```

"The engine is a plain executable with a JSON contract: I can test, debug and benchmark it without Python at all."

Part 2 — Tech-stack primers

Each primer starts from what you already know and stops exactly at the depth this project uses. The code samples are small and self-contained; Parts 3–4 then show the real code.

2.1 Python features used in this project

Modules, packages and `python -m`

- A **module** is one `.py` file. A **package** is a folder of modules with an `__init__.py` (here: `flexsched/`).
- `from .db import connect`: the leading dot is a **relative import** ("from the module `db` in *this* package").
- `python -m flexsched demo` runs the package as a program: Python executes `flexsched/__main__.py`, which calls `cli.main()`.

Type hints and `from __future__ import annotations`

```
def minutes_between(origin: datetime, moment: datetime) -> int: ...
def load(path: Union[str, Path, None] = None) -> Optional["DurationPredictor"]: ...
```

Type hints document what goes in and out. Python does **not** enforce them at run time; tools such as mypy and IDEs use them. `from __future__ import annotations` makes all hints lazy strings, so you can write `list[float]` or `date | None` on older Python versions and refer to classes defined later in the file.

`Optional[X]` means "X or None". `Union[A, B]` means "A or B". `Iterable[X]` means "anything you can loop over".

Dataclasses

```
@dataclass
class Task:
    title: str
    difficulty: int
    deadline: Optional[datetime] = None
```

`@dataclass` is a **decorator** (a function that takes a class and returns an improved class). It generates `__init__`, `__repr__` and `__eq__` from the annotated fields. `@dataclass(frozen=True)` (used for `SchedulerParams`) makes instances immutable, so a parameter set can't be changed by accident halfway through a run. `field(default_factory=list)` gives each instance its *own* new list. A plain `= []` default would be shared by all instances, a classic Python bug.

Context managers (`with`)

```
with conn:                      # sqlite3: COMMIT on success, ROLLBACK on exception
    conn.execute(...)
```

`with` guarantees clean-up code runs even if an exception happens. FLEXSched uses it for database transactions and pytest uses the same idea for `pytest.raises`.

Comprehensions

```
marks = ",".join("?" for _ in statuses)                    # generator expression
rows = [_row_to_task(r) for r in rows]                     # list comprehension
fatigue_by_id = {s.id: f for (s, _), f in zip(day_rows, fatigue)}   # dict comprehension
```

`zip(a, b)` walks two sequences in parallel; `(s, _)` **unpacks** a tuple and `_` means "I don't need this value".

Exceptions and custom exception classes

```
class EngineError(RuntimeError): ...
class EngineInputError(EngineError): ...      # a more specific kind of EngineError

raise EngineError("engine timed out") from exc   # keep the original cause attached
```

A hierarchy lets callers catch broadly (`except EngineError`) or precisely (`except EngineInputError`). `raise ... from exc` chains exceptions so the traceback shows the root cause.

`@property` , `@classmethod` , `@staticmethod`

- `@property` turns a method into a computed attribute: `session.actual_min` has no parentheses.
- `@classmethod` receives the class (`cls`) instead of an instance, and is used for alternative constructors: `DurationPredictor.load(path)` .

Standard library modules used

Module	Used for
`sqlite3`	the database driver (ships with Python)
`subprocess`	starting the C++ engine and piping JSON in/out
`json`	encoding requests, decoding responses, storing metrics
`argparse`	the command-line interface (sub-commands, typed options)
`datetime`	`datetime` , `date` , `time` , `timedelta` arithmetic
`pathlib`	`Path` objects instead of string paths (`path.parent.mkdir(...)`)
`math`	`exp` for the fatigue formulas
`tempfile`	the throw-away folder for `demo`
`shutil.which`	finding the engine on `PATH`

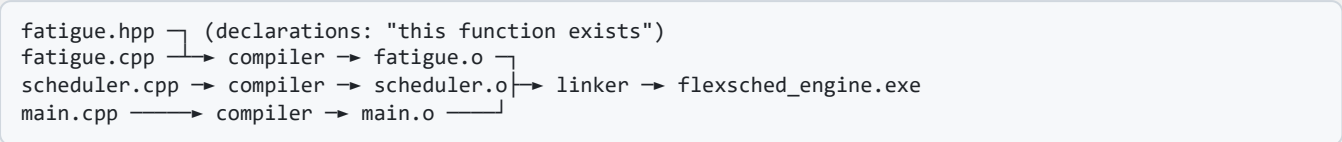
`*args` , `**kwargs` and keyword forwarding

```
def compare(conn, start, days=1, **kwargs):
    kwargs["persist"] = False
    return {s: plan(conn, start, days, strategy=s, **kwargs) for s in STRATEGIES}
```

`**kwargs` collects extra keyword arguments into a dict, and `**kwargs` in a call spreads them out again. That lets `compare` forward every option of `plan` without repeating them.

2.2 C++ primer (just what the engine needs)

How C++ code becomes a program



- **Header (`.hpp`)**: declarations, the *interface*. Other files `#include` it.
- **Source (`.cpp`)**: definitions, the *implementation*. Compiled once into an object file.
- **Linker**: stitches object files and libraries into an executable.
- `#pragma once` at the top of a header prevents it being included twice in one file.

Unlike Python, C++ is **compiled ahead of time** and **statically typed**: every type is known at compile time, so many mistakes are caught before the program runs and the machine code is fast.

Namespaces

```cpp
namespace flexsched {
double clamp01(double x);
}
flexsched::clamp01(1.7);   // or: using namespace flexsched; (only inside .cpp/tests)
```

Namespaces prevent name collisions between libraries (like Python modules do).

Fundamental types used

Type	Meaning
`int`	integer (≥ 32 bits)
`std::int64_t` (alias `Minutes`)	exactly 64-bit integer: minute offsets never overflow
`double`	64-bit floating point (fatigue, scores)
`bool`	true/false
`std::size_t`	unsigned integer used for sizes and indices
`std::string`	text
`std::vector<T>`	growable array, like a Python list of one type

`using Minutes = std::int64_t;` creates a **type alias**: the code reads like the domain.

`struct` vs `class`

Both define types. By convention: `struct` for plain data with public fields (`Interval` , `Task` , `ScheduledBlock`); `class` for objects with behaviour and private state (`FatigueModel` , `Scheduler`). Fields can have **default member initialisers**: `int priority = 3;` .

References, `const`, pointers

```
double after_rest(double fatigue, Minutes minutes) const;          // by value (cheap types)
ScheduleResult run(const std::vector<Task>& tasks, ...) const;      // by const reference
std::vector<const Task*> remaining;                                  // pointers to tasks
```

- `const std::vector<Task>&` means "a reference to the caller's vector, which I promise not to modify". No copy is made, which matters for big objects.
- A `const` **member function** (`... ) const;`) promises not to modify the object.
- `const Task*` is a **pointer** (an address) to a task that cannot be modified through it. The scheduler keeps pointers so it can reorder/erase cheaply without copying tasks.
- `*ptr` dereferences; `ptr->field` is shorthand for `(*ptr).field`.

RAII and `std::unique_ptr`

```
std::unique_ptr<SelectionPolicy> policy = std::make_unique<EdfPolicy>();
```

RAII (Resource Acquisition Is Initialisation): an object's destructor frees what it owns automatically when it goes out of scope. `std::unique_ptr` owns a heap object and deletes it automatically, so there is no `delete` and no leak. It is **move-only** (can't be copied; ownership is transferred with `std::move`), which is why `Scheduler` is also move-only.

Polymorphism: virtual functions

```
class SelectionPolicy {
public:
    virtual ~SelectionPolicy() = default;
    virtual std::size_t choose(...) const = 0;   // "= 0": pure virtual, must be overridden
};
class EdfPolicy final : public SelectionPolicy {
    std::size_t choose(...) const override;       // compiler checks it really overrides
};
```

Calling `policy_->choose(...)` through a base-class pointer runs the **derived** class's version, decided at run time (dynamic dispatch via a *vtable*). The base class needs a **virtual destructor** so deleting through a base pointer destroys the derived object correctly. `final` forbids further inheritance.

Lambdas

```
std::sort(v.begin(), v.end(), [&](std::size_t a, std::size_t b) { return scores[a] > scores[b]; });
```

A lambda is an inline anonymous function. `[&]` **captures** surrounding variables by reference. Python's `lambda` is the same idea, limited to one expression.

Algorithms from `<algorithm>`

- `std::sort(first, last, comp)` : O(n log n). **Requirement:** `comp` must be a *strict weak ordering* (irreflexive, transitive…). An epsilon-based "equal" breaks transitivity and is undefined behaviour; that's why FLEXSched compares scores exactly.
- `std::stable_sort` : keeps the original order of equal elements.
- `std::remove_if` + `erase` : the "erase–remove idiom" for deleting matching elements.

- `std::min`, `std::max`, `std::clamp(x, lo, hi)`, `std::any_of`, `std::adjacent_find`.

Exceptions

```
if (t.difficulty < 1 || t.difficulty > 5) throw std::invalid_argument("difficulty must be 1..5");
try { ... } catch (const std::invalid_argument& e) { std::cerr << e.what(); return 2; }
```

The engine validates input and throws `std::invalid_argument`; `main` catches it and turns it into exit code 2 plus a message on stderr, which Python turns into `EngineInputError`.

Other syntax you will meet

- `static` member function: belongs to the class, not an instance (`FatigueAwarePolicy::score`).
- `constexpr`: a value computed at compile time (`constexpr Minutes kDay = 24 * 60;`).
- `auto`: let the compiler infer the type (`const auto slots = free_slots(...)`).
- Range-for: `for (const Task& t : tasks)`.
- `static_cast<double>(x)`: explicit, checked type conversion.
- `1'000'000`: digit separators for readability.
- Anonymous namespace `namespace { ... }`: helpers visible only inside one `.cpp` file.
- `R"( ... )"`: raw string literal (no escaping), used for JSON in tests.

2.3 CMake (the C++ build system)

Compiling many files with the right flags, downloading dependencies and producing test executables by hand is error-prone. **CMake** describes the build once and generates build files for any toolchain (Ninja, Make, Visual Studio).

```
add_library(flexsched_core STATIC src/fatigue.cpp ...)        # a target: static library
target_include_directories(flexsched_core PUBLIC include)    # users of it see include/
target_link_libraries(flexsched_core PUBLIC nlohmann_json::nlohmann_json)
add_executable(flexsched_engine src/main.cpp)                 # the CLI program
target_link_libraries(flexsched_engine PRIVATE flexsched_core)
```

- **Targets** are things to build. Properties are attached *per target* ("modern CMake").
- `PUBLIC` = needed by me *and* by whoever links me; `PRIVATE` = only by me.
- **FetchContent** downloads a dependency at configure time and makes its targets available (Catch2, nlohmann/json), so no manual installs are needed.
- **CTest**: `enable_testing()` + `catch_discover_tests(flexsched_tests)` registers every Catch2 test case, so `ctest` runs them all and reports per case.
- Two-step workflow: `cmake -S engine -B build` (configure) → `cmake --build build` (compile).

Why split into `flexsched_core` (library) + `flexsched_engine` (exe) + `flexsched_tests` (exe)? The **same compiled logic** is linked into both the program and the tests, so the tests exercise exactly what ships.

2.4 SQLite and SQL

SQLite is a full SQL database engine in a library: the whole database is one file, there is no server, and Python ships the driver (`sqlite3`). It is ACID-compliant and is the most widely deployed database in the world (phones, browsers, apps).

Schema concepts used

Concept	In FLEXSched
`INTEGER PRIMARY KEY AUTOINCREMENT`	unique row id, never reused
`NOT NULL`	value required
`CHECK (difficulty BETWEEN 1 AND 5)`	the DB itself rejects invalid data
`DEFAULT 'pending'`, `DEFAULT (datetime('now'))`	automatic values
`REFERENCES tasks(id) ON DELETE CASCADE`	foreign key; deleting a task deletes its sessions
`ON DELETE SET NULL`	a schedule item survives task deletion with `task_id = NULL`
`CREATE INDEX idx_sessions_started ON work_sessions(started_at)`	B-tree index → fast range queries by date
`IF NOT EXISTS`	idempotent schema script: safe to run every start-up

Gotcha: SQLite has foreign keys **off by default**. `db.connect()` runs `PRAGMA foreign_keys = ON` on every connection, and a test proves it works.

Dates in SQLite: there's no DATE type; FLEXSched stores ISO-8601 text `'YYYY-MM-DD HH:MM:SS'`, which sorts correctly as text and works with `strftime()`.

Queries used

```sql
-- JOIN + feature extraction in SQL
SELECT t.category, t.difficulty, s.actual_min,
       CAST(strftime('%H', s.started_at) AS INTEGER) AS hour_of_day
FROM work_sessions s JOIN tasks t ON t.id = s.task_id
WHERE s.fatigue_at_start IS NOT NULL
ORDER BY s.started_at;

-- Aggregation
SELECT t.category, COUNT(*) AS sessions, AVG(1.0 * s.actual_min / t.estimate_min) AS avg_ratio
FROM work_sessions s JOIN tasks t ON t.id = s.task_id
GROUP BY t.category HAVING COUNT(*) >= 1 ORDER BY avg_ratio DESC;
```

- `JOIN ... ON` combines rows of two tables that match.
- `GROUP BY` makes one output row per group; `COUNT` / `AVG` aggregate within the group; `HAVING` filters *groups* (`WHERE` filters *rows*).
- `1.0 * a / b` forces floating-point division (integer / integer truncates in SQLite).
- `LEFT JOIN` (in `runs()`) keeps runs even when they have zero items.

Transactions and ACID

A **transaction** groups statements so they all happen or none do. Saving a schedule writes one `schedule_runs` row and N `schedule_items` rows; if item 7 fails, the run row must disappear too. `with conn:` gives exactly that, and a test forces a failure and checks the rollback.

ACID = **A**tomic (all or nothing), **C**onsistent (constraints hold), **I**solated (concurrent transactions don't see half-done work), **D**urable (committed data survives a crash).

Parameterised queries (SQL injection)

```
conn.execute("SELECT * FROM tasks WHERE id = ?", (task_id,))      # ✅ safe
conn.execute(f"SELECT * FROM tasks WHERE id = {task_id}")          # ❌ never do this
```

With `?` placeholders the driver sends data separately from SQL text, so input like `1; DROP TABLE tasks` is just a weird value, never code.

Schema versioning

`PRAGMA user_version` is an integer slot SQLite gives you for free. FLEXSched sets it to 1; a future migration would check `if current < 2: ALTER TABLE ...`.

2.5 Machine learning with scikit-learn

The task: supervised regression

- **Features (X)**: inputs: category, estimate, difficulty, priority, fatigue at start, hour of day, day of week.
- **Target (y)**: what we predict. Here it's the **ratio** `actual_min / estimate_min`, and the prediction in minutes is `ratio × estimate`.
- **Regression** = predicting a number (vs classification = predicting a label).

Train / validation / test and leakage

- Fit on **training** data, choose between models with **validation** (cross-validation), report once on an untouched **test (hold-out)** set.
- **Leakage** = information from the test set influencing training or model choice; results look great and then fail in reality.
- **Temporal data needs temporal splits.** We predict the *future* from the *past*, so FLEXSched holds out the most recent 20% of sessions, and cross-validation uses `TimeSeriesSplit` (each fold trains on earlier data and validates on later data). A random split would let the model "peek" at future habits.

```
TimeSeriesSplit(n_splits=5) on the training part:
fold 1: [train]               [val]
fold 2: [train  train]        [val]
fold 3: [train  train  train] [val]
...                                    (always: train before validation)
```

Baselines

A model is only interesting if it beats something simple:

- `trust_estimate`: ratio = 1, i.e. believe the user. (`DummyRegressor(strategy="constant", constant=1.0)`)
- `mean_ratio`: multiply every estimate by the average historical ratio. (`DummyRegressor(strategy="mean")`)

Preprocessing

- **One-hot encoding** turns `category="coding"` into columns `[coding=1, study=0, writing=0, ...]`. Models need numbers, and encoding categories as 1,2,3 would invent a fake order.

`handle_unknown="ignore"` keeps prediction working if a new category appears.

- **Standard scaling** (`(x - mean) / std`) puts numeric features on one scale. Ridge needs it because its penalty treats all coefficients equally; tree models don't.

`Pipeline` and `ColumnTransformer`

```
Pipeline([
    ("pre", ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ignore"), ["category"]),
                               ("num", StandardScaler(), NUMERIC)])),
    ("reg", Ridge(alpha=1.0)),
])
```

- `ColumnTransformer` applies different preprocessing to different columns.
- `Pipeline` chains preprocessing + model into **one object** with `fit` / `predict`.
- Why it matters: (1) preprocessing statistics (e.g. the scaler's mean) are learned **only from the training fold** inside cross-validation, which prevents leakage; (2) the saved model file contains everything needed to predict from raw columns.

The models

Model	Idea	Strengths / weaknesses
Ridge	linear regression + L2 penalty `α·Σw²` that shrinks coefficients	fast, stable, interpretable; only linear relationships
Random Forest	average of many decision trees, each trained on a bootstrap sample with random feature subsets (*bagging*)	captures non-linearity and interactions, robust; can't extrapolate, bigger
Gradient Boosting	trees added one at a time, each fitting the previous ensemble's errors (*boosting*), scaled by a learning rate	often the most accurate on tabular data; more hyper-parameters, can overfit

Why did **Ridge win here?** In ratio space the simulated habits are close to multiplicative factors that combine almost linearly, so a regularised linear model fits them with very little variance. The tree models are close behind (and have slightly higher R^2) but don't beat it on cross-validated MAE. "*I let validation decide rather than assuming the fanciest model wins*" is a good interview line.

Metrics

- **MAE**, mean absolute error, in minutes: "on average we're off by 6.7 minutes". Easy to explain and robust to outliers.
- **MAPE**, mean absolute percentage error: scale-free ("10% off").
- **R^2**: share of variance explained; 1.0 is perfect, 0 is as good as predicting the mean.

Permutation importance

Shuffle one feature column in the hold-out set and measure how much the error grows. A big increase means the model relies on that feature. It is model-agnostic and computed on unseen data. FLEXSched's report shows `category`, `difficulty` and `fatigue_at_start` as most important and `day_of_week` as ≈ 0, which matches the simulated habits (there is no weekday effect in the generator). That is a nice sanity check.

Persistence

`joblib.dump(obj, path)` / `joblib.load(path)` save and load Python objects (efficient for NumPy arrays). FLEXSched stores the pipeline together with metadata: model name, training time, report and **scikit-learn version** (models pickled with one version may not load in another).

2.6 Testing concepts

Kinds of tests here

Kind	Example
Unit (one function/class in isolation)	Catch2 `free_slots` cases; pytest `validate_task` cases
Edge-case	empty curve, zero-length intervals, task longer than every slot, out-of-range input
Invariant/property-style	"blocks never overlap and always lie inside free slots", checked on a varied 25-task workload for both strategies
Determinism	reversed input order gives an identical schedule
Integration	Python → real engine binary → Python, `plan()` end to end with SQLite
Cross-language parity	Python fatigue model replays the engine's output and must match to 1e-9
CLI / smoke	full `seed → train → add → plan → runs` workflow through `main()`

Catch2 (C++)

```
TEST_CASE("Late tasks are flagged and lateness is summed", "[scheduler]") {
    const auto r = make("edf").run(tasks, work_days(1));
    REQUIRE(r.blocks.size() == 1);     // stop this test case if false
    CHECK(r.blocks[0].late);           // record failure but keep going
    SECTION("...") { ... }             // each SECTION re-runs the test case from the top
}
```

- `TEST_CASE(name, tags)` : tags like `[edge]` let you run subsets: `flexsched_tests "[edge]"`.
- `REQUIRE` vs `CHECK` : stop vs continue on failure. Use `REQUIRE` when later lines would crash (e.g. indexing `blocks[0]`).
- `CHECK_THROWS_AS(expr, std::invalid_argument)` : asserts the right exception type.
- `CHECK_THAT(x, WithinAbs(expected, 1e-12))` : floating-point comparison with tolerance.
- Expression decomposition: on failure Catch2 prints both sides (`61 == 60`).

pytest (Python)

```
@pytest.fixture
def conn():
    connection = connect(":memory:")   # setup
    yield connection                   # test runs here
    connection.close()                 # teardown

@pytest.mark.parametrize("field,value", [("difficulty", 0), ("priority", 9)])
def test_validation_rejects_bad_tasks(conn, field, value): ...
```

- **Fixtures** provide fresh resources by argument name; `scope="module"` builds an expensive one (90 days of history) once per file.
- `parametrize` runs one test body for many inputs.
- Built-in fixtures: `tmp_path` (a temp folder), `capsys` (captures printed output).
- `pytest.raises(ValueError)` : asserts an exception.
- `skipif` marker: integration tests are skipped (not failed) when the engine isn't built.

Continuous Integration

`.github/workflows/ci.yml` runs on every push on **Ubuntu (GCC) and Windows (MSVC)**: configure → build → `ctest` → `pytest`. A green badge means the project builds and passes on two operating systems and two compilers, not just on my laptop.

Part 3 — C++ engine: code walkthrough

Folder layout:

```
engine/
  CMakeLists.txt
  include/flexsched/   types  fatigue  duration  calendar  policy  scheduler  json_io  (.hpp)
  src/                 fatigue  duration  calendar  policy  scheduler  json_io  main  (.cpp)
  tests/               test_*.cpp (Catch2, one file per module)
  examples/request.json
```

Who includes whom (no cycles; `types` is the foundation):

Module	Depends on
`types`	(only the standard library)
`fatigue`	`types`
`duration`	`types`, `fatigue` (for `clamp01`)
`calendar`	`types`
`policy`	`types`, `fatigue`
`scheduler`	`types`, `fatigue`, `duration`, `policy`
`json_io`	`types`, nlohmann/json
`main`	`calendar`, `policy`, `scheduler`, `json_io`

Reading order: `types` → `fatigue` → `duration` → `calendar` → `policy` → `scheduler` → `json_io` → `main` → `CMakeLists.txt` → tests.

3.1 `types.hpp`: the shared vocabulary

File: `engine/include/flexsched/types.hpp`

```cpp
// types.hpp - Plain data types shared by every engine module.
//
// Time convention: all times are integer MINUTES measured from the start of
// the planning horizon (minute 0 = first planned day at 00:00). Python owns
// the conversion between real datetimes and these offsets, so the engine
// never has to deal with time zones or calendars.
#pragma once

#include <cstdint>
#include <string>
#include <vector>

namespace flexsched {

using Minutes = std::int64_t;

// A half-open time interval [start, end).
struct Interval {
    Minutes start = 0;
    Minutes end = 0;

    Minutes length() const { return end - start; }
```

```
    bool empty() const { return end <= start; }
};

// A task as the engine sees it. The duration is not a single number but a
// curve: the predicted duration (minutes) at evenly spaced fatigue levels
// 0.0, 0.25, 0.5, 0.75, 1.0 (or any N >= 1 points spread evenly over [0, 1]).
struct Task {
    int id = 0;
    int difficulty = 3;           // 1 (trivial) .. 5 (very demanding)
    int priority = 3;             // 1 (low) .. 5 (critical)
    Minutes deadline = 0;         // minute offset; larger than horizon = "no deadline"
    std::vector<double> duration_curve;
};

// Tunable parameters. Defaults mirror flexsched/config.py on the Python side.
struct Params {
    // Fatigue dynamics (see fatigue.hpp for the formulas).
    double work_rate = 0.012;      // fatigue gained per minute at full load
    double recovery_rate = 0.05;   // fraction of fatigue lost per minute of rest

    // Break policy.
    double break_threshold = 0.65; // insert a break once fatigue reaches this
    Minutes break_minutes = 15;

    // Score weights for the fatigue-aware policy.
    double w_priority = 0.35;
    double w_urgency = 0.45;
    double w_energy = 0.20;
    double urgency_horizon = 480.0; // minutes of slack at which urgency ~ 0.37
};

enum class BlockKind { Task, Break };

// One entry of the final timetable.
struct ScheduledBlock {
    BlockKind kind = BlockKind::Task;
    int task_id = -1;             // -1 for breaks
    Minutes start = 0;
    Minutes end = 0;
    double fatigue_start = 0.0;
    double fatigue_end = 0.0;
    bool late = false;            // finished after its deadline
};

struct Unscheduled {
    int task_id = 0;
    std::string reason;
};

// Aggregate numbers used to compare strategies.
struct Metrics {
    Minutes total_work_minutes = 0;   // sum of task block lengths
    Minutes makespan = 0;             // end of the last block
    int tasks_scheduled = 0;
    int tasks_late = 0;
    Minutes total_lateness = 0;       // sum of (end - deadline) over late tasks
    int breaks = 0;
    double peak_fatigue = 0.0;
    double avg_fatigue_at_start = 0.0;
};

struct ScheduleResult {
    std::vector<ScheduledBlock> blocks;
    std::vector<Unscheduled> unscheduled;
    Metrics metrics;
};

}  // namespace flexsched
```

Notes

- **Time as integer minutes from the horizon start.** Integers make interval maths exact (no floating-point drift), and keeping calendars/time zones in Python keeps the engine small. `int64_t` can't realistically overflow.
- **Half-open intervals `[start, end)`.** A block ending at 600 and one starting at 600 touch but don't overlap. This convention removes off-by-one bugs (Python's `range` uses it too).
- `Task::duration_curve` is a `std::vector<double>`: the ML prediction at evenly spaced fatigue levels. The engine doesn't care how many points there are (≥ 1).
- `Params` holds every tunable number with a default; Python always sends its own values, so the defaults only matter for direct engine use and tests.
- `enum class BlockKind { Task, Break }`: a *scoped* enum. You must write `BlockKind::Task`, and it doesn't silently convert to `int`.
- `Metrics` is what the benchmark compares between strategies.

3.2 `fatigue.hpp` / `fatigue.cpp`: the fatigue model

File: `engine/include/flexsched/fatigue.hpp`

```
// fatigue.hpp - The fatigue model: how tiredness builds up and recovers.
//
// Fatigue F is a number in [0, 1]: 0 = fully fresh, 1 = exhausted.
//
//   Working m minutes on a task of difficulty d (load = d / 5):
//       F' = 1 - (1 - F) * exp(-work_rate * load * m)
//     -> saturating growth: F approaches 1 but never exceeds it, and the
//        same work tires a fresh person more (in absolute terms) than an
//        already-tired one. Harder tasks tire faster.
//
//   Resting m minutes (breaks, gaps between work windows, nights):
//       F' = F * exp(-recovery_rate * m)
//     -> exponential decay: a short break removes a big chunk of fatigue,
//        a long night removes almost all of it.
//
// The same two formulas are implemented in flexsched/fatigue.py so that
// Python can reconstruct historic fatigue levels; a test checks they agree.
#pragma once

#include "flexsched/types.hpp"

namespace flexsched {

class FatigueModel {
public:
    explicit FatigueModel(double work_rate = 0.012, double recovery_rate = 0.05);

    // Fatigue after working `minutes` on a task of the given difficulty (1..5).
    double after_work(double fatigue, int difficulty, Minutes minutes) const;

    // Fatigue after resting for `minutes`.
    double after_rest(double fatigue, Minutes minutes) const;

private:
    double work_rate_;
    double recovery_rate_;
};

// Clamp helper used across modules.
double clamp01(double x);

}  // namespace flexsched
```

File: `engine/src/fatigue.cpp`

```cpp
#include "flexsched/fatigue.hpp"

#include <algorithm>
#include <cmath>
#include <stdexcept>

namespace flexsched {

double clamp01(double x) { return std::min(1.0, std::max(0.0, x)); }

FatigueModel::FatigueModel(double work_rate, double recovery_rate)
    : work_rate_(work_rate), recovery_rate_(recovery_rate) {
    if (work_rate_ < 0.0 || recovery_rate_ < 0.0) {
        throw std::invalid_argument("fatigue rates must be non-negative");
    }
}

double FatigueModel::after_work(double fatigue, int difficulty, Minutes minutes) const {
    if (minutes <= 0) return clamp01(fatigue);
    const int d = std::clamp(difficulty, 1, 5);
    const double load = d / 5.0;
    const double decay = std::exp(-work_rate_ * load * static_cast<double>(minutes));
    return clamp01(1.0 - (1.0 - clamp01(fatigue)) * decay);
}

double FatigueModel::after_rest(double fatigue, Minutes minutes) const {
    if (minutes <= 0) return clamp01(fatigue);
    return clamp01(clamp01(fatigue) * std::exp(-recovery_rate_ * static_cast<double>(minutes)));
}

}  // namespace flexsched
```

Notes

- `explicit` on the constructor prevents accidental conversion from a `double` to a `FatigueModel`.
- `: work_rate_(work_rate), ...` is a **member initialiser list**: members are initialised directly rather than assigned afterwards. Trailing underscores mark private members (a common style).
- The constructor **validates** rates: a negative rate would make fatigue explode.
- `after_work` clamps difficulty into 1..5 and fatigue into [0, 1] *defensively*, so invalid inputs can't produce invalid outputs.
- Why these formulas? Work: `F' = 1 − (1 − F)·e^{−r·load·m}`. The "headroom" `1 − F` shrinks exponentially, so fatigue rises fast when fresh and saturates near 1, never exceeding it. Rest: `F' = F·e^{−k·m}` is exponential recovery (a 15-minute break at k = 0.05 removes 53% of fatigue; a 16-hour night removes essentially all of it). Both are standard first-order dynamics: simple, bounded, monotonic and explainable in one sentence each.

3.3 `duration.hpp` / `duration.cpp`: reading the duration curve

File: `engine/include/flexsched/duration.hpp`

```cpp
// duration.hpp - Turning a predicted duration curve into a concrete number.
//
// The ML model (Python) predicts how long a task takes at several fatigue
// levels. The scheduler only knows the fatigue at the moment it wants to
// start the task, so it linearly interpolates between the two nearest
// points of the curve and rounds UP to whole minutes (never under-book).
#pragma once

#include <vector>

#include "flexsched/types.hpp"
```

```cpp
namespace flexsched {

// Throws std::invalid_argument if the curve is empty or has a non-positive value.
Minutes effective_duration(const std::vector<double>& curve, double fatigue);

}  // namespace flexsched
```

File: `engine/src/duration.cpp`

```cpp
#include "flexsched/duration.hpp"

#include <cmath>
#include <stdexcept>

#include "flexsched/fatigue.hpp"

namespace flexsched {

Minutes effective_duration(const std::vector<double>& curve, double fatigue) {
    if (curve.empty()) {
        throw std::invalid_argument("duration curve must not be empty");
    }
    for (double v : curve) {
        if (!(v > 0.0)) {  // also rejects NaN
            throw std::invalid_argument("duration curve values must be positive");
        }
    }
    double value = curve.front();
    if (curve.size() > 1) {
        // Curve points sit at fatigue = i / (n - 1) for i = 0..n-1.
        const double f = clamp01(fatigue);
        const double pos = f * static_cast<double>(curve.size() - 1);
        const auto lo = static_cast<std::size_t>(std::floor(pos));
        const std::size_t hi = (lo + 1 < curve.size()) ? lo + 1 : lo;
        const double frac = pos - static_cast<double>(lo);
        value = curve[lo] + (curve[hi] - curve[lo]) * frac;
    }

    // Small epsilon so 60.0000000001 (floating point noise) does not become 61.
    return static_cast<Minutes>(std::ceil(value - 1e-9));
}

}  // namespace flexsched
```

Notes

- Curve points sit at fatigue `i/(n-1)`. For fatigue `f`: `pos = f·(n-1)`, `lo = ⌊pos⌋`, `frac = pos − lo`, `value = curve[lo] + (curve[hi] − curve[lo])·frac`. That's **linear interpolation**.
- Validation rejects empty curves and non-positive values. `!(v > 0.0)` also catches **NaN**, because every comparison with NaN is false.
- `std::ceil(value − 1e-9)`: always round **up** (never under-book time), but ignore floating-point noise like `60.000000000001`. A Catch2 test caught a bug where the single-point path skipped this epsilon; both paths now share one rounding line.

3.4 `calendar.hpp` / `calendar.cpp`: free-slot computation

File: `engine/include/flexsched/calendar.hpp`

```cpp
// calendar.hpp - Computing the free time we are allowed to schedule into.
//
// Input:  working windows (e.g. 09:00-17:00 each day) and blocked intervals
```

```
//         (meetings, lectures, gym ...), all as minute offsets.
// Output: a sorted list of non-overlapping FREE slots = windows minus blocked.
//
// This is the classic "interval subtraction" problem:
//   1. sort + merge the windows (they might overlap),
//   2. sort + merge the blocked intervals,
//   3. sweep through both lists once, cutting blocked pieces out.
// Complexity: O(W log W + B log B) for sorting, O(W + B) for the sweep.
#pragma once

#include <vector>

#include "flexsched/types.hpp"

namespace flexsched {

// Sort by start and merge overlapping or touching intervals. Empty
// (end <= start) intervals are dropped.
std::vector<Interval> merge_intervals(std::vector<Interval> intervals);

// Free slots = merged windows minus merged blocked intervals.
std::vector<Interval> free_slots(const std::vector<Interval>& windows,
                                 const std::vector<Interval>& blocked);

}  // namespace flexsched
```

File: `engine/src/calendar.cpp`

```
#include "flexsched/calendar.hpp"

#include <algorithm>

namespace flexsched {

std::vector<Interval> merge_intervals(std::vector<Interval> intervals) {
    // Drop empty intervals first so they can never "bridge" two real ones.
    intervals.erase(std::remove_if(intervals.begin(), intervals.end(),
                                   [](const Interval& iv) { return iv.empty(); }),
                    intervals.end());

    std::sort(intervals.begin(), intervals.end(),
              [](const Interval& a, const Interval& b) {
                  return a.start < b.start || (a.start == b.start && a.end < b.end);
              });

    std::vector<Interval> merged;
    for (const Interval& iv : intervals) {
        if (!merged.empty() && iv.start <= merged.back().end) {
            merged.back().end = std::max(merged.back().end, iv.end);  // overlap/touch
        } else {
            merged.push_back(iv);
        }
    }
    return merged;
}

std::vector<Interval> free_slots(const std::vector<Interval>& windows,
                                 const std::vector<Interval>& blocked) {
    const std::vector<Interval> w = merge_intervals(windows);
    const std::vector<Interval> b = merge_intervals(blocked);

    std::vector<Interval> result;
    std::size_t j = 0;  // index into blocked; only ever moves forward

    for (const Interval& window : w) {
        Minutes cursor = window.start;

        // Skip blocked intervals that end before this window starts.
        while (j < b.size() && b[j].end <= window.start) ++j;
```

```
        // Cut out every blocked interval that overlaps this window.
        std::size_t k = j;
        while (k < b.size() && b[k].start < window.end) {
            if (b[k].start > cursor) {
                result.push_back({cursor, b[k].start});
            }
            cursor = std::max(cursor, b[k].end);
            ++k;
        }
        if (cursor < window.end) {
            result.push_back({cursor, window.end});
        }
        // Do not advance j past k: a long blocked interval may span
        // into the next window as well.
    }
    return result;
}

}  // namespace flexsched
```

Notes

- `merge_intervals` takes its argument **by value** (`std::vector<Interval> intervals`) because it needs its own copy to sort anyway; callers keep their data untouched.
- Erase–remove idiom drops empty intervals first so a zero-length interval can't "bridge" two real ones.
- Merging after sorting: if the next interval starts at or before the current end (`<=`, so *touching* intervals merge too), extend; otherwise start a new one.
- `free_slots` is a **two-pointer sweep**: `j` only moves forward through the blocked list, so the sweep is linear after sorting. Subtle detail: `j` is *not* advanced past intervals that might continue into the next window (a block from 16:00 day 1 to 10:00 day 2 must cut both windows). There's a Catch2 test for exactly that.
- Complexity: O(W log W + B log B) to sort, O(W + B) to sweep.

3.5 `policy.hpp` / `policy.cpp`: the decision rules (Strategy pattern)

File: `engine/include/flexsched/policy.hpp`

```
// policy.hpp - "Which task should I do next?" as swappable strategies.
//
// The scheduler loop (scheduler.hpp) is the same for every strategy: walk
// forward through the free slots, track fatigue, and every time we are free
// ask a SelectionPolicy to pick one of the tasks that fits right now.
// This is the Strategy design pattern: the algorithm skeleton is fixed and
// the decision rule is a pluggable object behind a virtual interface.
#pragma once

#include <memory>
#include <string>
#include <vector>

#include "flexsched/types.hpp"

namespace flexsched {

// A task that fits in the current free time, together with how long it
// would take if started right now (given the current fatigue).
struct Candidate {
    const Task* task = nullptr;
    Minutes duration = 0;
};

// Everything a policy may look at when deciding.
```

```cpp
struct DecisionContext {
    Minutes now = 0;
    double fatigue = 0.0;
    const Params* params = nullptr;
    // Optional view of the free slots (current one first). When present,
    // lookahead simulations respect slot boundaries such as nights.
    const std::vector<Interval>* slots = nullptr;
    std::size_t slot_index = 0;
    // Optional: ALL unscheduled tasks (a superset of the candidates, which
    // only contain tasks that fit right now). Used by lookahead simulations.
    const std::vector<Candidate>* pending = nullptr;
};

class SelectionPolicy {
public:
    virtual ~SelectionPolicy() = default;

    virtual std::string name() const = 0;

    // Does the scheduler insert recovery breaks when fatigue gets high?
    virtual bool takes_breaks() const = 0;

    // Return the index (into `candidates`) of the task to schedule next.
    // Precondition: candidates is not empty.
    virtual std::size_t choose(const std::vector<Candidate>& candidates,
                               const DecisionContext& ctx) const = 0;
};

// Baseline: Earliest Deadline First, fatigue-blind (no breaks, no energy
// matching). EDF is the textbook optimal rule for minimising maximum
// lateness on a single machine when durations are fixed - a strong,
// well-known baseline to compare against.
class EdfPolicy final : public SelectionPolicy {
public:
    std::string name() const override { return "edf"; }
    bool takes_breaks() const override { return false; }
    std::size_t choose(const std::vector<Candidate>& candidates,
                       const DecisionContext& ctx) const override;
};

// FLEXSched's policy: weighted score of priority, deadline urgency and
// "energy match" (hard tasks when fresh, easy tasks when tired), plus breaks,
// plus a rollout deadline guard: a task may only be picked if
// "do it first, then the rest in EDF order" produces no more late tasks and
// no more lateness than plain EDF would from the same moment.
class FatigueAwarePolicy final : public SelectionPolicy {
public:
    std::string name() const override { return "fatigue_aware"; }
    bool takes_breaks() const override { return true; }
    std::size_t choose(const std::vector<Candidate>& candidates,
                       const DecisionContext& ctx) const override;

    // Exposed so unit tests can check the individual terms.
    static double priority_term(int priority);
    static double urgency_term(Minutes slack, double horizon);
    static double energy_term(int difficulty, double fatigue);
    static double score(const Candidate& c, const DecisionContext& ctx);
    // Candidate indices sorted best-first (ties: earlier deadline, lower id).
    static std::vector<std::size_t> rank_by_score(const std::vector<Candidate>& candidates,
                                                  const DecisionContext& ctx);
    static std::size_t best_by_score(const std::vector<Candidate>& candidates,
                                     const DecisionContext& ctx);

    struct Lateness {
        int late_tasks = 0;
        Minutes total_minutes = 0;
    };
    // Lateness of running candidates one after another from ctx.now: `first`
    // first (pass candidates.size() for "none"), the rest in deadline order.
    // Tasks are packed into the free slots from ctx (or back-to-back if none).
    static Lateness rollout_lateness(const std::vector<Candidate>& candidates,
                                     const DecisionContext& ctx, std::size_t first);
```

```
};

// Factory: "edf" or "fatigue_aware". Throws std::invalid_argument otherwise.
std::unique_ptr<SelectionPolicy> make_policy(const std::string& name);

}  // namespace flexsched
```

File: `engine/src/policy.cpp`

```
#include "flexsched/policy.hpp"

#include <algorithm>
#include <cmath>
#include <stdexcept>

#include "flexsched/fatigue.hpp"

namespace flexsched {

// ---------------------------------------------------------------- EDF ----

std::size_t EdfPolicy::choose(const std::vector<Candidate>& candidates,
                              const DecisionContext& /*ctx*/) const {
    std::size_t best = 0;
    for (std::size_t i = 1; i < candidates.size(); ++i) {
        const Task& a = *candidates[i].task;
        const Task& b = *candidates[best].task;
        // Earlier deadline wins; ties -> higher priority; ties -> lower id.
        if (a.deadline != b.deadline) {
            if (a.deadline < b.deadline) best = i;
        } else if (a.priority != b.priority) {
            if (a.priority > b.priority) best = i;
        } else if (a.id < b.id) {
            best = i;
        }
    }
    return best;
}

// ------------------------------------------------------- Fatigue-aware ----

double FatigueAwarePolicy::priority_term(int priority) {
    // Map 1..5 to 0..1.
    return (std::clamp(priority, 1, 5) - 1) / 4.0;
}

double FatigueAwarePolicy::urgency_term(Minutes slack, double horizon) {
    // slack = minutes left between finishing the task and its deadline.
    // No slack (or already late) -> maximum urgency 1.0.
    // Lots of slack -> urgency decays exponentially towards 0.
    if (slack <= 0) return 1.0;
    if (horizon <= 0.0) return 0.0;
    return std::exp(-static_cast<double>(slack) / horizon);
}

double FatigueAwarePolicy::energy_term(int difficulty, double fatigue) {
    // How well does the task's difficulty match the energy we have left?
    // energy = 1 - fatigue, difficulty normalised to 0..1.
    // Perfect match (hard task when fresh, easy task when tired) -> 1.0.
    const double diff = (std::clamp(difficulty, 1, 5) - 1) / 4.0;
    const double energy = 1.0 - clamp01(fatigue);
    return 1.0 - std::abs(diff - energy);
}

double FatigueAwarePolicy::score(const Candidate& c, const DecisionContext& ctx) {
    const Params& p = *ctx.params;
    const Minutes finish = ctx.now + c.duration;
    const Minutes slack = c.task->deadline - finish;
    return p.w_priority * priority_term(c.task->priority) +
           p.w_urgency * urgency_term(slack, p.urgency_horizon) +
```

```
           p.w_energy * energy_term(c.task->difficulty, ctx.fatigue);
}

std::size_t FatigueAwarePolicy::choose(const std::vector<Candidate>& candidates,
                                       const DecisionContext& ctx) const {
    const std::vector<std::size_t> ranked = rank_by_score(candidates, ctx);

    // Rollout deadline guard. Simulate over every unscheduled task if the
    // scheduler gave us the full list, otherwise over the candidates only.
    const std::vector<Candidate>& pool = ctx.pending ? *ctx.pending : candidates;
    auto index_in_pool = [&](std::size_t candidate_idx) {
        for (std::size_t j = 0; j < pool.size(); ++j) {
            if (pool[j].task == candidates[candidate_idx].task) return j;
        }
        return pool.size();
    };

    // What lateness would pure EDF give from here?
    const Lateness edf = rollout_lateness(pool, ctx, pool.size());
    // Take the highest-scored task whose "do it first, then EDF" rollout is
    // no worse than that.
    for (std::size_t idx : ranked) {
        const Lateness with_first = rollout_lateness(pool, ctx, index_in_pool(idx));
        if (with_first.late_tasks <= edf.late_tasks &&
            with_first.total_minutes <= edf.total_minutes) {
            return idx;
        }
    }
    // Nothing passed (possible when EDF's first task does not fit right now):
    // fall back to plain EDF among the candidates.
    return EdfPolicy{}.choose(candidates, ctx);
}

FatigueAwarePolicy::Lateness FatigueAwarePolicy::rollout_lateness(
    const std::vector<Candidate>& candidates, const DecisionContext& ctx, std::size_t first) {
    // Order: optional `first` candidate, then everything else by deadline (EDF).
    std::vector<std::size_t> order;
    order.reserve(candidates.size());
    for (std::size_t i = 0; i < candidates.size(); ++i) {
        if (i != first) order.push_back(i);
    }
    std::stable_sort(order.begin(), order.end(), [&](std::size_t a, std::size_t b) {
        return candidates[a].task->deadline < candidates[b].task->deadline;
    });
    if (first < candidates.size()) order.insert(order.begin(), first);

    // Cheap simulation at today's durations (approximation: it ignores future
    // fatigue changes and breaks). Tasks are packed into the free slots so a
    // night or a meeting in between is taken into account. O(k log k + S).
    Lateness result;
    Minutes t = ctx.now;
    std::size_t s = ctx.slot_index;
    for (std::size_t i : order) {
        const Minutes d = candidates[i].duration;
        if (ctx.slots != nullptr) {
            // Move to the first slot where the task fits; if none is left,
            // pretend it runs right after the last slot (it will be late).
            while (s < ctx.slots->size() &&
                   std::max(t, (*ctx.slots)[s].start) + d > (*ctx.slots)[s].end) {
                ++s;
            }
            if (s < ctx.slots->size()) t = std::max(t, (*ctx.slots)[s].start);
        }
        t += d;
        const Minutes over = t - candidates[i].task->deadline;
        if (over > 0) {
            ++result.late_tasks;
            result.total_minutes += over;
        }
    }
    return result;
}
```

```cpp
std::vector<std::size_t> FatigueAwarePolicy::rank_by_score(
    const std::vector<Candidate>& candidates, const DecisionContext& ctx) {
    // Score each candidate once, then sort indices: higher score first;
    // ties -> earlier deadline -> lower id (deterministic output).
    std::vector<double> scores(candidates.size());
    std::vector<std::size_t> ranked(candidates.size());
    for (std::size_t i = 0; i < candidates.size(); ++i) {
        scores[i] = score(candidates[i], ctx);
        ranked[i] = i;
    }
    // Exact comparisons only: std::sort requires a strict weak ordering, and an
    // epsilon-based "equal" would break transitivity (undefined behaviour).
    std::sort(ranked.begin(), ranked.end(), [&](std::size_t a, std::size_t b) {
        if (scores[a] != scores[b]) return scores[a] > scores[b];
        const Task& ta = *candidates[a].task;
        const Task& tb = *candidates[b].task;
        if (ta.deadline != tb.deadline) return ta.deadline < tb.deadline;
        return ta.id < tb.id;
    });
    return ranked;
}

std::size_t FatigueAwarePolicy::best_by_score(const std::vector<Candidate>& candidates,
                                              const DecisionContext& ctx) {
    return rank_by_score(candidates, ctx).front();
}

// ---------------------------------------------------------------- Factory ----

std::unique_ptr<SelectionPolicy> make_policy(const std::string& name) {
    if (name == "edf") return std::make_unique<EdfPolicy>();
    if (name == "fatigue_aware") return std::make_unique<FatigueAwarePolicy>();
    throw std::invalid_argument("unknown strategy: '" + name +
                                "' (expected 'edf' or 'fatigue_aware')");
}

}  // namespace flexsched
```

Notes

- `SelectionPolicy` is an **abstract base class** (it has pure virtual functions). The scheduler only knows this interface, so adding a new strategy means writing one new class and one line in `make_policy`; the scheduler never changes. That's the **Open/Closed principle**.
- `make_policy` is a **factory**: it maps a string from JSON to an object and returns `std::unique_ptr<SelectionPolicy>`, so ownership is clear and no manual `delete` exists.
- **EDF** (`EdfPolicy::choose`): linear scan for the earliest deadline, with deterministic tie-breaks (higher priority, then lower id). Determinism matters: same input → same schedule → reproducible tests and benchmarks.
- **Score terms** (`FatigueAwarePolicy`):
  - `priority_term`: `(p - 1)/4` maps 1..5 to 0..1;
  - `urgency_term`: `1` if no slack left, else `e^{-slack/horizon}`. It decays smoothly, and with horizon = 480 min a task due a full working day after it would finish has urgency ≈ 0.37;
  - `energy_term`: `1 - |difficulty_norm - (1 - F)|`. It's 1 when a hard task meets a fresh mind or an easy task meets a tired one.
  - `score = 0.35·priority + 0.45·urgency + 0.20·energy` (weights from `Params`).
- `rank_by_score` scores each candidate **once** (not inside the comparator) and sorts indices with an **exact** comparator. `std::sort` requires a strict weak ordering, and an epsilon comparison would violate transitivity (undefined behaviour).

- **Rollout deadline guard** (`choose`): 1. compute the lateness of "do everything in EDF order from now" (the reference); 2. walk candidates best-score-first; for each, compute the lateness of "do this one first, then EDF for the rest"; 3. return the first candidate that is **no worse** than the reference in both late tasks and total late minutes; 4. if none passes (possible when EDF's first task doesn't fit the current slot), fall back to EDF among the candidates.
- `rollout_lateness` packs tasks into the real free slots (`ctx.slots`), so a night or meeting between tasks is respected. It deliberately ignores future fatigue changes and breaks to stay cheap. That's a *heuristic* lookahead, not a proof. Part 5 tells the story of how this guard evolved, with numbers.
- `EdfPolicy{}.choose(...)` : `EdfPolicy{}` creates a temporary object; it has no state, so this is cheap.

3.6 `scheduler.hpp` / `scheduler.cpp` : the main loop

File: `engine/include/flexsched/scheduler.hpp`

```
// scheduler.hpp - The greedy, fatigue-simulating scheduling loop.
//
// Algorithm (event-driven simulation of one person's work days):
//
//   t = start of first free slot, F = 0 (fresh)
//   while tasks remain and free slots remain:
//       if t is before the current slot: rest until the slot starts
//       if the policy takes breaks and F >= threshold: insert a break
//       candidates = remaining tasks whose duration-at-F fits before slot end
//       if none fit: move on to the next slot
//       else: ask the policy to choose one, place it at t, update F and t
//   anything left over is reported as unscheduled, with a reason
//
// Greedy = we commit to each decision and never revisit it. This is fast
// (O(n^2) for n tasks) and easy to explain, but not guaranteed optimal; the
// general problem (deadlines + variable durations) is NP-hard.
#pragma once

#include <memory>
#include <vector>

#include "flexsched/fatigue.hpp"
#include "flexsched/policy.hpp"
#include "flexsched/types.hpp"

namespace flexsched {

class Scheduler {
public:
    Scheduler(Params params, std::unique_ptr<SelectionPolicy> policy);

    // `slots` must be sorted and non-overlapping (use free_slots()).
    // Throws std::invalid_argument on invalid tasks or parameters.
    ScheduleResult run(const std::vector<Task>& tasks,
                       const std::vector<Interval>& slots) const;

    const SelectionPolicy& policy() const { return *policy_; }

private:
    void validate(const std::vector<Task>& tasks) const;

    Params params_;
    std::unique_ptr<SelectionPolicy> policy_;
    FatigueModel fatigue_;
};

// Computes summary metrics from a finished list of blocks.
Metrics compute_metrics(const std::vector<ScheduledBlock>& blocks,
                        const std::vector<Task>& tasks);
```

```
}  // namespace flexsched
```

File: `engine/src/scheduler.cpp`

```
#include "flexsched/scheduler.hpp"

#include <algorithm>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <unordered_set>

#include "flexsched/duration.hpp"

namespace flexsched {

Scheduler::Scheduler(Params params, std::unique_ptr<SelectionPolicy> policy)
    : params_(params),
      policy_(std::move(policy)),
      fatigue_(params.work_rate, params.recovery_rate) {
    if (!policy_) throw std::invalid_argument("policy must not be null");
    if (!(params_.break_threshold > 0.0 && params_.break_threshold <= 1.0)) {
        throw std::invalid_argument("break_threshold must be in (0, 1]");
    }
    if (params_.break_minutes <= 0) {
        throw std::invalid_argument("break_minutes must be positive");
    }
}

void Scheduler::validate(const std::vector<Task>& tasks) const {
    std::unordered_set<int> seen;
    for (const Task& t : tasks) {
        const std::string where = "task " + std::to_string(t.id) + ": ";
        if (!seen.insert(t.id).second) throw std::invalid_argument(where + "duplicate id");
        if (t.difficulty < 1 || t.difficulty > 5)
            throw std::invalid_argument(where + "difficulty must be 1..5");
        if (t.priority < 1 || t.priority > 5)
            throw std::invalid_argument(where + "priority must be 1..5");
        effective_duration(t.duration_curve, 0.0);  // throws if the curve is invalid
    }
}

ScheduleResult Scheduler::run(const std::vector<Task>& tasks,
                              const std::vector<Interval>& slots) const {
    validate(tasks);

    ScheduleResult result;

    // Work on pointers so we never copy task data; sort by id so that the
    // output is deterministic regardless of input order.
    std::vector<const Task*> remaining;
    remaining.reserve(tasks.size());
    for (const Task& t : tasks) remaining.push_back(&t);
    std::sort(remaining.begin(), remaining.end(),
              [](const Task* a, const Task* b) { return a->id < b->id; });

    double fatigue = 0.0;
    std::size_t s = 0;
    Minutes now = slots.empty() ? 0 : slots.front().start;

    while (!remaining.empty() && s < slots.size()) {
        const Interval& slot = slots[s];

        // 1. Idle time before the slot (night, meeting, lunch) counts as rest.
        if (now < slot.start) {
            fatigue = fatigue_.after_rest(fatigue, slot.start - now);
            now = slot.start;
        }
        if (now >= slot.end) {  // slot fully consumed
```

```
            ++s;
            continue;
        }

        // 2. Recovery break if we are too tired (only for policies that use breaks).
        if (policy_->takes_breaks() && fatigue >= params_.break_threshold) {
            if (now + params_.break_minutes >= slot.end) {
                // The slot ends before a full break would: the idle gap after
                // the slot (evening, meeting) already provides the rest.
                ++s;
                continue;
            }
            const Minutes break_end = now + params_.break_minutes;
            ScheduledBlock brk;
            brk.kind = BlockKind::Break;
            brk.start = now;
            brk.end = break_end;
            brk.fatigue_start = fatigue;
            fatigue = fatigue_.after_rest(fatigue, break_end - now);
            brk.fatigue_end = fatigue;
            result.blocks.push_back(brk);
            now = break_end;
            continue;  // re-evaluate: slot may be over, or we may need another break
        }

        // 3. Which remaining tasks fit in what is left of this slot right now?
        std::vector<Candidate> candidates;
        std::vector<Candidate> pending;          // every remaining task (for lookahead)
        std::vector<std::size_t> candidate_pos;  // index into `remaining`
        for (std::size_t i = 0; i < remaining.size(); ++i) {
            const Minutes d = effective_duration(remaining[i]->duration_curve, fatigue);
            pending.push_back({remaining[i], d});
            if (now + d <= slot.end) {
                candidates.push_back({remaining[i], d});
                candidate_pos.push_back(i);
            }
        }
        if (candidates.empty()) {
            ++s;  // nothing fits here; the rest of this slot stays free
            continue;
        }

        // 4. Let the policy decide, then commit the decision.
        const DecisionContext ctx{now, fatigue, &params_, &slots, s, &pending};
        const std::size_t pick = policy_->choose(candidates, ctx);
        const Candidate& chosen = candidates.at(pick);

        ScheduledBlock block;
        block.kind = BlockKind::Task;
        block.task_id = chosen.task->id;
        block.start = now;
        block.end = now + chosen.duration;
        block.fatigue_start = fatigue;
        fatigue = fatigue_.after_work(fatigue, chosen.task->difficulty, chosen.duration);
        block.fatigue_end = fatigue;
        block.late = block.end > chosen.task->deadline;
        result.blocks.push_back(block);

        now = block.end;
        remaining.erase(remaining.begin() + static_cast<std::ptrdiff_t>(candidate_pos[pick]));
    }

    // 5. Explain why leftovers could not be placed.
    Minutes longest_slot = 0;
    for (const Interval& iv : slots) longest_slot = std::max(longest_slot, iv.length());
    for (const Task* t : remaining) {
        const Minutes fresh = effective_duration(t->duration_curve, 0.0);
        std::string reason = fresh > longest_slot
                                 ? "needs " + std::to_string(fresh) +
                                       " min but the longest free slot is " +
                                       std::to_string(longest_slot) + " min"
                                 : "not enough free time left in the planning horizon";
```

```
        result.unscheduled.push_back({t->id, reason});
    }

    result.metrics = compute_metrics(result.blocks, tasks);
    return result;
}

Metrics compute_metrics(const std::vector<ScheduledBlock>& blocks,
                        const std::vector<Task>& tasks) {
    std::unordered_map<int, const Task*> by_id;
    for (const Task& t : tasks) by_id[t.id] = &t;

    Metrics m;
    double fatigue_sum = 0.0;
    for (const ScheduledBlock& b : blocks) {
        m.makespan = std::max(m.makespan, b.end);
        if (b.kind == BlockKind::Break) {
            ++m.breaks;
            continue;
        }
        ++m.tasks_scheduled;
        m.total_work_minutes += b.end - b.start;
        fatigue_sum += b.fatigue_start;
        m.peak_fatigue = std::max(m.peak_fatigue, b.fatigue_end);
        if (b.late) {
            ++m.tasks_late;
            const auto it = by_id.find(b.task_id);
            if (it != by_id.end()) m.total_lateness += b.end - it->second->deadline;
        }
    }
    if (m.tasks_scheduled > 0) m.avg_fatigue_at_start = fatigue_sum / m.tasks_scheduled;
    return m;
}

}  // namespace flexsched
```

Notes, following `run()` step by step

- **Validation first** (`validate`): duplicate ids (detected with an `unordered_set` ; `insert(...).second` is false if already present), out-of-range difficulty/priority, invalid curves. Bad input fails loudly instead of producing a quietly wrong schedule.
- **Deterministic order**: tasks are referenced by pointer and sorted by id.
- **Step 1, rest before the slot**: any gap (night, meeting, lunch) reduces fatigue via `after_rest` .
- **Step 2, breaks**: only if the policy wants breaks and `F ≥ threshold` . If the slot would end before a full break, skip to the next slot instead, because the gap after the slot already provides rest. (A real bug found in the demo: pointless 3-minute "breaks" at 16:57; fixed and covered by a Catch2 test.) `continue` re-evaluates the loop, because one break might not be enough.
- **Step 3, candidates**: compute every remaining task's duration *at the current fatigue*; those that fit before the slot ends are candidates; all go into `pending` for the lookahead.
- **Step 4, decide and commit**: build a `DecisionContext` (now, fatigue, params, slots, current slot, pending), ask the policy, append the block, update fatigue, advance the clock, erase the task from `remaining` .
- **Termination**: every iteration either advances time, consumes a slot or schedules a task. Slots and tasks are finite, so the loop must end (and `break_minutes > 0` is validated so breaks always advance time).
- **Step 5, explanations**: leftovers get a human-readable reason: longer than any free slot, or the horizon ran out.
- `compute_metrics` is a free function so it can be tested and reused independently.
- **Complexity**: with n tasks and S slots, each decision computes n durations and the guard runs up to k rollouts of O(n log n + S). Worst case is roughly $O(n^2 \cdot (n \log n + S))$ per plan. For realistic n (≤ 100)

that's milliseconds.

3.7 `json_io.hpp` / `json_io.cpp`: the contract with Python

File: `engine/include/flexsched/json_io.hpp`

```
// json_io.hpp - The contract between Python and the C++ engine.
//
// Python writes ONE JSON request to the engine's stdin, the engine writes
// ONE JSON response to stdout. Keeping the boundary as plain JSON means:
//   * the engine can be tested on its own (pipe a file in, read the output),
//   * either side can be rewritten without touching the other,
//   * there is no fragile binary/ABI coupling between Python and C++.
//
// Request:
// {
//   "strategy": "fatigue_aware" | "edf",
//   "params":   { "work_rate": 0.012, ... }            (optional, all keys optional)
//   "windows":  [ {"start": 540, "end": 1020}, ... ]   (working hours)
//   "blocked":  [ {"start": 720, "end": 780}, ... ]    (optional)
//   "tasks":    [ {"id": 1, "difficulty": 4, "priority": 5,
//                  "deadline": 2000, "duration_curve": [60, 66, 72, 80, 90]} ]
// }
//
// Response: { "strategy", "blocks": [...], "unscheduled": [...], "metrics": {...} }
#pragma once

#include <string>
#include <vector>

#include <nlohmann/json.hpp>

#include "flexsched/types.hpp"

namespace flexsched {

struct Request {
    std::string strategy = "fatigue_aware";
    Params params;
    std::vector<Interval> windows;
    std::vector<Interval> blocked;
    std::vector<Task> tasks;
};

// Throws std::invalid_argument with a readable message on malformed input.
Request parse_request(const nlohmann::json& j);

nlohmann::json to_json(const ScheduleResult& result, const std::string& strategy);

}  // namespace flexsched
```

File: `engine/src/json_io.cpp`

```
#include "flexsched/json_io.hpp"

#include <stdexcept>

namespace flexsched {

using nlohmann::json;

namespace {

Interval parse_interval(const json& j, const char* field) {
    if (!j.is_object() || !j.contains("start") || !j.contains("end")) {
        throw std::invalid_argument(std::string(field) +
```

```
                                    ": each interval needs 'start' and 'end'");
    }
    Interval iv{j.at("start").get<Minutes>(), j.at("end").get<Minutes>()};
    if (iv.end < iv.start) {
        throw std::invalid_argument(std::string(field) + ": interval end before start");
    }
    return iv;
}

std::vector<Interval> parse_intervals(const json& j, const char* field) {
    std::vector<Interval> out;
    if (!j.contains(field)) return out;
    const json& arr = j.at(field);
    if (!arr.is_array()) throw std::invalid_argument(std::string(field) + " must be an array");
    for (const json& item : arr) out.push_back(parse_interval(item, field));
    return out;
}

Params parse_params(const json& j) {
    Params p;  // start from defaults, override whatever is provided
    if (!j.contains("params")) return p;
    const json& q = j.at("params");
    p.work_rate = q.value("work_rate", p.work_rate);
    p.recovery_rate = q.value("recovery_rate", p.recovery_rate);
    p.break_threshold = q.value("break_threshold", p.break_threshold);
    p.break_minutes = q.value("break_minutes", p.break_minutes);
    p.w_priority = q.value("w_priority", p.w_priority);
    p.w_urgency = q.value("w_urgency", p.w_urgency);
    p.w_energy = q.value("w_energy", p.w_energy);
    p.urgency_horizon = q.value("urgency_horizon", p.urgency_horizon);
    return p;
}

Task parse_task(const json& j) {
    for (const char* key : {"id", "difficulty", "priority", "deadline", "duration_curve"}) {
        if (!j.contains(key)) {
            throw std::invalid_argument(std::string("task is missing '") + key + "'");
        }
    }
    Task t;
    t.id = j.at("id").get<int>();
    t.difficulty = j.at("difficulty").get<int>();
    t.priority = j.at("priority").get<int>();
    t.deadline = j.at("deadline").get<Minutes>();
    t.duration_curve = j.at("duration_curve").get<std::vector<double>>();
    return t;
}

const char* kind_name(BlockKind k) { return k == BlockKind::Task ? "task" : "break"; }

}  // namespace

Request parse_request(const json& j) {
    if (!j.is_object()) throw std::invalid_argument("request must be a JSON object");
    Request r;
    r.strategy = j.value("strategy", r.strategy);
    r.params = parse_params(j);
    r.windows = parse_intervals(j, "windows");
    r.blocked = parse_intervals(j, "blocked");
    if (j.contains("tasks")) {
        if (!j.at("tasks").is_array()) throw std::invalid_argument("tasks must be an array");
        for (const json& item : j.at("tasks")) r.tasks.push_back(parse_task(item));
    }
    return r;
}

json to_json(const ScheduleResult& result, const std::string& strategy) {
    json blocks = json::array();
    for (const ScheduledBlock& b : result.blocks) {
        json jb = {{"kind", kind_name(b.kind)},
                   {"start", b.start},
                   {"end", b.end},
```

```
                    {"fatigue_start", b.fatigue_start},
                    {"fatigue_end", b.fatigue_end}};
        if (b.kind == BlockKind::Task) {
            jb["task_id"] = b.task_id;
            jb["late"] = b.late;
        }
        blocks.push_back(jb);
    }

    json unscheduled = json::array();
    for (const Unscheduled& u : result.unscheduled) {
        unscheduled.push_back({{"task_id", u.task_id}, {"reason", u.reason}});
    }

    const Metrics& m = result.metrics;
    json metrics = {{"total_work_minutes", m.total_work_minutes},
                    {"makespan", m.makespan},
                    {"tasks_scheduled", m.tasks_scheduled},
                    {"tasks_late", m.tasks_late},
                    {"total_lateness", m.total_lateness},
                    {"breaks", m.breaks},
                    {"peak_fatigue", m.peak_fatigue},
                    {"avg_fatigue_at_start", m.avg_fatigue_at_start}};

    return {{"strategy", strategy},
            {"blocks", blocks},
            {"unscheduled", unscheduled},
            {"metrics", metrics}};
}

}  // namespace flexsched
```

Notes

- Uses **nlohmann/json**, the de-facto standard header-only JSON library for C++. `j.at("key")` throws if missing; `j.value("key", default)` returns a default.
- `parse_params` starts from defaults and overrides only provided keys. This is forward-compatible: older clients still work when new parameters are added.
- Every malformed shape becomes `std::invalid_argument` with a readable message ("task is missing 'deadline'"), and tests cover those cases.
- `to_json` omits `task_id` / `late` for breaks, so the output is precisely what each block kind needs.

3.8 `main.cpp`: the executable

File: `engine/src/main.cpp`

```
// main.cpp - Command-line entry point of the scheduling engine.
//
// Usage:
//   flexsched_engine < request.json > response.json
//   flexsched_engine --version
//
// Exit codes: 0 = success, 2 = invalid input, 1 = unexpected internal error.
// Errors are written to stderr so stdout only ever contains valid JSON.
#include <iostream>
#include <iterator>
#include <stdexcept>
#include <string>

#include <nlohmann/json.hpp>

#include "flexsched/calendar.hpp"
#include "flexsched/json_io.hpp"
#include "flexsched/policy.hpp"
```

```cpp
#include "flexsched/scheduler.hpp"

namespace {
constexpr const char* kVersion = "1.0.0";
}

int main(int argc, char** argv) {
    if (argc > 1) {
        const std::string arg = argv[1];
        if (arg == "--version") {
            std::cout << "flexsched_engine " << kVersion << "\n";
            return 0;
        }
        std::cerr << "usage: flexsched_engine < request.json\n";
        return arg == "--help" ? 0 : 2;
    }

    try {
        const std::string input((std::istreambuf_iterator<char>(std::cin)),
                                std::istreambuf_iterator<char>());
        const nlohmann::json request_json = nlohmann::json::parse(input);
        const flexsched::Request req = flexsched::parse_request(request_json);

        flexsched::Scheduler scheduler(req.params, flexsched::make_policy(req.strategy));
        const auto slots = flexsched::free_slots(req.windows, req.blocked);
        const auto result = scheduler.run(req.tasks, slots);

        std::cout << flexsched::to_json(result, req.strategy).dump() << "\n";
        return 0;
    } catch (const nlohmann::json::exception& e) {
        std::cerr << "invalid JSON: " << e.what() << "\n";
        return 2;
    } catch (const std::invalid_argument& e) {
        std::cerr << "invalid request: " << e.what() << "\n";
        return 2;
    } catch (const std::exception& e) {
        std::cerr << "internal error: " << e.what() << "\n";
        return 1;
    }
}
```

Notes

- Reads **all of stdin** into a string with `std::istreambuf_iterator`, parses, runs, prints one JSON line.
- **Exit codes are part of the contract**: 0 = success, 2 = bad input, 1 = internal error. Errors go to **stderr** so stdout is always valid JSON or empty. `engine_bridge.py` maps code 2 to `EngineInputError`.
- `catch` order matters: the most specific exception types come first.

3.9 `CMakeLists.txt`

File: `engine/CMakeLists.txt`

```cmake
cmake_minimum_required(VERSION 3.16)
project(flexsched_engine VERSION 1.0.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
  set(CMAKE_BUILD_TYPE Release CACHE STRING "Build type" FORCE)
endif()

option(FLEXSCHED_BUILD_TESTS "Build the Catch2 unit tests" ON)
```

```
# ---- Third-party dependencies (downloaded at configure time) -------------
include(FetchContent)

FetchContent_Declare(
  nlohmann_json
  URL https://github.com/nlohmann/json/releases/download/v3.12.0/json.tar.xz
  DOWNLOAD_EXTRACT_TIMESTAMP TRUE)
FetchContent_MakeAvailable(nlohmann_json)

# ---- Core library: all scheduling logic, no I/O --------------------------
add_library(flexsched_core STATIC
  src/fatigue.cpp
  src/duration.cpp
  src/calendar.cpp
  src/policy.cpp
  src/scheduler.cpp
  src/json_io.cpp)
target_include_directories(flexsched_core PUBLIC include)
target_link_libraries(flexsched_core PUBLIC nlohmann_json::nlohmann_json)

if(MSVC)
  target_compile_options(flexsched_core PRIVATE /W4)
else()
  target_compile_options(flexsched_core PRIVATE -Wall -Wextra -Wpedantic)
endif()

# ---- Command-line executable used by Python ------------------------------
add_executable(flexsched_engine src/main.cpp)
target_link_libraries(flexsched_engine PRIVATE flexsched_core)
if(MINGW)
  # Bundle the C++ runtime so the .exe runs without MinGW on PATH.
  target_link_options(flexsched_engine PRIVATE -static)
endif()

# ---- Unit tests ----------------------------------------------------------
if(FLEXSCHED_BUILD_TESTS)
  FetchContent_Declare(
    Catch2
    URL https://github.com/catchorg/Catch2/archive/refs/tags/v3.7.1.tar.gz
    DOWNLOAD_EXTRACT_TIMESTAMP TRUE)
  FetchContent_MakeAvailable(Catch2)

  add_executable(flexsched_tests
    tests/test_fatigue.cpp
    tests/test_duration.cpp
    tests/test_calendar.cpp
    tests/test_policy.cpp
    tests/test_scheduler.cpp
    tests/test_json_io.cpp)
  target_link_libraries(flexsched_tests PRIVATE flexsched_core Catch2::Catch2WithMain)
  if(MINGW)
    target_link_options(flexsched_tests PRIVATE -static)
  endif()

  enable_testing()
  list(APPEND CMAKE_MODULE_PATH ${catch2_SOURCE_DIR}/extras)
  include(Catch)
  catch_discover_tests(flexsched_tests)
endif()
```

Notes

- C++17 required, no compiler extensions (portable code).
- Release is the default build type (optimised).
- Warnings on (`-Wall -Wextra -Wpedantic` / `/W4`). The project compiles with **zero warnings** on GCC 16.
- `-static` on MinGW bundles the C++ runtime into the `.exe`, so it runs on Windows without MinGW installed.
- `option(FLEXSCHED_BUILD_TESTS ...)` lets packagers skip downloading Catch2.

3.10 Catch2 test files

Each module has its own test file. Read them as **executable documentation**: every `TEST_CASE` name is a sentence describing a behaviour.

`test_fatigue.cpp`

File: `engine/tests/test_fatigue.cpp`

```cpp
#include <cmath>
#include <stdexcept>

#include <catch2/catch_test_macros.hpp>
#include <catch2/matchers/catch_matchers_floating_point.hpp>

#include "flexsched/fatigue.hpp"

using flexsched::FatigueModel;
using Catch::Matchers::WithinAbs;

TEST_CASE("Zero or negative minutes leave fatigue unchanged", "[fatigue]") {
    FatigueModel m;
    CHECK(m.after_work(0.3, 3, 0) == 0.3);
    CHECK(m.after_work(0.3, 3, -10) == 0.3);
    CHECK(m.after_rest(0.3, 0) == 0.3);
}

TEST_CASE("Work follows the saturating formula", "[fatigue]") {
    FatigueModel m(0.012, 0.05);
    // 60 minutes at difficulty 5 (load 1.0) from fresh: 1 - exp(-0.72)
    CHECK_THAT(m.after_work(0.0, 5, 60), WithinAbs(1.0 - std::exp(-0.72), 1e-12));
    // From F = 0.5 the remaining "headroom" (0.5) shrinks by the same factor.
    CHECK_THAT(m.after_work(0.5, 5, 60), WithinAbs(1.0 - 0.5 * std::exp(-0.72), 1e-12));
}

TEST_CASE("Harder tasks tire faster than easy ones", "[fatigue]") {
    FatigueModel m;
    const double easy = m.after_work(0.0, 1, 90);
    const double hard = m.after_work(0.0, 5, 90);
    CHECK(hard > easy);
    CHECK(easy > 0.0);
}

TEST_CASE("Fatigue never leaves [0, 1]", "[fatigue][edge]") {
    FatigueModel m;
    CHECK(m.after_work(0.0, 5, 100000) <= 1.0);
    CHECK(m.after_work(0.99, 5, 100000) <= 1.0);
    CHECK(m.after_rest(1.0, 100000) >= 0.0);
    // Out-of-range inputs are clamped rather than propagated.
    CHECK(m.after_work(1.7, 3, 10) <= 1.0);
    CHECK(m.after_rest(-0.4, 10) == 0.0);
    // Out-of-range difficulty is clamped to 1..5.
    CHECK(m.after_work(0.0, 99, 30) == m.after_work(0.0, 5, 30));
    CHECK(m.after_work(0.0, -3, 30) == m.after_work(0.0, 1, 30));
}

TEST_CASE("Rest decays fatigue exponentially", "[fatigue]") {
    FatigueModel m(0.012, 0.05);
    CHECK_THAT(m.after_rest(0.8, 15), WithinAbs(0.8 * std::exp(-0.75), 1e-12));
    // A night (~16h) wipes out practically all fatigue.
    CHECK(m.after_rest(1.0, 16 * 60) < 1e-6);
    // More rest -> less fatigue (monotonic).
    CHECK(m.after_rest(0.8, 30) < m.after_rest(0.8, 10));
}

TEST_CASE("Negative rates are rejected", "[fatigue][edge]") {
    CHECK_THROWS_AS(FatigueModel(-0.1, 0.05), std::invalid_argument);
```

```
    CHECK_THROWS_AS(FatigueModel(0.01, -0.05), std::invalid_argument);
}
```

Covers: no-op for zero/negative minutes, exact formula values, harder-is-more-tiring, bounds `[0,1]` under extreme inputs, clamping of out-of-range inputs, exponential rest and monotonicity, rejection of negative rates.

`test_duration.cpp`

File: `engine/tests/test_duration.cpp`

```cpp
#include <cmath>
#include <limits>
#include <stdexcept>

#include <catch2/catch_test_macros.hpp>

#include "flexsched/duration.hpp"

using flexsched::effective_duration;

TEST_CASE("Curve endpoints are returned exactly", "[duration]") {
    const std::vector<double> curve{60, 70, 80, 90, 100};
    CHECK(effective_duration(curve, 0.0) == 60);
    CHECK(effective_duration(curve, 1.0) == 100);
    CHECK(effective_duration(curve, 0.5) == 80);
}

TEST_CASE("Values between points are linearly interpolated", "[duration]") {
    const std::vector<double> curve{60, 70, 80, 90, 100};
    // 0.125 is half-way between the points at 0.0 and 0.25.
    CHECK(effective_duration(curve, 0.125) == 65);
    CHECK(effective_duration(curve, 0.875) == 95);
}

TEST_CASE("Durations are rounded up, never down", "[duration]") {
    CHECK(effective_duration({60.2}, 0.3) == 61);
    CHECK(effective_duration({59.0001, 59.0001}, 0.5) == 60);
    // ...but floating point noise just above an integer is ignored.
    CHECK(effective_duration({60.0 + 1e-12}, 0.0) == 60);
}

TEST_CASE("A single-point curve ignores fatigue", "[duration][edge]") {
    CHECK(effective_duration({45}, 0.0) == 45);
    CHECK(effective_duration({45}, 1.0) == 45);
}

TEST_CASE("Fatigue outside [0, 1] is clamped", "[duration][edge]") {
    const std::vector<double> curve{60, 100};
    CHECK(effective_duration(curve, -5.0) == 60);
    CHECK(effective_duration(curve, 7.0) == 100);
}

TEST_CASE("Invalid curves are rejected", "[duration][edge]") {
    CHECK_THROWS_AS(effective_duration({}, 0.0), std::invalid_argument);
    CHECK_THROWS_AS(effective_duration({30, 0}, 0.0), std::invalid_argument);
    CHECK_THROWS_AS(effective_duration({30, -5}, 0.0), std::invalid_argument);
    CHECK_THROWS_AS(effective_duration({std::numeric_limits<double>::quiet_NaN()}, 0.0),
                    std::invalid_argument);
}
```

Covers: endpoints, interpolation midpoints, round-up rule and epsilon, single-point curves, clamping, rejection of empty/zero/negative/NaN curves.

`test_calendar.cpp`

File: `engine/tests/test_calendar.cpp`

```cpp
#include <catch2/catch_test_macros.hpp>

#include "flexsched/calendar.hpp"

using flexsched::free_slots;
using flexsched::Interval;
using flexsched::merge_intervals;

namespace {
bool same(const std::vector<Interval>& a, const std::vector<Interval>& b) {
    if (a.size() != b.size()) return false;
    for (std::size_t i = 0; i < a.size(); ++i) {
        if (a[i].start != b[i].start || a[i].end != b[i].end) return false;
    }
    return true;
}
}  // namespace

TEST_CASE("merge_intervals sorts and merges", "[calendar]") {
    SECTION("overlapping intervals merge") {
        CHECK(same(merge_intervals({{0, 10}, {5, 20}}), {{0, 20}}));
    }
    SECTION("touching intervals merge") {
        CHECK(same(merge_intervals({{0, 10}, {10, 20}}), {{0, 20}}));
    }
    SECTION("unsorted input is sorted") {
        CHECK(same(merge_intervals({{30, 40}, {0, 10}}), {{0, 10}, {30, 40}}));
    }
    SECTION("contained interval disappears") {
        CHECK(same(merge_intervals({{0, 100}, {20, 30}}), {{0, 100}}));
    }
    SECTION("empty intervals are dropped") {
        CHECK(same(merge_intervals({{5, 5}, {9, 3}}), {}));
        CHECK(merge_intervals({}).empty());
    }
}

TEST_CASE("free_slots subtracts blocked time from windows", "[calendar]") {
    const std::vector<Interval> day{{540, 1020}};  // 09:00-17:00

    SECTION("nothing blocked -> whole window") {
        CHECK(same(free_slots(day, {}), {{540, 1020}}));
    }
    SECTION("meeting in the middle splits the window") {
        CHECK(same(free_slots(day, {{720, 780}}), {{540, 720}, {780, 1020}}));
    }
    SECTION("block at the start or end trims the window") {
        CHECK(same(free_slots(day, {{500, 600}}), {{600, 1020}}));
        CHECK(same(free_slots(day, {{1000, 1100}}), {{540, 1000}}));
    }
    SECTION("block covering the whole window leaves nothing") {
        CHECK(free_slots(day, {{0, 2000}}).empty());
    }
    SECTION("blocks outside the window are ignored") {
        CHECK(same(free_slots(day, {{0, 100}, {1200, 1300}}), {{540, 1020}}));
    }
    SECTION("overlapping blocks are merged first") {
        CHECK(same(free_slots(day, {{600, 700}, {650, 760}}), {{540, 600}, {760, 1020}}));
    }
    SECTION("no windows -> no free time") {
        CHECK(free_slots({}, {{0, 10}}).empty());
    }
}

TEST_CASE("A blocked interval may span two windows", "[calendar][edge]") {
    // Two days 09:00-17:00; something blocks from 16:00 day 1 to 10:00 day 2.
```

```
    const std::vector<Interval> windows{{540, 1020}, {1980, 2460}};
    const std::vector<Interval> blocked{{960, 2040}};
    CHECK(same(free_slots(windows, blocked), {{540, 960}, {2040, 2460}}));
}
```

Covers: overlap/touch/unsorted/contained/empty merging; free slots with nothing blocked, middle split, edge trims, full cover, outside blocks, overlapping blocks, no windows, and the tricky **multi-window block**.

`test_policy.cpp`

File: `engine/tests/test_policy.cpp`

```
#include <stdexcept>

#include <catch2/catch_test_macros.hpp>
#include <catch2/matchers/catch_matchers_floating_point.hpp>

#include "flexsched/policy.hpp"

using namespace flexsched;
using Catch::Matchers::WithinAbs;

namespace {
Task make_task(int id, int difficulty, int priority, Minutes deadline) {
    Task t;
    t.id = id;
    t.difficulty = difficulty;
    t.priority = priority;
    t.deadline = deadline;
    t.duration_curve = {60};
    return t;
}
}  // namespace

TEST_CASE("Score terms behave as documented", "[policy]") {
    SECTION("priority maps 1..5 onto 0..1") {
        CHECK(FatigueAwarePolicy::priority_term(1) == 0.0);
        CHECK(FatigueAwarePolicy::priority_term(3) == 0.5);
        CHECK(FatigueAwarePolicy::priority_term(5) == 1.0);
    }
    SECTION("urgency is 1 with no slack and decays with more slack") {
        CHECK(FatigueAwarePolicy::urgency_term(0, 480) == 1.0);
        CHECK(FatigueAwarePolicy::urgency_term(-30, 480) == 1.0);
        CHECK_THAT(FatigueAwarePolicy::urgency_term(480, 480), WithinAbs(0.367879, 1e-6));
        CHECK(FatigueAwarePolicy::urgency_term(100, 480) >
              FatigueAwarePolicy::urgency_term(1000, 480));
    }
    SECTION("energy match rewards hard-when-fresh and easy-when-tired") {
        CHECK(FatigueAwarePolicy::energy_term(5, 0.0) == 1.0);
        CHECK(FatigueAwarePolicy::energy_term(1, 1.0) == 1.0);
        CHECK(FatigueAwarePolicy::energy_term(1, 0.0) == 0.0);
        CHECK(FatigueAwarePolicy::energy_term(5, 1.0) == 0.0);
    }
}

TEST_CASE("EDF picks the earliest deadline with deterministic ties", "[policy]") {
    EdfPolicy edf;
    Params p;
    const DecisionContext ctx{0, 0.0, &p};
    const Task a = make_task(1, 3, 1, 500);
    const Task b = make_task(2, 3, 5, 300);
    const Task c = make_task(3, 3, 5, 300);

    CHECK(edf.choose({{&a, 60}, {&b, 60}}, ctx) == 1);          // earlier deadline
    CHECK(edf.choose({{&c, 60}, {&b, 60}}, ctx) == 1);          // tie -> lower id (b)
    const Task d = make_task(4, 3, 2, 300);
    CHECK(edf.choose({{&d, 60}, {&c, 60}}, ctx) == 1);          // tie -> higher priority
    CHECK_FALSE(edf.takes_breaks());
```

```
}

TEST_CASE("Fatigue-aware policy matches task difficulty to energy", "[policy]") {
    FatigueAwarePolicy pol;
    Params p;
    const Task hard = make_task(1, 5, 3, 100000);
    const Task easy = make_task(2, 1, 3, 100000);
    const std::vector<Candidate> cands{{&hard, 60}, {&easy, 60}};

    const DecisionContext fresh{0, 0.0, &p};
    const DecisionContext tired{0, 0.9, &p};
    CHECK(pol.choose(cands, fresh) == 0);  // fresh -> hard task
    CHECK(pol.choose(cands, tired) == 1);  // tired -> easy task
    CHECK(pol.takes_breaks());
}

TEST_CASE("Urgency outweighs energy matching", "[policy]") {
    FatigueAwarePolicy pol;
    Params p;
    // We are fresh, so the hard task is the better energy match,
    // but the easy task is due right now.
    const Task hard_later = make_task(1, 5, 3, 100000);
    const Task easy_due = make_task(2, 1, 3, 60);
    const DecisionContext fresh{0, 0.0, &p};
    CHECK(pol.choose({{&hard_later, 60}, {&easy_due, 60}}, fresh) == 1);
}

TEST_CASE("Deadline guard protects the most urgent task", "[policy]") {
    FatigueAwarePolicy pol;
    Params p;
    const DecisionContext fresh{0, 0.0, &p};
    // The hard, high-priority task scores higher (0.55 vs ~0.41)...
    const Task hard = make_task(1, 5, 5, 100000);
    const Task easy_due_100 = make_task(2, 1, 1, 100);
    REQUIRE(FatigueAwarePolicy::best_by_score({{&hard, 60}, {&easy_due_100, 60}}, fresh) == 0);
    // ...but doing it first (0-60) would push the easy task (60-120) past
    // its deadline of 100, while starting the easy task now is on time.
    CHECK(pol.choose({{&hard, 60}, {&easy_due_100, 60}}, fresh) == 1);

    SECTION("no override when postponing is still on time") {
        const Task easy_due_130 = make_task(2, 1, 1, 130);
        CHECK(pol.choose({{&hard, 60}, {&easy_due_130, 60}}, fresh) == 0);
    }
    SECTION("an already-late task is still not postponed further") {
        // Due at 30 but takes 45: 15 min late if started now, 75 min late if
        // postponed. The guard minimises total lateness, so it goes first.
        const Task easy_due_30 = make_task(2, 1, 1, 30);
        CHECK(pol.choose({{&hard, 60}, {&easy_due_30, 45}}, fresh) == 1);
    }
}

TEST_CASE("Rollout lateness simulates back-to-back execution", "[policy]") {
    const Task a = make_task(1, 3, 3, 50);   // takes 60 -> 10 late if first
    const Task b = make_task(2, 3, 3, 200);  // takes 60
    const std::vector<Candidate> c{{&a, 60}, {&b, 60}};

    Params p;
    const DecisionContext ctx{0, 0.0, &p};

    const auto edf = FatigueAwarePolicy::rollout_lateness(c, ctx, c.size());
    CHECK(edf.late_tasks == 1);
    CHECK(edf.total_minutes == 10);

    const auto b_first = FatigueAwarePolicy::rollout_lateness(c, ctx, 1);
    CHECK(b_first.late_tasks == 1);
    CHECK(b_first.total_minutes == 70);  // a finishes at 120, deadline 50

    SECTION("slot boundaries push work to the next slot") {
        // Free 0-90 and 300-400: the second task cannot fit before 90,
        // so it runs 300-360 and misses its deadline of 200 by 160.
        const std::vector<Interval> slots{{0, 90}, {300, 400}};
        const DecisionContext with_slots{0, 0.0, &p, &slots, 0};
```

```
        const Task early = make_task(3, 3, 3, 100);
        const std::vector<Candidate> c2{{&early, 60}, {&b, 60}};
        const auto r = FatigueAwarePolicy::rollout_lateness(c2, with_slots, c2.size());
        CHECK(r.late_tasks == 1);
        CHECK(r.total_minutes == 160);
    }
}

TEST_CASE("make_policy builds known strategies and rejects others", "[policy][edge]") {
    CHECK(make_policy("edf")->name() == "edf");
    CHECK(make_policy("fatigue_aware")->name() == "fatigue_aware");
    CHECK_THROWS_AS(make_policy("random"), std::invalid_argument);
    CHECK_THROWS_AS(make_policy(""), std::invalid_argument);
}
```

Covers: each score term, EDF tie-breaking, energy matching when fresh vs tired, urgency beating energy, the deadline guard (override, no override, already-late task), rollout simulation including **slot boundaries**, factory errors.

`test_scheduler.cpp`

File: `engine/tests/test_scheduler.cpp`

```
#include <algorithm>
#include <stdexcept>

#include <catch2/catch_test_macros.hpp>

#include "flexsched/calendar.hpp"
#include "flexsched/scheduler.hpp"

using namespace flexsched;

namespace {

constexpr Minutes kDay = 24 * 60;
constexpr Minutes kNoDeadline = 1'000'000;

Task make_task(int id, int difficulty, int priority, Minutes deadline,
               std::vector<double> curve) {
    Task t;
    t.id = id;
    t.difficulty = difficulty;
    t.priority = priority;
    t.deadline = deadline;
    t.duration_curve = std::move(curve);
    return t;
}

// Working hours 09:00-17:00 on `days` consecutive days.
std::vector<Interval> work_days(int days) {
    std::vector<Interval> w;
    for (int d = 0; d < days; ++d) w.push_back({d * kDay + 540, d * kDay + 1020});
    return w;
}

Scheduler make(const std::string& strategy, Params p = {}) {
    return Scheduler(p, make_policy(strategy));
}

}  // namespace

TEST_CASE("No tasks -> empty schedule", "[scheduler][edge]") {
    const auto r = make("fatigue_aware").run({}, work_days(1));
    CHECK(r.blocks.empty());
    CHECK(r.unscheduled.empty());
    CHECK(r.metrics.tasks_scheduled == 0);
}
```

```
TEST_CASE("No free time -> everything unscheduled", "[scheduler][edge]") {
    const std::vector<Task> tasks{make_task(1, 3, 3, kNoDeadline, {30})};
    const auto r = make("fatigue_aware").run(tasks, {});
    CHECK(r.blocks.empty());
    REQUIRE(r.unscheduled.size() == 1);
    CHECK(r.unscheduled[0].task_id == 1);
}

TEST_CASE("A single task starts at the beginning of the first slot", "[scheduler]") {
    const std::vector<Task> tasks{make_task(7, 3, 3, kNoDeadline, {45})};
    const auto r = make("fatigue_aware").run(tasks, work_days(1));
    REQUIRE(r.blocks.size() == 1);
    CHECK(r.blocks[0].task_id == 7);
    CHECK(r.blocks[0].start == 540);
    CHECK(r.blocks[0].end == 585);
    CHECK(r.blocks[0].fatigue_start == 0.0);
    CHECK(r.blocks[0].fatigue_end > 0.0);
    CHECK_FALSE(r.blocks[0].late);
}

TEST_CASE("Blocks never overlap and always stay inside free slots", "[scheduler][invariant]") {
    std::vector<Task> tasks;
    for (int i = 1; i <= 25; ++i) {
        const double base = 20.0 + (i * 37) % 100;  // varied durations 20..119
        tasks.push_back(make_task(i, 1 + i % 5, 1 + (i * 3) % 5, (i % 4 + 1) * kDay,
                                  {base, base * 1.1, base * 1.2, base * 1.35, base * 1.5}));
    }
    const auto slots = free_slots(work_days(3), {{720, 780}, {kDay + 600, kDay + 660}});

    for (const std::string strategy : {"edf", "fatigue_aware"}) {
        const auto r = make(strategy).run(tasks, slots);
        INFO("strategy = " << strategy);

        // Every task is either scheduled once or reported as unscheduled.
        std::vector<int> seen;
        for (const auto& b : r.blocks)
            if (b.kind == BlockKind::Task) seen.push_back(b.task_id);
        for (const auto& u : r.unscheduled) seen.push_back(u.task_id);
        std::sort(seen.begin(), seen.end());
        REQUIRE(seen.size() == tasks.size());
        CHECK(std::adjacent_find(seen.begin(), seen.end()) == seen.end());

        for (std::size_t i = 0; i < r.blocks.size(); ++i) {
            const auto& b = r.blocks[i];
            CHECK(b.end > b.start);
            if (i > 0) CHECK(b.start >= r.blocks[i - 1].end);  // no overlap, ordered
            const bool inside = std::any_of(slots.begin(), slots.end(), [&](const Interval& s) {
                return b.start >= s.start && b.end <= s.end;
            });
            CHECK(inside);
            CHECK(b.fatigue_start >= 0.0);
            CHECK(b.fatigue_end <= 1.0);
        }
    }
}

TEST_CASE("Task longer than every slot is reported with a reason", "[scheduler][edge]") {
    const std::vector<Task> tasks{make_task(1, 3, 3, kNoDeadline, {600})};  // 10h
    const auto r = make("fatigue_aware").run(tasks, work_days(2));          // 8h days
    CHECK(r.blocks.empty());
    REQUIRE(r.unscheduled.size() == 1);
    CHECK(r.unscheduled[0].reason.find("longest free slot") != std::string::npos);
}

TEST_CASE("Running out of horizon is reported differently", "[scheduler][edge]") {
    std::vector<Task> tasks;
    for (int i = 1; i <= 3; ++i) tasks.push_back(make_task(i, 1, 3, kNoDeadline, {300}));
    const auto r = make("edf").run(tasks, work_days(1));  // only one 300-min task fits
    CHECK(r.metrics.tasks_scheduled == 1);
    REQUIRE(r.unscheduled.size() == 2);
    CHECK(r.unscheduled[0].reason.find("horizon") != std::string::npos);
```

```
}

TEST_CASE("Late tasks are flagged and lateness is summed", "[scheduler]") {
    // Deadline 09:30 but the task takes 60 min from 09:00 -> 30 min late.
    const std::vector<Task> tasks{make_task(1, 2, 5, 570, {60})};
    const auto r = make("edf").run(tasks, work_days(1));
    REQUIRE(r.blocks.size() == 1);
    CHECK(r.blocks[0].late);
    CHECK(r.metrics.tasks_late == 1);
    CHECK(r.metrics.total_lateness == 30);
}

TEST_CASE("Fatigue-aware strategy inserts breaks; EDF does not", "[scheduler]") {
    std::vector<Task> tasks;
    for (int i = 1; i <= 6; ++i) tasks.push_back(make_task(i, 5, 3, kNoDeadline, {60}));

    const auto aware = make("fatigue_aware").run(tasks, work_days(1));
    const auto edf = make("edf").run(tasks, work_days(1));

    CHECK(aware.metrics.breaks > 0);
    CHECK(edf.metrics.breaks == 0);
    // Breaks keep fatigue lower at the start of tasks.
    CHECK(aware.metrics.avg_fatigue_at_start < edf.metrics.avg_fatigue_at_start);
    // A break is always followed by lower fatigue than it started with.
    for (const auto& b : aware.blocks) {
        if (b.kind == BlockKind::Break) CHECK(b.fatigue_end < b.fatigue_start);
    }
}

TEST_CASE("No break is squeezed in right before a slot ends", "[scheduler][edge]") {
    // A 470-minute hard task runs 09:00-16:50 and leaves fatigue near 1.0.
    // Only 10 minutes of the day remain (< 15-minute break), so instead of a
    // useless 10-minute break the scheduler should move on to the next day.
    const std::vector<Task> tasks{make_task(1, 5, 3, kNoDeadline, {470}),
                                  make_task(2, 1, 3, kNoDeadline, {60})};
    const auto r = make("fatigue_aware").run(tasks, work_days(2));
    REQUIRE(r.blocks.size() == 2);
    CHECK(r.metrics.breaks == 0);
    CHECK(r.blocks[1].task_id == 2);
    CHECK(r.blocks[1].start == kDay + 540);  // next morning
}

TEST_CASE("Fatigue makes later tasks longer via the duration curve", "[scheduler]") {
    // Same task twice; the second one starts tired so it takes longer.
    const std::vector<Task> tasks{make_task(1, 5, 3, kNoDeadline, {60, 70, 80, 90, 100}),
                                  make_task(2, 5, 3, kNoDeadline, {60, 70, 80, 90, 100})};
    const auto r = make("edf").run(tasks, work_days(1));
    REQUIRE(r.blocks.size() == 2);
    CHECK(r.blocks[0].end - r.blocks[0].start == 60);
    CHECK(r.blocks[1].end - r.blocks[1].start > 60);
}

TEST_CASE("Overnight rest resets fatigue", "[scheduler]") {
    // Two 400-minute tasks: one per day.
    const std::vector<Task> tasks{make_task(1, 5, 3, kNoDeadline, {400}),
                                  make_task(2, 5, 3, kNoDeadline, {400})};
    Params p;
    p.break_threshold = 1.0;  // effectively never take breaks
    const auto r = make("fatigue_aware", p).run(tasks, work_days(2));
    REQUIRE(r.blocks.size() == 2);
    CHECK(r.blocks[1].start == kDay + 540);
    CHECK(r.blocks[1].fatigue_start < 1e-6);
}

TEST_CASE("Output does not depend on input order", "[scheduler][determinism]") {
    std::vector<Task> tasks{make_task(3, 2, 4, 2000, {40}), make_task(1, 5, 2, 3000, {90}),
                            make_task(2, 3, 5, 1500, {55})};
    std::vector<Task> reversed(tasks.rbegin(), tasks.rend());
    const auto a = make("fatigue_aware").run(tasks, work_days(2));
    const auto b = make("fatigue_aware").run(reversed, work_days(2));
    REQUIRE(a.blocks.size() == b.blocks.size());
    for (std::size_t i = 0; i < a.blocks.size(); ++i) {
```

```
        CHECK(a.blocks[i].task_id == b.blocks[i].task_id);
        CHECK(a.blocks[i].start == b.blocks[i].start);
    }
}

TEST_CASE("Invalid input is rejected", "[scheduler][edge]") {
    auto sched = make("edf");
    SECTION("duplicate ids") {
        const std::vector<Task> t{make_task(1, 3, 3, 0, {10}), make_task(1, 3, 3, 0, {10})};
        CHECK_THROWS_AS(sched.run(t, work_days(1)), std::invalid_argument);
    }
    SECTION("difficulty out of range") {
        CHECK_THROWS_AS(sched.run({make_task(1, 6, 3, 0, {10})}, work_days(1)),
                        std::invalid_argument);
    }
    SECTION("priority out of range") {
        CHECK_THROWS_AS(sched.run({make_task(1, 3, 0, 0, {10})}, work_days(1)),
                        std::invalid_argument);
    }
    SECTION("empty duration curve") {
        CHECK_THROWS_AS(sched.run({make_task(1, 3, 3, 0, {})}, work_days(1)),
                        std::invalid_argument);
    }
    SECTION("bad parameters") {
        Params p;
        p.break_threshold = 0.0;
        CHECK_THROWS_AS(Scheduler(p, make_policy("edf")), std::invalid_argument);
        p = Params{};
        p.break_minutes = 0;
        CHECK_THROWS_AS(Scheduler(p, make_policy("edf")), std::invalid_argument);
        CHECK_THROWS_AS(Scheduler(Params{}, nullptr), std::invalid_argument);
    }
}
```

Covers: empty inputs; first-slot placement; **invariants** (no overlap, inside free slots, every task scheduled exactly once or reported) for both strategies on a 25-task workload; reasons for unscheduled tasks; lateness accounting; breaks only for fatigue-aware and their effect on fatigue; no break squeezed in at slot end; fatigue lengthening later tasks; overnight recovery; **order independence**; every validation error.

`test_json_io.cpp`

File: `engine/tests/test_json_io.cpp`

```
#include <stdexcept>

#include <catch2/catch_test_macros.hpp>
#include <nlohmann/json.hpp>

#include "flexsched/json_io.hpp"

using namespace flexsched;
using nlohmann::json;

TEST_CASE("A minimal request uses default parameters", "[json]") {
    const json j = json::parse(R"({
        "windows": [{"start": 540, "end": 1020}],
        "tasks": [{"id": 1, "difficulty": 4, "priority": 2,
                   "deadline": 900, "duration_curve": [30, 40]}]
    })");
    const Request r = parse_request(j);
    CHECK(r.strategy == "fatigue_aware");
    CHECK(r.params.break_minutes == Params{}.break_minutes);
    REQUIRE(r.windows.size() == 1);
    CHECK(r.blocked.empty());
    REQUIRE(r.tasks.size() == 1);
    CHECK(r.tasks[0].duration_curve.size() == 2);
}
```

```
TEST_CASE("Provided parameters override defaults", "[json]") {
    const json j = {{"strategy", "edf"},
                    {"params", {{"break_minutes", 20}, {"w_energy", 0.5}}}};
    const Request r = parse_request(j);
    CHECK(r.strategy == "edf");
    CHECK(r.params.break_minutes == 20);
    CHECK(r.params.w_energy == 0.5);
    CHECK(r.params.work_rate == Params{}.work_rate);  // untouched
}

TEST_CASE("Malformed requests are rejected with a clear error", "[json][edge]") {
    CHECK_THROWS_AS(parse_request(json::array()), std::invalid_argument);
    CHECK_THROWS_AS(parse_request({{"windows", {{{"start", 10}}}}}), std::invalid_argument);
    CHECK_THROWS_AS(parse_request({{"windows", {{{"start", 10}, {"end", 5}}}}}),
                    std::invalid_argument);
    CHECK_THROWS_AS(parse_request({{"tasks", {{{"id", 1}}}}}), std::invalid_argument);
    CHECK_THROWS_AS(parse_request({{"tasks", "nope"}}), std::invalid_argument);
}

TEST_CASE("Results serialise to the documented shape", "[json]") {
    ScheduleResult res;
    ScheduledBlock task_block;
    task_block.task_id = 3;
    task_block.start = 540;
    task_block.end = 600;
    task_block.late = true;
    ScheduledBlock break_block;
    break_block.kind = BlockKind::Break;
    break_block.start = 600;
    break_block.end = 615;
    res.blocks = {task_block, break_block};
    res.unscheduled = {{9, "too long"}};
    res.metrics.tasks_scheduled = 1;

    const json out = to_json(res, "fatigue_aware");
    CHECK(out["strategy"] == "fatigue_aware");
    REQUIRE(out["blocks"].size() == 2);
    CHECK(out["blocks"][0]["kind"] == "task");
    CHECK(out["blocks"][0]["task_id"] == 3);
    CHECK(out["blocks"][0]["late"] == true);
    CHECK(out["blocks"][1]["kind"] == "break");
    CHECK_FALSE(out["blocks"][1].contains("task_id"));
    CHECK(out["unscheduled"][0]["reason"] == "too long");
    CHECK(out["metrics"]["tasks_scheduled"] == 1);
}
```

Covers: defaults, parameter overrides, malformed requests, and the exact output shape.

Part 4 — Python package: code walkthrough

The package is **layered**: each module imports only from layers below it (never upwards), so lower layers can be tested without the upper ones.

Layer	Modules	Role
4. Interface	`cli.py`, `__main__.py`	parse commands, print results
3. Services	`planner.py`, `benchmark.py` (uses `planner.build_request`), `synthetic.py`	orchestrate the use cases
2. ML & integration	`predictor.py`, `engine_bridge.py`, `fatigue.py`	models, C++ bridge, fatigue maths
1. Persistence	`repository.py`, `db.py`, `schema.sql`	all SQL and connections
0. Foundations	`models.py`, `config.py`	dataclasses and settings

4.1 `config.py`: single source of truth

File: `flexsched/config.py`

```python
"""Central configuration: file locations and model parameters.

Every tunable number lives here so that there is exactly ONE source of truth.
The Python side sends these values to the C++ engine inside each request,
so the engine's compiled-in defaults are only a fallback.
"""

from __future__ import annotations

import os
from dataclasses import asdict, dataclass
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parent.parent

# Locations can be overridden with environment variables (handy for tests/CI).
DB_PATH = Path(os.environ.get("FLEXSCHED_DB", PROJECT_ROOT / "data" / "flexsched.db"))
MODEL_PATH = Path(
    os.environ.get("FLEXSCHED_MODEL", PROJECT_ROOT / "models" / "duration_model.joblib")
)

# Fatigue levels at which the ML model predicts a duration. The engine
# interpolates between these points (see engine/include/flexsched/duration.hpp).
FATIGUE_LEVELS: tuple[float, ...] = (0.0, 0.25, 0.5, 0.75, 1.0)

# Allowed task categories (mirrors the CHECK constraint in schema.sql).
CATEGORIES: tuple[str, ...] = ("coding", "study", "writing", "admin", "review", "design")


@dataclass(frozen=True)
class SchedulerParams:
    """Parameters of the fatigue model and the scheduling policy."""

    work_rate: float = 0.012       # fatigue gained per minute at full load
    recovery_rate: float = 0.05    # fraction of fatigue lost per minute of rest
    break_threshold: float = 0.65  # take a break when fatigue reaches this
    break_minutes: int = 15
    w_priority: float = 0.35
```

```
    w_urgency: float = 0.45
    w_energy: float = 0.20
    urgency_horizon: float = 480.0

    def to_dict(self) -> dict:
        return asdict(self)


DEFAULT_PARAMS = SchedulerParams()
```

Notes

- `Path(__file__).resolve().parent.parent` is the repository root, wherever it's cloned.
- `os.environ.get("FLEXSCHED_DB", default)` : environment variables override paths, which is useful for tests, CI and deployment (a *12-factor app* habit).
- `SchedulerParams` is **frozen**: parameters can't be mutated during a run. `to_dict()` uses `dataclasses.asdict`, and that dict becomes the `"params"` object of the engine request, so Python's values always win over the engine's compiled defaults.
- `CATEGORIES` mirrors the SQL `CHECK` constraint (validated in two layers).

4.2 `models.py` : domain dataclasses

File: `flexsched/models.py`

```
"""Domain objects as dataclasses.

A dataclass is a normal Python class where ``__init__``, ``__repr__`` and
``__eq__`` are generated from the type-annotated fields. They make the rest
of the code read like "task.difficulty" instead of "row[3]".
"""

from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional


@dataclass
class Task:
    title: str
    category: str
    difficulty: int          # 1..5
    priority: int            # 1..5
    estimate_min: int        # the user's own guess, in minutes
    deadline: Optional[datetime] = None
    status: str = "pending"  # pending | scheduled | done | cancelled
    id: Optional[int] = None
    created_at: Optional[datetime] = None


@dataclass
class WorkSession:
    """One uninterrupted stretch of work on a task (the ML training unit)."""

    task_id: int
    started_at: datetime
    ended_at: datetime
    fatigue_at_start: Optional[float] = None
    id: Optional[int] = None

    @property
    def actual_min(self) -> int:
        return int(round((self.ended_at - self.started_at).total_seconds() / 60))
```

```python
@dataclass
class PlannedBlock:
    """One row of a generated timetable (a task or a break)."""

    kind: str                       # "task" | "break"
    start: datetime
    end: datetime
    fatigue_start: float
    fatigue_end: float
    task_id: Optional[int] = None
    late: bool = False
    title: str = ""


@dataclass
class PlanResult:
    strategy: str
    blocks: list[PlannedBlock] = field(default_factory=list)
    unscheduled: list[tuple[int, str]] = field(default_factory=list)
    metrics: dict = field(default_factory=dict)
    run_id: Optional[int] = None
```

Notes

- `Task.id` is `Optional[int]`: `None` until the database assigns one.
- `WorkSession.actual_min` is a `@property` computed from the timestamps, so it can never disagree with them.
- `PlanResult.blocks` uses `field(default_factory=list)` (see Part 2.1).

4.3 `schema.sql`: the database design

File: `flexsched/schema.sql`

```sql
-- FLEXSched database schema (SQLite).
-- Executed by flexsched/db.py; every statement is idempotent (IF NOT EXISTS),
-- so running it on an existing database is safe.

-- Tasks the user wants to get done.
CREATE TABLE IF NOT EXISTS tasks (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    title         TEXT    NOT NULL CHECK (length(trim(title)) > 0),
    category      TEXT    NOT NULL
                  CHECK (category IN ('coding','study','writing','admin','review','design')),
    difficulty    INTEGER NOT NULL CHECK (difficulty BETWEEN 1 AND 5),
    priority      INTEGER NOT NULL CHECK (priority BETWEEN 1 AND 5),
    estimate_min  INTEGER NOT NULL CHECK (estimate_min > 0),
    deadline      TEXT,                              -- ISO-8601 'YYYY-MM-DD HH:MM:SS' or NULL
    status        TEXT    NOT NULL DEFAULT 'pending'
                  CHECK (status IN ('pending','scheduled','done','cancelled')),
    created_at    TEXT    NOT NULL DEFAULT (datetime('now'))
);

CREATE INDEX IF NOT EXISTS idx_tasks_status ON tasks(status);

-- Historical record of actual work: the training data for the ML model.
CREATE TABLE IF NOT EXISTS work_sessions (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    task_id           INTEGER NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
    started_at        TEXT    NOT NULL,
    ended_at          TEXT    NOT NULL,
    actual_min        INTEGER NOT NULL CHECK (actual_min > 0),
    fatigue_at_start  REAL    CHECK (fatigue_at_start BETWEEN 0 AND 1),
    CHECK (ended_at > started_at)
```

```
);

CREATE INDEX IF NOT EXISTS idx_sessions_started ON work_sessions(started_at);
CREATE INDEX IF NOT EXISTS idx_sessions_task ON work_sessions(task_id);

-- Every time the planner runs we store the result, so plans can be compared later.
CREATE TABLE IF NOT EXISTS schedule_runs (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    created_at     TEXT NOT NULL DEFAULT (datetime('now')),
    strategy       TEXT NOT NULL,
    horizon_start  TEXT NOT NULL,
    horizon_days   INTEGER NOT NULL CHECK (horizon_days > 0),
    metrics_json   TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS schedule_items (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    run_id         INTEGER NOT NULL REFERENCES schedule_runs(id) ON DELETE CASCADE,
    task_id        INTEGER REFERENCES tasks(id) ON DELETE SET NULL,  -- NULL for breaks
    kind           TEXT NOT NULL CHECK (kind IN ('task','break')),
    start_at       TEXT NOT NULL,
    end_at         TEXT NOT NULL,
    fatigue_start  REAL NOT NULL,
    fatigue_end    REAL NOT NULL,
    is_late        INTEGER NOT NULL DEFAULT 0 CHECK (is_late IN (0,1))
);

CREATE INDEX IF NOT EXISTS idx_items_run ON schedule_items(run_id);
```

Entity–relationship view

```
tasks (1) ──────< (N) work_sessions        a task can have many sessions
tasks (1) ──────< (N) schedule_items       an item points to a task (or NULL for a break)
schedule_runs (1) ──< (N) schedule_items   a run contains many items
```

Design choices to defend

- **Normalised**: task attributes live only in `tasks`; sessions and schedule items reference them by id (no duplicated titles that could disagree).
- **Constraints in the database** as well as in Python (defence in depth): even a bug or a manual `INSERT` can't store difficulty 7 or a session that ends before it starts (`CHECK (ended_at > started_at)` works because ISO text compares chronologically).
- `actual_min` is stored even though it's derivable: it makes training queries simple and fast. That's a conscious, small denormalisation.
- `metrics_json` stores a JSON blob: metrics are read whole and never queried by field, so a column per metric would add schema churn for no benefit.
- **Indexes** match the queries: `status` (list open tasks), `started_at` (sessions of a day, training order), `task_id` (joins), `run_id` (items of a run).

4.4 `db.py`: connections and schema initialisation

File: `flexsched/db.py`

```
"""SQLite connection handling and schema initialisation.

sqlite3 ships with Python - no server to install. The whole database is a
single file (``data/flexsched.db``), or ``:memory:`` for tests.
"""

from __future__ import annotations
```

```python
import sqlite3
from datetime import datetime
from pathlib import Path
from typing import Optional, Union

from . import config

SCHEMA_PATH = Path(__file__).with_name("schema.sql")
SCHEMA_VERSION = 1

# One consistent text format for datetimes. ISO-8601 strings sort correctly
# as plain text, so ORDER BY / comparisons work without conversion.
DATETIME_FMT = "%Y-%m-%d %H:%M:%S"


def to_db_time(value: Optional[datetime]) -> Optional[str]:
    return None if value is None else value.strftime(DATETIME_FMT)


def from_db_time(value: Optional[str]) -> Optional[datetime]:
    return None if value is None else datetime.strptime(value, DATETIME_FMT)


def connect(path: Union[str, Path, None] = None) -> sqlite3.Connection:
    """Open (and if needed create) the database, returning a ready connection."""
    target = str(path if path is not None else config.DB_PATH)
    if target != ":memory:":
        Path(target).parent.mkdir(parents=True, exist_ok=True)

    conn = sqlite3.connect(target)
    conn.row_factory = sqlite3.Row             # rows behave like dicts: row["title"]
    conn.execute("PRAGMA foreign_keys = ON")   # SQLite has FKs OFF by default!
    init_schema(conn)
    return conn


def init_schema(conn: sqlite3.Connection) -> None:
    """Create tables if missing and record the schema version."""
    conn.executescript(SCHEMA_PATH.read_text(encoding="utf-8"))
    current = conn.execute("PRAGMA user_version").fetchone()[0]
    if current < SCHEMA_VERSION:
        # Future migrations would go here: "if current < 2: ALTER TABLE ...".
        conn.execute(f"PRAGMA user_version = {SCHEMA_VERSION}")
    conn.commit()
```

Notes

- `":memory:"` creates a private in-RAM database; every test gets a fresh one.
- `conn.row_factory = sqlite3.Row` lets code write `row["title"]` instead of `row[1]`.
- `PRAGMA foreign_keys = ON` **per connection** (SQLite's default is OFF).
- `executescript` runs the whole schema file; `IF NOT EXISTS` makes it idempotent.
- `user_version` records the schema version for future migrations.
- `to_db_time` / `from_db_time` keep one datetime text format in a single place.

4.5 `repository.py`: all SQL in one place

File: `flexsched/repository.py`

```python
"""Repository layer: the ONLY place that contains SQL.

Repository pattern: the rest of the program asks for domain objects
("give me the pending tasks") and never sees SQL or sqlite3 rows. If we
later swap SQLite for PostgreSQL, only this file has to change.
```

```
All queries use ``?`` placeholders (parameterised queries). Never build SQL
with f-strings from user input - that is how SQL injection happens.
"""

from __future__ import annotations

import json
import sqlite3
from datetime import date, datetime, timedelta
from typing import Iterable, Optional

from . import config
from .db import from_db_time, to_db_time
from .models import PlanResult, Task, WorkSession

VALID_STATUSES = ("pending", "scheduled", "done", "cancelled")


def validate_task(task: Task) -> None:
    """Application-level validation with friendly messages.

    The database CHECK constraints enforce the same rules (defence in depth),
    but their error messages are cryptic, so we check first.
    """
    if not task.title or not task.title.strip():
        raise ValueError("title must not be empty")
    if task.category not in config.CATEGORIES:
        raise ValueError(f"category must be one of {', '.join(config.CATEGORIES)}")
    if not 1 <= task.difficulty <= 5:
        raise ValueError("difficulty must be between 1 and 5")
    if not 1 <= task.priority <= 5:
        raise ValueError("priority must be between 1 and 5")
    if task.estimate_min <= 0:
        raise ValueError("estimate must be a positive number of minutes")
    if task.status not in VALID_STATUSES:
        raise ValueError(f"status must be one of {', '.join(VALID_STATUSES)}")


def _row_to_task(row: sqlite3.Row) -> Task:
    return Task(
        id=row["id"],
        title=row["title"],
        category=row["category"],
        difficulty=row["difficulty"],
        priority=row["priority"],
        estimate_min=row["estimate_min"],
        deadline=from_db_time(row["deadline"]),
        status=row["status"],
        created_at=from_db_time(row["created_at"]),
    )


class TaskRepository:
    def __init__(self, conn: sqlite3.Connection):
        self.conn = conn

    def add(self, task: Task) -> Task:
        validate_task(task)
        cur = self.conn.execute(
            """INSERT INTO tasks (title, category, difficulty, priority,
                                  estimate_min, deadline, status)
               VALUES (?, ?, ?, ?, ?, ?, ?)""",
            (task.title.strip(), task.category, task.difficulty, task.priority,
             task.estimate_min, to_db_time(task.deadline), task.status),
        )
        self.conn.commit()
        task.id = cur.lastrowid
        return task

    def get(self, task_id: int) -> Optional[Task]:
        row = self.conn.execute("SELECT * FROM tasks WHERE id = ?", (task_id,)).fetchone()
```

```
        return _row_to_task(row) if row else None

    def list(self, statuses: Optional[Iterable[str]] = None) -> list[Task]:
        if statuses is None:
            rows = self.conn.execute("SELECT * FROM tasks ORDER BY id").fetchall()
        else:
            statuses = list(statuses)
            marks = ",".join("?" for _ in statuses)  # "?,?,?" - still parameterised
            rows = self.conn.execute(
                f"SELECT * FROM tasks WHERE status IN ({marks}) ORDER BY id", statuses
            ).fetchall()
        return [_row_to_task(r) for r in rows]

    def set_status(self, task_ids: Iterable[int], status: str) -> None:
        if status not in VALID_STATUSES:
            raise ValueError(f"invalid status {status!r}")
        self.conn.executemany(
            "UPDATE tasks SET status = ? WHERE id = ?", [(status, i) for i in task_ids]
        )
        self.conn.commit()


class SessionRepository:
    def __init__(self, conn: sqlite3.Connection):
        self.conn = conn

    def add(self, session: WorkSession, commit: bool = True) -> WorkSession:
        if session.ended_at <= session.started_at:
            raise ValueError("a session must end after it starts")
        cur = self.conn.execute(
            """INSERT INTO work_sessions (task_id, started_at, ended_at,
                                          actual_min, fatigue_at_start)
               VALUES (?, ?, ?, ?, ?)""",
            (session.task_id, to_db_time(session.started_at), to_db_time(session.ended_at),
             session.actual_min, session.fatigue_at_start),
        )
        if commit:
            self.conn.commit()
        session.id = cur.lastrowid
        return session

    def for_day(self, day: date) -> list[tuple[WorkSession, int]]:
        """Sessions on a calendar day with their task difficulty, in time order."""
        start = datetime.combine(day, datetime.min.time())
        rows = self.conn.execute(
            """SELECT s.*, t.difficulty
               FROM work_sessions s JOIN tasks t ON t.id = s.task_id
               WHERE s.started_at >= ? AND s.started_at < ?
               ORDER BY s.started_at""",
            (to_db_time(start), to_db_time(start + timedelta(days=1))),
        ).fetchall()
        return [
            (WorkSession(id=r["id"], task_id=r["task_id"],
                         started_at=from_db_time(r["started_at"]),
                         ended_at=from_db_time(r["ended_at"]),
                         fatigue_at_start=r["fatigue_at_start"]), r["difficulty"])
            for r in rows
        ]

    def count(self) -> int:
        return self.conn.execute("SELECT COUNT(*) FROM work_sessions").fetchone()[0]

    # SQL used to build the ML training set: a JOIN plus feature extraction
    # done directly in SQL with strftime().
    TRAINING_SQL = """
        SELECT s.id                                     AS session_id,
               t.category, t.difficulty, t.priority, t.estimate_min,
               s.started_at, s.actual_min, s.fatigue_at_start,
               CAST(strftime('%H', s.started_at) AS INTEGER) AS hour_of_day,
               CAST(strftime('%w', s.started_at) AS INTEGER) AS day_of_week
        FROM work_sessions s
        JOIN tasks t ON t.id = s.task_id
```

```
        WHERE s.fatigue_at_start IS NOT NULL
        ORDER BY s.started_at
    """

    # Aggregation query: how wrong are the user's estimates, per category?
    ESTIMATION_BIAS_SQL = """
        SELECT t.category,
               COUNT(*)                                        AS sessions,
               ROUND(AVG(t.estimate_min), 1)                   AS avg_estimate,
               ROUND(AVG(s.actual_min), 1)                     AS avg_actual,
               ROUND(AVG(1.0 * s.actual_min / t.estimate_min), 3) AS avg_ratio
        FROM work_sessions s
        JOIN tasks t ON t.id = s.task_id
        GROUP BY t.category
        HAVING COUNT(*) >= 1
        ORDER BY avg_ratio DESC
    """

    def estimation_bias(self) -> list[sqlite3.Row]:
        return self.conn.execute(self.ESTIMATION_BIAS_SQL).fetchall()


class ScheduleRepository:
    def __init__(self, conn: sqlite3.Connection):
        self.conn = conn

    def save(self, plan: PlanResult, horizon_start: datetime, horizon_days: int) -> int:
        """Store a run and all its blocks in ONE transaction (all or nothing)."""
        with self.conn:  # commits on success, rolls back on exception
            cur = self.conn.execute(
                """INSERT INTO schedule_runs (strategy, horizon_start, horizon_days, metrics_json)
                   VALUES (?, ?, ?, ?)""",
                (plan.strategy, to_db_time(horizon_start), horizon_days,
                 json.dumps(plan.metrics)),
            )
            run_id = cur.lastrowid
            self.conn.executemany(
                """INSERT INTO schedule_items (run_id, task_id, kind, start_at, end_at,
                                               fatigue_start, fatigue_end, is_late)
                   VALUES (?, ?, ?, ?, ?, ?, ?, ?)""",
                [(run_id, b.task_id, b.kind, to_db_time(b.start), to_db_time(b.end),
                  b.fatigue_start, b.fatigue_end, int(b.late)) for b in plan.blocks],
            )
        plan.run_id = run_id
        return run_id

    def runs(self, limit: int = 10) -> list[sqlite3.Row]:
        return self.conn.execute(
            """SELECT r.id, r.created_at, r.strategy, r.horizon_start, r.horizon_days,
                      r.metrics_json, COUNT(i.id) AS items
               FROM schedule_runs r LEFT JOIN schedule_items i ON i.run_id = r.id
               GROUP BY r.id
               ORDER BY r.id DESC
               LIMIT ?""",
            (limit,),
        ).fetchall()
```

Notes

- **Repository pattern**: the rest of the code calls `TaskRepository(conn).list(["pending"])` and gets `Task` objects. It never sees SQL. Swapping SQLite for PostgreSQL would touch this file (and `db.py`) only.
- `validate_task` gives friendly errors; the DB constraints are the safety net.
- Dynamic `IN (?, ?, ?)`: the *number* of placeholders is built dynamically, but values are still passed separately, so it stays injection-safe.
- `executemany` runs one statement for many parameter tuples (efficient bulk update).

- `ScheduleRepository.save` uses `with self.conn:` → **atomic** (run row + all item rows, or nothing). `test_schedule_save_rolls_back_on_error` proves it.
- `TRAINING_SQL` does feature extraction in SQL (`strftime('%H')`, `strftime('%w')`). Note SQLite's `%w` is Sunday=0, while Python's `weekday()` is Monday=0; `planner.py` converts with `_sqlite_weekday` so training and prediction agree.
- `ESTIMATION_BIAS_SQL` powers `flexsched stats`: `GROUP BY category` with `AVG(actual/estimate)`, the "how wrong am I per category" table.

4.6 `fatigue.py`: Python mirror of the engine's model

File: `flexsched/fatigue.py`

```python
"""Python mirror of the C++ fatigue model (engine/src/fatigue.cpp).

Why duplicate it? The ML model needs ``fatigue_at_start`` for every
historical work session, i.e. "how tired was the user when they began this
task?". Python reconstructs that by replaying the day's sessions through
the same formulas the engine uses. tests/test_parity.py runs both
implementations on the same input and asserts they agree.

    work:  F' = 1 - (1 - F) * exp(-work_rate * (difficulty/5) * minutes)
    rest:  F' = F * exp(-recovery_rate * minutes)
"""

from __future__ import annotations

import math
from datetime import datetime
from typing import Iterable

from .config import DEFAULT_PARAMS, SchedulerParams


def _clamp01(x: float) -> float:
    return min(1.0, max(0.0, x))


def after_work(fatigue: float, difficulty: int, minutes: float,
               params: SchedulerParams = DEFAULT_PARAMS) -> float:
    if minutes <= 0:
        return _clamp01(fatigue)
    load = min(5, max(1, difficulty)) / 5.0
    decay = math.exp(-params.work_rate * load * minutes)
    return _clamp01(1.0 - (1.0 - _clamp01(fatigue)) * decay)


def after_rest(fatigue: float, minutes: float,
               params: SchedulerParams = DEFAULT_PARAMS) -> float:
    if minutes <= 0:
        return _clamp01(fatigue)
    return _clamp01(_clamp01(fatigue) * math.exp(-params.recovery_rate * minutes))


def replay_day(sessions: Iterable[tuple[datetime, datetime, int]],
               params: SchedulerParams = DEFAULT_PARAMS) -> list[float]:
    """Return fatigue at the start of each (start, end, difficulty) session.

    Sessions must be from the same day and sorted by start time. The day
    starts fresh (F = 0); gaps between sessions count as rest.
    """
    fatigue = 0.0
    previous_end: datetime | None = None
    result: list[float] = []
    for start, end, difficulty in sessions:
        if previous_end is not None:
```

```
            gap = (start - previous_end).total_seconds() / 60
            fatigue = after_rest(fatigue, gap, params)
        result.append(fatigue)
        worked = (end - start).total_seconds() / 60
        fatigue = after_work(fatigue, difficulty, worked, params)
        previous_end = end
    return result
```

Notes

- Same formulas as `fatigue.cpp`. Why duplicate? The ML feature `fatigue_at_start` must be computed for **historical** sessions, which never went through the engine.
- Duplication risk is controlled by a **parity test** (`test_python_and_cpp_fatigue_models_agree`): Python replays the engine's output block by block and must match every `fatigue_start` / `fatigue_end` within 1e-9.
- `replay_day`: start fresh, rest during gaps, work during sessions. Returns the fatigue at the start of each session.

4.7 `synthetic.py`: the simulated user

File: `flexsched/synthetic.py`

```
"""Synthetic work-history generator.

A brand-new user has no history, and a demo needs data. This module
simulates a person working through tasks for N days and writes the result
into the same tables real usage would fill (tasks + work_sessions).

The simulated person has HIDDEN habits that the ML model has to discover:
    * they underestimate some categories (coding x1.35) more than others (admin x0.9)
    * they underestimate hard tasks more (+8% per difficulty step above 3)
    * fatigue slows them down (up to +55% when exhausted)
    * there is a post-lunch dip (13:00-15:00 is ~10% slower)
    * plus random day-to-day noise (log-normal, ~12%)

Being honest about this matters in an interview: the ML results show the
model CAN recover such patterns; with real data the patterns are whatever
the real user's habits are.
"""

from __future__ import annotations

import sqlite3
from datetime import date, datetime, timedelta

import numpy as np

from . import config
from .db import to_db_time
from .fatigue import after_rest, after_work

CATEGORY_BIAS = {"coding": 1.35, "study": 1.10, "writing": 1.25,
                 "admin": 0.90, "review": 1.00, "design": 1.20}
ESTIMATE_CHOICES = np.array([15, 20, 30, 45, 60, 75, 90, 120])
ESTIMATE_WEIGHTS = np.array([6, 8, 14, 16, 18, 10, 9, 5], dtype=float)

TITLES = {
    "coding": ["Implement feature", "Fix bug", "Refactor module", "Write unit tests"],
    "study": ["Read paper", "Lecture revision", "Practice problems", "Course module"],
    "writing": ["Draft report", "Write documentation", "Blog post", "Edit chapter"],
    "admin": ["Answer emails", "Plan week", "Fill forms", "Expense report"],
    "review": ["Code review", "Review notes", "Proofread", "Check submissions"],
    "design": ["Sketch UI", "System design", "Architecture doc", "Wireframes"],
}
```

```python
def true_duration(estimate: float, category: str, difficulty: int, priority: int,
                  fatigue: float, hour: int, rng: np.random.Generator) -> int:
    """The hidden 'ground truth' of how long the simulated person takes."""
    factor = CATEGORY_BIAS[category]
    factor *= 1.0 + 0.08 * (difficulty - 3)
    factor *= 1.0 - 0.03 * (priority - 3)          # urgent work gets more focus
    factor *= 1.0 + 0.55 * fatigue
    if 13 <= hour < 15:
        factor *= 1.10                              # post-lunch dip
    elif hour < 11:
        factor *= 0.95                              # fresh morning focus
    factor *= rng.lognormal(mean=0.0, sigma=0.12)
    return max(5, int(round(estimate * factor)))


def generate_history(conn: sqlite3.Connection, days: int = 120, seed: int = 42,
                     end_day: date | None = None) -> int:
    """Simulate `days` days of work ending yesterday. Returns sessions created."""
    rng = np.random.default_rng(seed)  # seeded -> reproducible results
    end_day = end_day or (date.today() - timedelta(days=1))
    start_day = end_day - timedelta(days=days - 1)
    probs = ESTIMATE_WEIGHTS / ESTIMATE_WEIGHTS.sum()

    sessions = 0
    with conn:  # one transaction for everything: fast and atomic
        for offset in range(days):
            day = start_day + timedelta(days=offset)
            weekend = day.weekday() >= 5
            n_tasks = int(rng.integers(0, 3) if weekend else rng.integers(3, 8))

            clock = datetime.combine(day, datetime.min.time()) + timedelta(
                hours=9, minutes=int(rng.integers(0, 46)))
            fatigue = 0.0
            took_lunch = False

            for _ in range(n_tasks):
                category = str(rng.choice(config.CATEGORIES))
                difficulty = int(rng.integers(1, 6))
                priority = int(rng.integers(1, 6))
                estimate = int(rng.choice(ESTIMATE_CHOICES, p=probs))
                title = f"{rng.choice(TITLES[category])} #{int(rng.integers(100, 999))}"

                fatigue_at_start = fatigue
                minutes = true_duration(estimate, category, difficulty, priority,
                                        fatigue, clock.hour, rng)
                start, end = clock, clock + timedelta(minutes=minutes)

                cur = conn.execute(
                    """INSERT INTO tasks (title, category, difficulty, priority,
                                          estimate_min, deadline, status, created_at)
                       VALUES (?, ?, ?, ?, ?, NULL, 'done', ?)""",
                    (title, category, difficulty, priority, estimate,
                     to_db_time(start - timedelta(days=int(rng.integers(0, 4))))),
                )
                conn.execute(
                    """INSERT INTO work_sessions (task_id, started_at, ended_at,
                                                  actual_min, fatigue_at_start)
                       VALUES (?, ?, ?, ?, ?)""",
                    (cur.lastrowid, to_db_time(start), to_db_time(end), minutes,
                     round(fatigue_at_start, 6)),
                )
                sessions += 1

                # Advance the simulated clock and fatigue exactly like the engine would.
                fatigue = after_work(fatigue, difficulty, minutes)
                gap = int(rng.integers(0, 21))
                if not took_lunch and end.hour >= 12:
                    gap += int(rng.integers(40, 61))
                    took_lunch = True
                fatigue = after_rest(fatigue, gap)
```

```
                clock = end + timedelta(minutes=gap)
                if clock.hour >= 19:
                    break  # person stops working for the day
    return sessions
```

Notes

- `np.random.default_rng(seed)` is NumPy's modern random generator. The same seed gives the same data, so results are reproducible (and a test checks it).
- `true_duration` encodes the **hidden habits** (category bias, difficulty, priority focus, fatigue slowdown, morning focus, post-lunch dip, log-normal noise). The ML model never sees this function; it must rediscover the patterns from data.
- The simulation advances fatigue with the **same** `after_work` / `after_rest`, so stored `fatigue_at_start` values are exactly what `replay_day` would compute.
- All inserts happen in **one transaction** (`with conn:`), which is much faster than committing after each row.
- Weekends get 0–2 tasks, weekdays 3–7, with a lunch gap once the clock passes noon.

4.8 `predictor.py`: training, evaluation, duration curves

File: `flexsched/predictor.py`

```
"""Machine-learning duration prediction with scikit-learn.

Goal: given a task and the user's state, predict how many minutes it will
REALLY take (people are bad at estimating - the "planning fallacy").

Key design decisions (each one is an interview talking point):

1. Target = ratio actual / estimate, not raw minutes.
   The ratio is scale-free ("I take 1.3x my guess for coding"), so a
   pattern learnt on 30-minute tasks transfers to 120-minute tasks. Tree
   models cannot extrapolate beyond the target values they saw; predicting
   a ratio and multiplying back by the estimate side-steps that.

2. Temporal train/test split, not a random one.
   We train on the past and test on the most recent 20% of sessions,
   exactly how the model is used in reality. A random split would leak
   "future" information and give over-optimistic scores.

3. Model selection with TimeSeriesSplit cross-validation on the training
   part only. The hold-out set is touched once, for the final report.

4. Always compare against baselines: "just trust the estimate" and
   "multiply by the average ratio". A model is only useful if it beats them.

5. Fatigue is an input feature, so we can ask "how long at fatigue 0.0,
   0.25, ..., 1.0?" and give the engine a whole duration CURVE.
"""

from __future__ import annotations

import sqlite3
from dataclasses import dataclass, field
from datetime import datetime
from pathlib import Path
from typing import Optional, Union

import joblib
import numpy as np
import pandas as pd
import sklearn
```

```python
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.inspection import permutation_importance
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error, r2_score
from sklearn.model_selection import TimeSeriesSplit
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from . import config
from .models import Task
from .repository import SessionRepository

CATEGORICAL = ["category"]
NUMERIC = ["estimate_min", "difficulty", "priority", "fatigue_at_start",
           "hour_of_day", "day_of_week"]
FEATURES = CATEGORICAL + NUMERIC
MIN_TRAINING_ROWS = 40
DEFAULT_PLANNING_HOUR = 11  # hour used when the exact start time is not known yet


class InsufficientDataError(RuntimeError):
    pass


# ---------------------------------------------------------------- data -----

def load_training_frame(conn: sqlite3.Connection) -> pd.DataFrame:
    """Run the training SQL and return a DataFrame sorted by time."""
    df = pd.read_sql_query(SessionRepository.TRAINING_SQL, conn)
    df["started_at"] = pd.to_datetime(df["started_at"])
    return df.sort_values("started_at").reset_index(drop=True)


def to_ratio(df: pd.DataFrame) -> np.ndarray:
    return (df["actual_min"] / df["estimate_min"]).to_numpy(dtype=float)


# ---------------------------------------------------------------- models -----

def _preprocessor(scale_numeric: bool) -> ColumnTransformer:
    numeric = StandardScaler() if scale_numeric else "passthrough"
    return ColumnTransformer([
        ("cat", OneHotEncoder(handle_unknown="ignore"), CATEGORICAL),
        ("num", numeric, NUMERIC),
    ])


def candidate_models() -> dict[str, Pipeline]:
    """All models we compare. Each is a full Pipeline: preprocessing + regressor."""
    return {
        "trust_estimate": Pipeline([("pre", _preprocessor(False)),
                                    ("reg", DummyRegressor(strategy="constant",
                                                           constant=1.0))]),
        "mean_ratio": Pipeline([("pre", _preprocessor(False)),
                                ("reg", DummyRegressor(strategy="mean"))]),
        "ridge": Pipeline([("pre", _preprocessor(True)),
                           ("reg", Ridge(alpha=1.0))]),
        "random_forest": Pipeline([("pre", _preprocessor(False)),
                                   ("reg", RandomForestRegressor(
                                       n_estimators=200, min_samples_leaf=5,
                                       random_state=42, n_jobs=-1))]),
        "gradient_boosting": Pipeline([("pre", _preprocessor(False)),
                                       ("reg", GradientBoostingRegressor(
                                           n_estimators=300, learning_rate=0.05,
                                           max_depth=3, subsample=0.8,
                                           random_state=42))]),
    }


def _minutes_from_ratio(pipeline: Pipeline, X: pd.DataFrame) -> np.ndarray:
```

```
    ratio = np.clip(pipeline.predict(X[FEATURES]), 0.2, 5.0)  # guard against wild outputs
    return ratio * X["estimate_min"].to_numpy(dtype=float)


@dataclass
class TrainingReport:
    rows: int
    train_rows: int
    holdout_rows: int
    chosen_model: str
    results: dict[str, dict[str, float]] = field(default_factory=dict)
    feature_importance: dict[str, float] = field(default_factory=dict)


def train(conn: sqlite3.Connection, holdout_fraction: float = 0.2,
          n_splits: int = 5) -> tuple["DurationPredictor", TrainingReport]:
    df = load_training_frame(conn)
    if len(df) < MIN_TRAINING_ROWS:
        raise InsufficientDataError(
            f"need at least {MIN_TRAINING_ROWS} logged sessions to train, have {len(df)}")

    # 1. Temporal split: oldest 80% for training, newest 20% held out.
    cut = int(len(df) * (1 - holdout_fraction))
    train_df, hold_df = df.iloc[:cut], df.iloc[cut:]

    results: dict[str, dict[str, float]] = {}
    for name, model in candidate_models().items():
        # 2. Cross-validation that respects time order (folds only look backwards).
        fold_mae = []
        for tr_idx, va_idx in TimeSeriesSplit(n_splits=n_splits).split(train_df):
            tr, va = train_df.iloc[tr_idx], train_df.iloc[va_idx]
            model.fit(tr[FEATURES], to_ratio(tr))
            fold_mae.append(mean_absolute_error(va["actual_min"],
                                                _minutes_from_ratio(model, va)))

        # 3. Fit on the full training part, evaluate ONCE on the hold-out.
        model.fit(train_df[FEATURES], to_ratio(train_df))
        pred = _minutes_from_ratio(model, hold_df)
        results[name] = {
            "cv_mae": float(np.mean(fold_mae)),
            "holdout_mae": float(mean_absolute_error(hold_df["actual_min"], pred)),
            "holdout_mape": float(mean_absolute_percentage_error(hold_df["actual_min"], pred)),
            "holdout_r2": float(r2_score(hold_df["actual_min"], pred)),
        }

    # 4. Choose by cross-validation score (NOT by hold-out, which would leak).
    chosen = min(results, key=lambda n: results[n]["cv_mae"])

    # 5. Which features matter? Permutation importance on the hold-out set:
    #    shuffle one column, measure how much the error grows.
    chosen_model = candidate_models()[chosen].fit(train_df[FEATURES], to_ratio(train_df))
    importance: dict[str, float] = {}
    if chosen not in ("trust_estimate", "mean_ratio"):
        perm = permutation_importance(chosen_model, hold_df[FEATURES], to_ratio(hold_df),
                                      scoring="neg_mean_absolute_error",
                                      n_repeats=5, random_state=42)
        importance = {f: float(v) for f, v in zip(FEATURES, perm.importances_mean)}

    # 6. Final model: refit on ALL data so production uses everything we know.
    final = candidate_models()[chosen].fit(df[FEATURES], to_ratio(df))
    report = TrainingReport(rows=len(df), train_rows=len(train_df),
                            holdout_rows=len(hold_df), chosen_model=chosen,
                            results=results, feature_importance=importance)
    predictor = DurationPredictor(final, chosen, datetime.now(), report)
    return predictor, report


# ---------------------------------------------------------- inference -----

class DurationPredictor:
    """A trained pipeline plus the logic to turn it into duration curves."""
```

```
    def __init__(self, pipeline: Pipeline, model_name: str,
                 trained_at: datetime, report: Optional[TrainingReport] = None):
        self.pipeline = pipeline
        self.model_name = model_name
        self.trained_at = trained_at
        self.report = report

    def save(self, path: Union[str, Path, None] = None) -> Path:
        path = Path(path or config.MODEL_PATH)
        path.parent.mkdir(parents=True, exist_ok=True)
        joblib.dump({"pipeline": self.pipeline, "model_name": self.model_name,
                     "trained_at": self.trained_at, "report": self.report,
                     "sklearn_version": sklearn.__version__}, path)
        return path

    @classmethod
    def load(cls, path: Union[str, Path, None] = None) -> Optional["DurationPredictor"]:
        path = Path(path or config.MODEL_PATH)
        if not path.exists():
            return None
        blob = joblib.load(path)
        return cls(blob["pipeline"], blob["model_name"], blob["trained_at"], blob["report"])

    def duration_curves(self, tasks: list[Task], day_of_week: int,
                        hour: int = DEFAULT_PLANNING_HOUR) -> list[list[float]]:
        """Predicted minutes for each task at each level in config.FATIGUE_LEVELS.

        All tasks x all fatigue levels go through the model in ONE predict()
        call (vectorised). Calling predict() once per task would be ~10x slower
        because every call has fixed pandas/scikit-learn overhead.
        """
        if not tasks:
            return []
        levels = config.FATIGUE_LEVELS
        rows = pd.DataFrame([{
            "category": t.category, "estimate_min": t.estimate_min,
            "difficulty": t.difficulty, "priority": t.priority,
            "fatigue_at_start": f, "hour_of_day": hour, "day_of_week": day_of_week,
        } for t in tasks for f in levels])
        minutes = _minutes_from_ratio(self.pipeline, rows).reshape(len(tasks), len(levels))
        # Domain knowledge: being MORE tired never makes you FASTER. Tree models
        # can produce small non-monotonic wiggles, so enforce a rising curve
        # (running maximum along each row).
        minutes = np.maximum(np.maximum.accumulate(minutes, axis=1), 1.0)
        return [[round(float(m), 2) for m in row] for row in minutes]

    def duration_curve(self, task: Task, day_of_week: int,
                       hour: int = DEFAULT_PLANNING_HOUR) -> list[float]:
        return self.duration_curves([task], day_of_week, hour)[0]


def heuristic_curve(task: Task) -> list[float]:
    """Cold-start fallback when no model is trained: +50% at full fatigue."""
    return [round(task.estimate_min * (1.0 + 0.5 * f), 2) for f in config.FATIGUE_LEVELS]
```

Notes, in the order `train()` runs

1. `load_training_frame`: `pd.read_sql_query(TRAINING_SQL, conn)` gives a DataFrame sorted by time.
2. Refuse to train on fewer than 40 rows (`InsufficientDataError`): a model trained on 10 rows is worse than the baseline and gives false confidence.
3. **Temporal split**: `iloc[:cut]` (old 80%) vs `iloc[cut:]` (new 20%).
4. For each of the five candidate pipelines: `TimeSeriesSplit` cross-validation on the training part (MAE **in minutes**, computed by converting predicted ratios back), then fit on the whole training part and score once on the hold-out.
5. **Choose by CV MAE**, never by hold-out score (that would leak test information into the choice).
6. **Permutation importance** on the hold-out set for the chosen model.

7. **Refit the chosen model on all data** for production use: once evaluation is done, more data beats a held-back slice. * `_minutes_from_ratio` clips ratios into `[0.2, 5.0]`, a guard rail against absurd predictions on weird inputs. * `duration_curves` builds one DataFrame with *tasks × 5 fatigue levels* rows and calls `predict` **once** (vectorised), then `reshape`s to a matrix. It then applies `np.maximum.accumulate(axis=1)` (running maximum along each row) to enforce the domain rule that **more fatigue never makes you faster**, since tree models can produce small non-monotonic wiggles. * `heuristic_curve` is the **cold-start** fallback (+50% at full fatigue) used before any model exists. * The saved file includes the scikit-learn version, because pickled models are not guaranteed to load across versions.

4.9 `engine_bridge.py`: talking to C++

File: `flexsched/engine_bridge.py`

```
"""Bridge between Python and the compiled C++ engine.

Integration style: subprocess + JSON over stdin/stdout.
    Python  --(JSON request on stdin)-->  flexsched_engine(.exe)
    Python  <--(JSON response on stdout)--

Alternatives considered (good to mention in interviews):
    * pybind11 extension module - fastest, but couples the build to one exact
      Python version/compiler and complicates packaging on Windows.
    * ctypes + C ABI shared library - no compile-time coupling, but manual
      memory management across the boundary and no type safety.
    * subprocess + JSON (chosen) - ~ms of overhead per call, irrelevant for
      a planner that runs a few times a day; in exchange the engine is a
      standalone tool that can be tested, debugged and replaced on its own.
"""

from __future__ import annotations

import json
import os
import shutil
import subprocess
from pathlib import Path
from typing import Optional

from . import config


class EngineError(RuntimeError):
    """The engine crashed or returned something unusable."""


class EngineInputError(EngineError):
    """The engine rejected our request (exit code 2)."""


class EngineNotFoundError(EngineError):
    pass


_EXE = "flexsched_engine.exe" if os.name == "nt" else "flexsched_engine"


def find_engine() -> Path:
    """Locate the engine binary: env var, then common build folders, then PATH."""
    override = os.environ.get("FLEXSCHED_ENGINE")
    candidates: list[Optional[Path]] = [Path(override)] if override else []
    build = config.PROJECT_ROOT / "build"
    candidates += [build / _EXE, build / "Release" / _EXE, build / "Debug" / _EXE]
    on_path = shutil.which("flexsched_engine")
```

```
    candidates.append(Path(on_path) if on_path else None)

    for path in candidates:
        if path is not None and path.is_file():
            return path
    raise EngineNotFoundError(
        "C++ engine not found. Build it first:\n"
        "    cmake -S engine -B build -G Ninja -DCMAKE_BUILD_TYPE=Release\n"
        "    cmake --build build\n"
        "or set FLEXSCHED_ENGINE to the path of flexsched_engine.")


def run_engine(request: dict, timeout: float = 30.0,
               engine_path: Optional[Path] = None) -> dict:
    """Send one request to the engine and return the parsed response."""
    exe = engine_path or find_engine()
    try:
        proc = subprocess.run(
            [str(exe)],
            input=json.dumps(request),
            capture_output=True,
            text=True,
            timeout=timeout,
            check=False,  # we inspect the return code ourselves
        )
    except subprocess.TimeoutExpired as exc:
        raise EngineError(f"engine timed out after {timeout}s") from exc

    if proc.returncode == 2:
        raise EngineInputError(proc.stderr.strip())
    if proc.returncode != 0:
        raise EngineError(f"engine failed (exit {proc.returncode}): {proc.stderr.strip()}")
    try:
        return json.loads(proc.stdout)
    except json.JSONDecodeError as exc:
        raise EngineError(f"engine returned invalid JSON: {proc.stdout[:200]!r}") from exc
```

Notes

- `find_engine` searches: env var → `build/` → `build/Release` (MSVC layout) → `build/Debug` → `PATH`. If nothing is found it raises a helpful error **with the build commands**.
- `subprocess.run([...], input=json, capture_output=True, text=True, timeout=30, check=False)`: pass a list (no shell, so no quoting or injection issues), feed stdin, capture stdout/stderr as text, never hang forever, and inspect the return code ourselves.
- Errors are translated into a small **exception hierarchy**, so callers handle "bad input" differently from "engine crashed".
- The docstring records the alternatives considered (pybind11, ctypes) and why subprocess + JSON won. That's an important interview topic (Part 7).
- Measured overhead: about 50 ms per call on Windows (mostly process start-up). That's negligible for a planner run by a human, and the reason the benchmark takes ~25 s for 400 engine calls.

4.10 `planner.py`: orchestration

File: `flexsched/planner.py`

```
"""Planner: the orchestration layer (service layer).

    SQLite (pending tasks)  --->  ML predictor (duration curves)
              |                              |
              +-------> engine request <-----+
                              |
                    C++ engine (schedule)
```

```
                              |
           datetimes, titles, persistence  <-+

It also owns ``log_session``: recording real work and reconstructing the
fatigue level at the start of each session (the ML feature).
"""

from __future__ import annotations

import sqlite3
from datetime import date, datetime, time, timedelta
from typing import Iterable, Optional

from . import config
from .config import SchedulerParams
from .engine_bridge import run_engine
from .fatigue import replay_day
from .models import PlanResult, PlannedBlock, Task, WorkSession
from .predictor import DurationPredictor, heuristic_curve
from .repository import ScheduleRepository, SessionRepository, TaskRepository

STRATEGIES = ("fatigue_aware", "edf")
NO_DEADLINE_PADDING = timedelta(days=30)


def minutes_between(origin: datetime, moment: datetime) -> int:
    return int((moment - origin).total_seconds() // 60)


def work_windows(start: date, days: int, day_start: time, day_end: time,
                 skip_weekends: bool = False) -> list[tuple[datetime, datetime]]:
    if day_end <= day_start:
        raise ValueError("day_end must be after day_start")
    windows = []
    for i in range(days):
        day = start + timedelta(days=i)
        if skip_weekends and day.weekday() >= 5:
            continue
        windows.append((datetime.combine(day, day_start), datetime.combine(day, day_end)))
    return windows


def build_request(tasks: list[Task], origin: datetime, horizon_end: datetime,
                  windows: list[tuple[datetime, datetime]],
                  blocked: list[tuple[datetime, datetime]],
                  strategy: str, params: SchedulerParams,
                  predictor: Optional[DurationPredictor]) -> dict:
    """Translate Python objects into the engine's JSON contract (minute offsets)."""
    no_deadline = minutes_between(origin, horizon_end + NO_DEADLINE_PADDING)
    curves = (predictor.duration_curves(tasks, day_of_week=_sqlite_weekday(origin))
              if predictor else [heuristic_curve(t) for t in tasks])
    engine_tasks = []
    for t, curve in zip(tasks, curves):
        engine_tasks.append({
            "id": t.id,
            "difficulty": t.difficulty,
            "priority": t.priority,
            "deadline": minutes_between(origin, t.deadline) if t.deadline else no_deadline,
            "duration_curve": curve,
        })
    return {
        "strategy": strategy,
        "params": params.to_dict(),
        "windows": [{"start": minutes_between(origin, a), "end": minutes_between(origin, b)}
                    for a, b in windows],
        "blocked": [{"start": minutes_between(origin, a), "end": minutes_between(origin, b)}
                    for a, b in blocked],
        "tasks": engine_tasks,
    }


def _sqlite_weekday(d: datetime) -> int:
```

```
    """SQLite's strftime('%w'): Sunday=0 .. Saturday=6 (Python: Monday=0)."""
    return (d.weekday() + 1) % 7


def plan(conn: sqlite3.Connection, start: date, days: int = 1,
         strategy: str = "fatigue_aware", day_start: time = time(9, 0),
         day_end: time = time(17, 0), blocked: Iterable[tuple[datetime, datetime]] = (),
         params: SchedulerParams = config.DEFAULT_PARAMS,
         predictor: Optional[DurationPredictor] = None, persist: bool = True,
         skip_weekends: bool = False) -> PlanResult:
    if strategy not in STRATEGIES:
        raise ValueError(f"strategy must be one of {STRATEGIES}")
    if days < 1:
        raise ValueError("days must be >= 1")

    task_repo = TaskRepository(conn)
    tasks = task_repo.list(["pending", "scheduled"])
    origin = datetime.combine(start, time(0, 0))
    horizon_end = origin + timedelta(days=days)
    windows = work_windows(start, days, day_start, day_end, skip_weekends)
    blocked = list(blocked)

    request = build_request(tasks, origin, horizon_end, windows, blocked,
                            strategy, params, predictor)
    response = run_engine(request)

    titles = {t.id: t.title for t in tasks}
    result = PlanResult(strategy=strategy, metrics=response["metrics"])
    for b in response["blocks"]:
        result.blocks.append(PlannedBlock(
            kind=b["kind"],
            start=origin + timedelta(minutes=b["start"]),
            end=origin + timedelta(minutes=b["end"]),
            fatigue_start=b["fatigue_start"],
            fatigue_end=b["fatigue_end"],
            task_id=b.get("task_id"),
            late=b.get("late", False),
            title=titles.get(b.get("task_id"), "Break"),
        ))
    result.unscheduled = [(u["task_id"], u["reason"]) for u in response["unscheduled"]]

    if persist:
        ScheduleRepository(conn).save(result, origin, days)
        task_repo.set_status([b.task_id for b in result.blocks if b.kind == "task"],
                             "scheduled")
        task_repo.set_status([tid for tid, _ in result.unscheduled], "pending")
    return result


def compare(conn: sqlite3.Connection, start: date, days: int = 1,
            **kwargs) -> dict[str, PlanResult]:
    """Run every strategy on the same tasks without saving anything."""
    kwargs["persist"] = False
    return {s: plan(conn, start, days, strategy=s, **kwargs) for s in STRATEGIES}


def log_session(conn: sqlite3.Connection, task_id: int, started_at: datetime,
                minutes: int, mark_done: bool = True,
                params: SchedulerParams = config.DEFAULT_PARAMS) -> WorkSession:
    """Record real work and refresh fatigue_at_start for that whole day.

    Recomputing the entire day (not just the new row) keeps the data correct
    even if sessions are logged out of order.
    """
    task_repo = TaskRepository(conn)
    if task_repo.get(task_id) is None:
        raise ValueError(f"task {task_id} does not exist")
    if minutes <= 0:
        raise ValueError("minutes must be positive")

    sessions = SessionRepository(conn)
    session = sessions.add(WorkSession(task_id=task_id, started_at=started_at,
```

```
                                    ended_at=started_at + timedelta(minutes=minutes)),
                          commit=False)
    day_rows = sessions.for_day(started_at.date())
    fatigue = replay_day([(s.started_at, s.ended_at, diff) for s, diff in day_rows], params)
    conn.executemany("UPDATE work_sessions SET fatigue_at_start = ? WHERE id = ?",
                     [(round(f, 6), s.id) for (s, _), f in zip(day_rows, fatigue)])
    if mark_done:
        conn.execute("UPDATE tasks SET status = 'done' WHERE id = ?", (task_id,))
    conn.commit()
    fatigue_by_id = {s.id: f for (s, _), f in zip(day_rows, fatigue)}
    session.fatigue_at_start = fatigue_by_id[session.id]
    return session
```

Notes

- `work_windows` builds one `(start, end)` per day; `skip_weekends` is optional.
- `build_request` is **pure** (no I/O): objects in, dict out. That makes it easy to test and to reuse from `benchmark.py`.
- Tasks without a deadline get "horizon end + 30 days": far enough to be non-urgent, but still a finite integer for the engine.
- `plan()` maps engine output back to datetimes, saves the run (unless `persist=False`) and updates statuses: scheduled tasks become `scheduled`, unscheduled return to `pending`.
- `compare()` reuses `plan()` with `persist=False` for every strategy on identical input, a fair A/B comparison.
- `log_session` inserts the session **without committing**, re-reads the whole day (the same connection sees its own uncommitted rows), replays fatigue for **every** session of that day and updates them, then commits once. Logging sessions out of order still yields correct fatigue values, and a test logs the later session first to prove it.

4.11 `benchmark.py`: is fatigue-aware actually better?

File: `flexsched/benchmark.py`

```
"""Strategy benchmark: is fatigue-aware scheduling actually better than EDF?

One example proves nothing, so we generate many random-but-realistic
workloads, schedule each with BOTH strategies on identical input, and
compare the averages. Same seed -> same scenarios -> reproducible numbers.

Each scenario: 8-14 tasks, 2 working days (09:00-17:00) with a lunch block,
deadlines spread over the horizon, durations from the duration predictor
(or the heuristic curve if no model is trained).
"""

from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import Optional

import numpy as np

from . import config
from .engine_bridge import run_engine
from .models import Task
from .planner import STRATEGIES, build_request
from .predictor import DurationPredictor

METRIC_KEYS = ("total_work_minutes", "makespan", "tasks_scheduled", "tasks_late",
               "total_lateness", "breaks", "peak_fatigue", "avg_fatigue_at_start")
```

```python
@dataclass
class BenchmarkResult:
    scenarios: int
    means: dict[str, dict[str, float]]          # strategy -> metric -> mean
    paired_wins: dict[str, float]               # metric -> share of scenarios FA better


def random_tasks(rng: np.random.Generator, n: int, origin: datetime,
                 horizon_days: int) -> list[Task]:
    tasks = []
    for i in range(n):
        deadline_day = int(rng.integers(0, horizon_days + 1))
        deadline = origin + timedelta(days=deadline_day, hours=int(rng.integers(11, 18)))
        tasks.append(Task(
            id=i + 1,
            title=f"task {i + 1}",
            category=str(rng.choice(config.CATEGORIES)),
            difficulty=int(rng.integers(1, 6)),
            priority=int(rng.integers(1, 6)),
            estimate_min=int(rng.choice([20, 30, 45, 60, 90])),
            deadline=deadline if rng.random() < 0.8 else None,  # 20% have no deadline
        ))
    return tasks


def run_benchmark(scenarios: int = 100, seed: int = 7,
                  predictor: Optional[DurationPredictor] = None) -> BenchmarkResult:
    rng = np.random.default_rng(seed)
    origin = datetime(2030, 1, 7)  # a Monday; any fixed date keeps runs reproducible
    days = 2
    windows = [(origin + timedelta(days=d, hours=9), origin + timedelta(days=d, hours=17))
               for d in range(days)]
    blocked = [(origin + timedelta(days=d, hours=13), origin + timedelta(days=d, hours=13.75))
               for d in range(days)]

    per_strategy: dict[str, list[dict]] = {s: [] for s in STRATEGIES}
    for _ in range(scenarios):
        tasks = random_tasks(rng, int(rng.integers(8, 15)), origin, days)
        for strategy in STRATEGIES:
            request = build_request(tasks, origin, origin + timedelta(days=days), windows,
                                    blocked, strategy, config.DEFAULT_PARAMS, predictor)
            per_strategy[strategy].append(run_engine(request)["metrics"])

    means = {s: {k: float(np.mean([m[k] for m in rows])) for k in METRIC_KEYS}
             for s, rows in per_strategy.items()}

    # Paired comparison: in what share of scenarios is fatigue-aware strictly better?
    lower_is_better = ("total_work_minutes", "total_lateness", "tasks_late",
                       "avg_fatigue_at_start", "peak_fatigue")
    wins = {}
    for k in lower_is_better:
        fa = np.array([m[k] for m in per_strategy["fatigue_aware"]])
        edf = np.array([m[k] for m in per_strategy["edf"]])
        wins[k] = float(np.mean(fa < edf))
    fa_sched = np.array([m["tasks_scheduled"] for m in per_strategy["fatigue_aware"]])
    edf_sched = np.array([m["tasks_scheduled"] for m in per_strategy["edf"]])
    wins["tasks_scheduled"] = float(np.mean(fa_sched > edf_sched))
    return BenchmarkResult(scenarios=scenarios, means=means, paired_wins=wins)
```

Notes

- One example proves nothing; 200 seeded random scenarios give stable averages.
- **Paired comparison**: both strategies get the *same* tasks and calendar per scenario, so differences come from the policy alone. `paired_wins` reports the share of scenarios where fatigue-aware is strictly better per metric.
- A fixed date (`2030-01-07` , a Monday) keeps runs reproducible regardless of today's date.

4.12 `cli.py` and `__main__.py`: the user interface

File: `flexsched/cli.py`

```python
"""Command-line interface (argparse, standard library).

    python -m flexsched <command> [options]

Commands: init, add, list, done, seed, train, stats, plan, compare, runs,
benchmark, demo. Run ``python -m flexsched <command> -h`` for details.
"""

from __future__ import annotations

import argparse
import json
import sys
import tempfile
from datetime import date, datetime, time, timedelta
from pathlib import Path
from typing import Optional, Sequence

from . import __version__, config
from .benchmark import METRIC_KEYS, run_benchmark
from .db import connect
from .engine_bridge import EngineError
from .models import PlanResult, Task
from .planner import STRATEGIES, compare, log_session, plan
from .predictor import DurationPredictor, InsufficientDataError, train
from .repository import ScheduleRepository, SessionRepository, TaskRepository
from .synthetic import generate_history


# ---------------------------------------------------------------- parsing -----

def parse_datetime(text: str) -> datetime:
    for fmt in ("%Y-%m-%d %H:%M", "%Y-%m-%dT%H:%M", "%Y-%m-%d"):
        try:
            return datetime.strptime(text, fmt)
        except ValueError:
            continue
    raise argparse.ArgumentTypeError(f"invalid date/time {text!r} (use 'YYYY-MM-DD HH:MM')")


def parse_date(text: str) -> date:
    if text == "today":
        return date.today()
    if text == "tomorrow":
        return date.today() + timedelta(days=1)
    return parse_datetime(text).date()


def parse_clock(text: str) -> time:
    try:
        return datetime.strptime(text, "%H:%M").time()
    except ValueError as exc:
        raise argparse.ArgumentTypeError(f"invalid time {text!r} (use HH:MM)") from exc


def parse_block(text: str) -> tuple[datetime, datetime]:
    """'2026-10-07 13:00-14:00' -> (start, end)."""
    try:
        day_part, times = text.rsplit(" ", 1)
        start_s, end_s = times.split("-")
        start = parse_datetime(f"{day_part} {start_s}")
        end = parse_datetime(f"{day_part} {end_s}")
    except ValueError as exc:
        raise argparse.ArgumentTypeError(
            f"invalid block {text!r} (use 'YYYY-MM-DD HH:MM-HH:MM')") from exc
```

```
    if end <= start:
        raise argparse.ArgumentTypeError("block must end after it starts")
    return start, end


def int_range(lo: int, hi: int):
    def check(text: str) -> int:
        value = int(text)
        if not lo <= value <= hi:
            raise argparse.ArgumentTypeError(f"must be between {lo} and {hi}")
        return value
    return check


# ---------------------------------------------------------- printing -----

def table(headers: Sequence[str], rows: Sequence[Sequence[object]]) -> str:
    """Render a plain-text table with aligned columns."""
    cells = [[str(h) for h in headers]] + [[str(c) for c in r] for r in rows]
    widths = [max(len(r[i]) for r in cells) for i in range(len(headers))]
    lines = ["  ".join(c.ljust(w) for c, w in zip(r, widths)) for r in cells]
    lines.insert(1, "  ".join("-" * w for w in widths))
    return "\n".join(lines)


def fatigue_bar(f: float, width: int = 10) -> str:
    filled = int(round(f * width))
    return "#" * filled + "." * (width - filled)


def print_plan(result: PlanResult) -> None:
    rows = []
    for b in result.blocks:
        rows.append([
            b.start.strftime("%a %d %b"),
            f"{b.start:%H:%M}-{b.end:%H:%M}",
            "--- break ---" if b.kind == "break" else f"[{b.task_id}] {b.title}"[:40],
            f"{fatigue_bar(b.fatigue_start)} {b.fatigue_start:.2f}",
            "LATE" if b.late else "",
        ])
    print(f"\nStrategy: {result.strategy}"
          + (f"   (saved as run #{result.run_id})" if result.run_id else ""))
    print(table(["Day", "Time", "What", "Fatigue at start", ""], rows) if rows
          else "(nothing scheduled)")
    for task_id, reason in result.unscheduled:
        print(f"  ! task {task_id} not scheduled: {reason}")
    m = result.metrics
    print(f"\n  tasks: {m['tasks_scheduled']} scheduled, {m['tasks_late']} late "
          f"({m['total_lateness']} min total) | work: {m['total_work_minutes']} min | "
          f"breaks: {m['breaks']} | avg fatigue at start: {m['avg_fatigue_at_start']:.2f} | "
          f"peak: {m['peak_fatigue']:.2f}")


def print_comparison(results: dict[str, PlanResult]) -> None:
    rows = [[k] + [_fmt(results[s].metrics[k]) for s in STRATEGIES] for k in METRIC_KEYS]
    print(table(["metric"] + list(STRATEGIES), rows))


def _fmt(v: object) -> str:
    return f"{v:.3f}" if isinstance(v, float) else str(v)


# ---------------------------------------------------------- commands -----

def cmd_init(args, conn) -> None:
    print(f"Database ready at {args.db}")


def cmd_add(args, conn) -> None:
    task = TaskRepository(conn).add(Task(
        title=args.title, category=args.category, difficulty=args.difficulty,
        priority=args.priority, estimate_min=args.estimate, deadline=args.deadline))
```

```
    print(f"Added task #{task.id}: {task.title}")


def cmd_list(args, conn) -> None:
    statuses = None if args.all else ["pending", "scheduled"]
    tasks = TaskRepository(conn).list(statuses)
    if args.limit:
        tasks = tasks[-args.limit:]
    print(table(["id", "title", "category", "diff", "prio", "est", "deadline", "status"],
                [[t.id, t.title[:36], t.category, t.difficulty, t.priority, t.estimate_min,
                  f"{t.deadline:%Y-%m-%d %H:%M}" if t.deadline else "-", t.status]
                 for t in tasks]))


def cmd_done(args, conn) -> None:
    session = log_session(conn, args.task_id, args.start, args.minutes)
    print(f"Logged {args.minutes} min on task #{args.task_id} "
          f"(fatigue at start reconstructed as {session.fatigue_at_start:.2f})")


def cmd_seed(args, conn) -> None:
    n = generate_history(conn, days=args.days, seed=args.seed)
    print(f"Generated {n} synthetic work sessions over {args.days} days (seed {args.seed})")


def cmd_train(args, conn) -> None:
    predictor, report = train(conn)
    path = predictor.save(args.model)
    print(f"Trained on {report.rows} sessions "
          f"({report.train_rows} train / {report.holdout_rows} hold-out, time-ordered)\n")
    rows = [[name, f"{r['cv_mae']:.2f}", f"{r['holdout_mae']:.2f}",
             f"{100 * r['holdout_mape']:.1f}%", f"{r['holdout_r2']:.3f}",
             "<- chosen" if name == report.chosen_model else ""]
            for name, r in report.results.items()]
    print(table(["model", "CV MAE (min)", "hold-out MAE", "MAPE", "R^2", ""], rows))
    if report.feature_importance:
        print("\nPermutation importance (increase in error when a feature is shuffled):")
        for feat, val in sorted(report.feature_importance.items(), key=lambda kv: -kv[1]):
            print(f"  {feat:18s} {val:+.4f}")
    print(f"\nModel saved to {path}")


def cmd_stats(args, conn) -> None:
    rows = SessionRepository(conn).estimation_bias()
    if not rows:
        print("No work sessions logged yet.")
        return
    print("How long things REALLY take compared with your estimates:\n")
    print(table(["category", "sessions", "avg estimate", "avg actual", "actual/estimate"],
                [[r["category"], r["sessions"], r["avg_estimate"], r["avg_actual"],
                  r["avg_ratio"]] for r in rows]))


def _plan_kwargs(args) -> dict:
    return dict(day_start=args.day_start, day_end=args.day_end, blocked=args.block or [],
                predictor=DurationPredictor.load(args.model),
                skip_weekends=args.skip_weekends)


def cmd_plan(args, conn) -> None:
    kwargs = _plan_kwargs(args)
    if kwargs["predictor"] is None:
        print("(no trained model found - using the heuristic duration curve; "
              "run 'train' for ML predictions)")
    result = plan(conn, args.start, args.days, strategy=args.strategy,
                  persist=not args.dry_run, **kwargs)
    print_plan(result)


def cmd_compare(args, conn) -> None:
    results = compare(conn, args.start, args.days, **_plan_kwargs(args))
    print_comparison(results)
```

```
def cmd_runs(args, conn) -> None:
    rows = []
    for r in ScheduleRepository(conn).runs(args.limit):
        m = json.loads(r["metrics_json"])
        rows.append([r["id"], r["created_at"], r["strategy"], r["horizon_start"][:10],
                     r["horizon_days"], m.get("tasks_scheduled"), m.get("tasks_late"),
                     r["items"]])
    print(table(["run", "created", "strategy", "from", "days", "scheduled", "late", "items"],
                rows))


def cmd_benchmark(args, conn) -> None:
    predictor = DurationPredictor.load(args.model)
    print(f"Running {args.scenarios} random scenarios with both strategies "
          f"({'ML model' if predictor else 'heuristic durations'})...")
    result = run_benchmark(args.scenarios, args.seed, predictor)
    rows = [[k] + [f"{result.means[s][k]:.3f}" for s in STRATEGIES] for k in METRIC_KEYS]
    print(table(["metric (mean)"] + list(STRATEGIES), rows))
    print("\nShare of scenarios where fatigue_aware is strictly better than edf:")
    for k, v in result.paired_wins.items():
        print(f"  {k:22s} {100 * v:5.1f}%")


DEMO_TASKS = [
    # title, category, difficulty, priority, estimate, deadline (day offset, hour)
    ("Implement auth module", "coding", 5, 5, 120, (1, 17)),
    ("Fix flaky integration test", "coding", 4, 4, 60, (0, 15)),
    ("Read scheduling paper", "study", 4, 2, 90, (2, 17)),
    ("Reply to project emails", "admin", 1, 3, 20, (0, 12)),
    ("Write sprint report", "writing", 3, 4, 45, (1, 12)),
    ("Review teammate PR", "review", 2, 4, 30, (0, 17)),
    ("Design database schema", "design", 4, 3, 75, (2, 12)),
    ("Update documentation", "writing", 2, 2, 40, None),
    ("Plan next week", "admin", 2, 2, 25, None),
    ("Practice DSA problems", "study", 5, 3, 60, (2, 17)),
]


def cmd_demo(args, _conn) -> None:
    """End-to-end tour on a throw-away database: seed -> train -> plan -> compare."""
    workdir = Path(tempfile.mkdtemp(prefix="flexsched_demo_"))
    conn = connect(workdir / "demo.db")
    model_path = workdir / "model.joblib"
    start = date.today() + timedelta(days=1)

    print("== 1. Generating 120 days of synthetic work history ==")
    print(f"   {generate_history(conn, days=120, seed=42)} sessions")

    print("\n== 2. Training duration models ==")
    predictor, report = train(conn)
    predictor.save(model_path)
    best = report.results[report.chosen_model]
    naive = report.results["trust_estimate"]
    print(f"   chosen: {report.chosen_model} | hold-out MAE {best['holdout_mae']:.1f} min "
          f"vs {naive['holdout_mae']:.1f} min when trusting your own estimate")

    print("\n== 3. Adding this week's tasks ==")
    repo = TaskRepository(conn)
    origin = datetime.combine(start, time(0, 0))
    for title, cat, diff, prio, est, dl in DEMO_TASKS:
        deadline = origin + timedelta(days=dl[0], hours=dl[1]) if dl else None
        repo.add(Task(title, cat, diff, prio, est, deadline))
    print(f"   {len(DEMO_TASKS)} tasks added")

    blocked = [(origin + timedelta(hours=13), origin + timedelta(hours=14)),
               (origin + timedelta(days=1, hours=10), origin + timedelta(days=1, hours=11))]
    print("\n== 4. Comparing strategies (same tasks, same calendar) ==")
    print_comparison(compare(conn, start, 3, blocked=blocked, predictor=predictor))

    print("\n== 5. The fatigue-aware plan ==")
```

```
    print_plan(plan(conn, start, 3, blocked=blocked, predictor=predictor))
    print(f"\n(demo files in {workdir})")


# ---------------------------------------------------------- parser -----

def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="flexsched",
                                description="Fatigue-aware task scheduler and planner")
    p.add_argument("--version", action="version", version=f"%(prog)s {__version__}")
    p.add_argument("--db", type=Path, default=config.DB_PATH, help="SQLite database file")
    p.add_argument("--model", type=Path, default=config.MODEL_PATH, help="trained model file")
    sub = p.add_subparsers(dest="command", required=True)

    sub.add_parser("init", help="create the database").set_defaults(func=cmd_init)

    a = sub.add_parser("add", help="add a task")
    a.add_argument("title")
    a.add_argument("--category", "-c", choices=config.CATEGORIES, required=True)
    a.add_argument("--difficulty", "-d", type=int_range(1, 5), default=3)
    a.add_argument("--priority", "-p", type=int_range(1, 5), default=3)
    a.add_argument("--estimate", "-e", type=int_range(1, 24 * 60), required=True,
                   help="your estimate in minutes")
    a.add_argument("--deadline", type=parse_datetime, help="'YYYY-MM-DD HH:MM'")
    a.set_defaults(func=cmd_add)

    l = sub.add_parser("list", help="list open tasks")
    l.add_argument("--all", action="store_true", help="include done/cancelled tasks")
    l.add_argument("--limit", type=int, default=0, help="show only the last N")
    l.set_defaults(func=cmd_list)

    d = sub.add_parser("done", help="log real work on a task (and mark it done)")
    d.add_argument("task_id", type=int)
    d.add_argument("--start", type=parse_datetime, required=True, help="'YYYY-MM-DD HH:MM'")
    d.add_argument("--minutes", type=int_range(1, 24 * 60), required=True)
    d.set_defaults(func=cmd_done)

    s = sub.add_parser("seed", help="generate synthetic history for demos")
    s.add_argument("--days", type=int_range(1, 3650), default=120)
    s.add_argument("--seed", type=int, default=42)
    s.set_defaults(func=cmd_seed)

    sub.add_parser("train", help="train the duration model").set_defaults(func=cmd_train)
    sub.add_parser("stats", help="estimation bias per category").set_defaults(func=cmd_stats)

    for name, func, help_text in (("plan", cmd_plan, "build and save a schedule"),
                                  ("compare", cmd_compare, "compare strategies (no save)")):
        q = sub.add_parser(name, help=help_text)
        q.add_argument("--start", type=parse_date, default="tomorrow",
                       help="first day: YYYY-MM-DD, 'today' or 'tomorrow'")
        q.add_argument("--days", type=int_range(1, 60), default=1)
        q.add_argument("--day-start", type=parse_clock, default=time(9, 0))
        q.add_argument("--day-end", type=parse_clock, default=time(17, 0))
        q.add_argument("--block", type=parse_block, action="append",
                       help="busy time, repeatable: 'YYYY-MM-DD HH:MM-HH:MM'")
        q.add_argument("--skip-weekends", action="store_true")
        if name == "plan":
            q.add_argument("--strategy", choices=STRATEGIES, default="fatigue_aware")
            q.add_argument("--dry-run", action="store_true", help="do not save the plan")
        q.set_defaults(func=func)

    r = sub.add_parser("runs", help="show saved schedule runs")
    r.add_argument("--limit", type=int, default=10)
    r.set_defaults(func=cmd_runs)

    b = sub.add_parser("benchmark", help="compare strategies on random scenarios")
    b.add_argument("--scenarios", type=int_range(1, 10000), default=100)
    b.add_argument("--seed", type=int, default=7)
    b.set_defaults(func=cmd_benchmark)

    sub.add_parser("demo", help="end-to-end demo on a temporary database"
                   ).set_defaults(func=cmd_demo)
```

```
    return p


def main(argv: Optional[Sequence[str]] = None) -> int:
    args = build_parser().parse_args(argv)
    conn = None if args.command == "demo" else connect(args.db)
    try:
        args.func(args, conn)
        return 0
    except (ValueError, InsufficientDataError, EngineError) as exc:
        print(f"error: {exc}", file=sys.stderr)
        return 1
    finally:
        if conn is not None:
            conn.close()
```

File: `flexsched/__main__.py`

```
"""Allows ``python -m flexsched ...``."""

import sys

from .cli import main

sys.exit(main())
```

Notes

- `argparse` sub-commands: each `sub.add_parser(...)` gets its own options and a `set_defaults(func=cmd_x)`, so `main()` just calls `args.func(args, conn)`. That's a simple **command dispatch** table.
- **Custom `type=` converters** (`parse_datetime`, `parse_block`, `int_range(1, 5)`) validate input at the boundary and raise `ArgumentTypeError`, so argparse prints a clean usage error.
- `int_range` is a **closure**: a function that returns a function remembering `lo` / `hi`.
- `main()` returns an exit code (0/1) and closes the DB in `finally`. Known errors are printed to **stderr** without a traceback.
- `table()` is a ~6-line column aligner, so there's no dependency for pretty output.
- `demo` runs everything on a temporary database so your real data is never touched.

4.13 pytest suite

`conftest.py`: shared fixtures

File: `tests/conftest.py`

```
"""Shared pytest fixtures.

A fixture is a function pytest runs before a test and passes in by name:
``def test_x(conn): ...`` receives a fresh in-memory database.
"""

from __future__ import annotations

import pytest

from flexsched.db import connect
from flexsched.engine_bridge import EngineNotFoundError, find_engine
from flexsched.synthetic import generate_history
```

```
@pytest.fixture
def conn():
    connection = connect(":memory:")  # brand-new, isolated DB for every test
    yield connection
    connection.close()


@pytest.fixture(scope="module")
def history_conn():
    """A database with 90 days of synthetic history (built once per module)."""
    connection = connect(":memory:")
    generate_history(connection, days=90, seed=1)
    yield connection
    connection.close()


def _engine_missing() -> bool:
    try:
        find_engine()
        return False
    except EngineNotFoundError:
        return True


requires_engine = pytest.mark.skipif(_engine_missing(),
                                     reason="C++ engine not built (see README)")
```

`test_repository.py`

File: `tests/test_repository.py`

```
import sqlite3
from datetime import datetime, timedelta

import pytest

from flexsched.models import PlanResult, PlannedBlock, Task, WorkSession
from flexsched.repository import ScheduleRepository, SessionRepository, TaskRepository


def make_task(**overrides) -> Task:
    fields = dict(title="Write tests", category="coding", difficulty=3, priority=3,
                  estimate_min=45)
    fields.update(overrides)
    return Task(**fields)


def test_add_and_get_round_trip(conn):
    repo = TaskRepository(conn)
    deadline = datetime(2026, 10, 8, 17, 0)
    task = repo.add(make_task(deadline=deadline))
    assert task.id is not None
    loaded = repo.get(task.id)
    assert loaded.title == "Write tests"
    assert loaded.deadline == deadline
    assert loaded.status == "pending"
    assert loaded.created_at is not None


def test_get_missing_returns_none(conn):
    assert TaskRepository(conn).get(999) is None


@pytest.mark.parametrize("field,value", [
    ("title", "   "), ("category", "gaming"), ("difficulty", 0), ("difficulty", 6),
    ("priority", 9), ("estimate_min", 0), ("status", "archived"),
])
def test_validation_rejects_bad_tasks(conn, field, value):
```

```
    with pytest.raises(ValueError):
        TaskRepository(conn).add(make_task(**{field: value}))


def test_database_constraints_are_a_second_line_of_defence(conn):
    # Bypass the Python validation: the CHECK constraint must still refuse it.
    with pytest.raises(sqlite3.IntegrityError):
        conn.execute("INSERT INTO tasks (title, category, difficulty, priority, estimate_min)"
                     " VALUES ('x', 'coding', 7, 3, 30)")


def test_foreign_keys_are_enforced(conn):
    with pytest.raises(sqlite3.IntegrityError):
        conn.execute("INSERT INTO work_sessions (task_id, started_at, ended_at, actual_min)"
                     " VALUES (12345, '2026-01-01 09:00:00', '2026-01-01 10:00:00', 60)")


def test_list_filters_by_status(conn):
    repo = TaskRepository(conn)
    a = repo.add(make_task(title="a"))
    b = repo.add(make_task(title="b"))
    repo.set_status([b.id], "done")
    assert [t.title for t in repo.list(["pending"])] == ["a"]
    assert {t.title for t in repo.list()} == {"a", "b"}
    assert repo.list(["cancelled"]) == []
    assert a.id < b.id


def test_session_requires_positive_duration(conn):
    task = TaskRepository(conn).add(make_task())
    start = datetime(2026, 10, 1, 9, 0)
    with pytest.raises(ValueError):
        SessionRepository(conn).add(WorkSession(task.id, start, start))


def test_for_day_returns_only_that_day_in_order(conn):
    task = TaskRepository(conn).add(make_task())
    repo = SessionRepository(conn)
    day = datetime(2026, 10, 1)
    for hour in (14, 9, 11):
        s = day + timedelta(hours=hour)
        repo.add(WorkSession(task.id, s, s + timedelta(minutes=30)))
    other = day + timedelta(days=1, hours=9)
    repo.add(WorkSession(task.id, other, other + timedelta(minutes=30)))

    rows = repo.for_day(day.date())
    assert [s.started_at.hour for s, _ in rows] == [9, 11, 14]
    assert all(diff == 3 for _, diff in rows)


def test_estimation_bias_groups_by_category(conn):
    tasks = TaskRepository(conn)
    sessions = SessionRepository(conn)
    start = datetime(2026, 10, 1, 9, 0)
    coding = tasks.add(make_task(category="coding", estimate_min=60))
    admin = tasks.add(make_task(category="admin", estimate_min=60))
    sessions.add(WorkSession(coding.id, start, start + timedelta(minutes=90)))
    sessions.add(WorkSession(admin.id, start, start + timedelta(minutes=30)))
    rows = {r["category"]: r for r in sessions.estimation_bias()}
    assert rows["coding"]["avg_ratio"] == pytest.approx(1.5)
    assert rows["admin"]["avg_ratio"] == pytest.approx(0.5)


def test_schedule_save_is_atomic_and_complete(conn):
    task = TaskRepository(conn).add(make_task())
    t0 = datetime(2026, 10, 7, 9, 0)
    plan = PlanResult(strategy="edf", metrics={"tasks_scheduled": 1}, blocks=[
        PlannedBlock("task", t0, t0 + timedelta(minutes=45), 0.0, 0.3, task_id=task.id),
        PlannedBlock("break", t0 + timedelta(minutes=45), t0 + timedelta(minutes=60), 0.3, 0.1),
    ])
    run_id = ScheduleRepository(conn).save(plan, t0, 1)
    assert plan.run_id == run_id
```

```
    items = conn.execute("SELECT kind, task_id FROM schedule_items WHERE run_id = ?",
                         (run_id,)).fetchall()
    assert [(r["kind"], r["task_id"]) for r in items] == [("task", task.id), ("break", None)]
    runs = ScheduleRepository(conn).runs()
    assert runs[0]["items"] == 2


def test_schedule_save_rolls_back_on_error(conn):
    t0 = datetime(2026, 10, 7, 9, 0)
    bad = PlanResult(strategy="edf", blocks=[
        PlannedBlock("not-a-kind", t0, t0 + timedelta(minutes=5), 0.0, 0.0)])
    with pytest.raises(sqlite3.IntegrityError):
        ScheduleRepository(conn).save(bad, t0, 1)
    # The run row inserted before the failure must have been rolled back too.
    assert conn.execute("SELECT COUNT(*) FROM schedule_runs").fetchone()[0] == 0


def test_schema_version_is_recorded(conn):
    assert conn.execute("PRAGMA user_version").fetchone()[0] == 1
```

`test_fatigue.py`

File: `tests/test_fatigue.py`

```
import math
from datetime import datetime, timedelta

import pytest

from flexsched.fatigue import after_rest, after_work, replay_day


def test_work_formula_matches_documentation():
    assert after_work(0.0, 5, 60) == pytest.approx(1 - math.exp(-0.72))


def test_rest_formula_matches_documentation():
    assert after_rest(0.8, 15) == pytest.approx(0.8 * math.exp(-0.75))


def test_values_stay_in_unit_interval():
    assert after_work(0.9, 5, 10_000) <= 1.0
    assert after_rest(1.0, 10_000) >= 0.0
    assert after_work(3.0, 3, 5) <= 1.0
    assert after_rest(-1.0, 5) == 0.0


def test_harder_work_is_more_tiring():
    assert after_work(0.0, 5, 60) > after_work(0.0, 1, 60)


def test_replay_day_starts_fresh_and_accumulates():
    day = datetime(2026, 10, 1, 9, 0)
    sessions = [
        (day, day + timedelta(minutes=60), 5),
        (day + timedelta(minutes=60), day + timedelta(minutes=120), 5),   # no gap
        (day + timedelta(minutes=180), day + timedelta(minutes=200), 1),  # 60 min gap
    ]
    f = replay_day(sessions)
    assert f[0] == 0.0
    assert f[1] == pytest.approx(after_work(0.0, 5, 60))
    expected_third = after_rest(after_work(f[1], 5, 60), 60)
    assert f[2] == pytest.approx(expected_third)


def test_replay_empty_day():
    assert replay_day([]) == []
```

`test_predictor.py`

File: `tests/test_predictor.py`

```python
import pytest

from flexsched.db import connect
from flexsched.models import Task
from flexsched.predictor import (DurationPredictor, InsufficientDataError, heuristic_curve,
                                 load_training_frame, train)
from flexsched.synthetic import generate_history


@pytest.fixture(scope="module")
def trained(history_conn):
    return train(history_conn)


def test_training_frame_has_features_and_is_time_ordered(history_conn):
    df = load_training_frame(history_conn)
    assert {"category", "estimate_min", "fatigue_at_start", "hour_of_day",
            "day_of_week", "actual_min"} <= set(df.columns)
    assert df["started_at"].is_monotonic_increasing
    assert df["hour_of_day"].between(0, 23).all()


def test_too_little_data_raises(conn):
    generate_history(conn, days=3, seed=0)
    with pytest.raises(InsufficientDataError):
        train(conn)


def test_learned_model_beats_naive_baselines(trained):
    _, report = trained
    chosen = report.results[report.chosen_model]
    assert report.chosen_model not in ("trust_estimate", "mean_ratio")
    assert chosen["holdout_mae"] < report.results["trust_estimate"]["holdout_mae"]
    assert chosen["holdout_mae"] < report.results["mean_ratio"]["holdout_mae"]


def test_split_is_80_20(trained):
    _, report = trained
    assert report.train_rows + report.holdout_rows == report.rows
    assert report.holdout_rows == pytest.approx(0.2 * report.rows, abs=1)


def test_fatigue_and_difficulty_are_learned_as_important(trained):
    _, report = trained
    imp = report.feature_importance
    assert imp["fatigue_at_start"] > imp["day_of_week"]
    assert imp["difficulty"] > imp["day_of_week"]


def test_duration_curve_is_monotonic_and_positive(trained):
    predictor, _ = trained
    task = Task("x", "coding", 4, 3, 60)
    curve = predictor.duration_curve(task, day_of_week=2)
    assert len(curve) == 5
    assert all(v > 0 for v in curve)
    assert all(a <= b for a, b in zip(curve, curve[1:]))
    assert curve[-1] > curve[0]  # tired takes longer


def test_batch_prediction_matches_single(trained):
    predictor, _ = trained
    tasks = [Task("a", "admin", 1, 2, 20), Task("b", "design", 5, 5, 120)]
    batch = predictor.duration_curves(tasks, day_of_week=3)
    assert batch == [predictor.duration_curve(t, day_of_week=3) for t in tasks]
    assert predictor.duration_curves([], day_of_week=3) == []
```

```python
def test_model_save_and_load_round_trip(trained, tmp_path):
    predictor, _ = trained
    path = predictor.save(tmp_path / "m.joblib")
    loaded = DurationPredictor.load(path)
    task = Task("x", "study", 3, 3, 45)
    assert loaded.model_name == predictor.model_name
    assert loaded.duration_curve(task, 1) == predictor.duration_curve(task, 1)


def test_loading_missing_model_returns_none(tmp_path):
    assert DurationPredictor.load(tmp_path / "nope.joblib") is None


def test_heuristic_curve_shape():
    assert heuristic_curve(Task("x", "admin", 1, 1, 40)) == [40.0, 45.0, 50.0, 55.0, 60.0]


def test_synthetic_generation_is_reproducible():
    a, b = connect(":memory:"), connect(":memory:")
    generate_history(a, days=10, seed=5, end_day=None)
    generate_history(b, days=10, seed=5, end_day=None)
    q = "SELECT actual_min, fatigue_at_start FROM work_sessions ORDER BY id"
    assert [tuple(r) for r in a.execute(q)] == [tuple(r) for r in b.execute(q)]
```

`test_engine_integration.py`

File: `tests/test_engine_integration.py`

```python
"""Tests that talk to the real compiled C++ engine via the JSON bridge."""

from datetime import date, datetime, time, timedelta

import pytest

from conftest import requires_engine
from flexsched.config import DEFAULT_PARAMS
from flexsched.engine_bridge import EngineInputError, run_engine
from flexsched.fatigue import after_rest, after_work
from flexsched.models import Task
from flexsched.planner import build_request, compare, log_session, plan, work_windows
from flexsched.repository import TaskRepository

pytestmark = requires_engine  # skip the whole module if the engine is not built


def test_engine_rejects_invalid_request_with_clear_error():
    with pytest.raises(EngineInputError, match="difficulty"):
        run_engine({"windows": [{"start": 0, "end": 60}],
                    "tasks": [{"id": 1, "difficulty": 9, "priority": 3, "deadline": 50,
                               "duration_curve": [10]}]})


def test_engine_rejects_unknown_strategy():
    with pytest.raises(EngineInputError, match="strategy"):
        run_engine({"strategy": "magic"})


def test_python_and_cpp_fatigue_models_agree():
    """Cross-language parity: replay the engine's blocks with the Python model."""
    origin = datetime(2030, 1, 7)
    tasks = [Task(f"t{i}", "coding", d, 3, est, id=i)
             for i, (d, est) in enumerate([(5, 90), (2, 30), (4, 60), (1, 20), (3, 45)], 1)]
    windows = work_windows(origin.date(), 1, time(9), time(17))
    request = build_request(tasks, origin, origin + timedelta(days=1), windows, [],
                            "fatigue_aware", DEFAULT_PARAMS, predictor=None)
    response = run_engine(request)
    difficulty = {t.id: t.difficulty for t in tasks}
```

```
    fatigue, clock = 0.0, response["blocks"][0]["start"]
    for block in response["blocks"]:
        fatigue = after_rest(fatigue, block["start"] - clock)
        assert block["fatigue_start"] == pytest.approx(fatigue, abs=1e-9)
        minutes = block["end"] - block["start"]
        if block["kind"] == "task":
            fatigue = after_work(fatigue, difficulty[block["task_id"]], minutes)
        else:
            fatigue = after_rest(fatigue, minutes)
        assert block["fatigue_end"] == pytest.approx(fatigue, abs=1e-9)
        clock = block["end"]


def _add_tasks(conn, start: date):
    repo = TaskRepository(conn)
    origin = datetime.combine(start, time())
    repo.add(Task("Hard and due", "coding", 5, 5, 90, origin + timedelta(hours=12)))
    repo.add(Task("Easy", "admin", 1, 2, 20))
    repo.add(Task("Medium", "writing", 3, 3, 45, origin + timedelta(days=1, hours=17)))
    repo.add(Task("Too long", "study", 3, 3, 1000))


def test_plan_end_to_end_persists_and_updates_statuses(conn):
    start = date(2030, 1, 7)
    _add_tasks(conn, start)
    blocked = [(datetime(2030, 1, 7, 13), datetime(2030, 1, 7, 14))]
    result = plan(conn, start, days=2, blocked=blocked)

    task_blocks = [b for b in result.blocks if b.kind == "task"]
    assert {b.title for b in task_blocks} == {"Hard and due", "Easy", "Medium"}
    assert [tid for tid, _ in result.unscheduled] == [4]
    assert result.run_id is not None
    for b in result.blocks:  # nothing inside the blocked lunch hour
        assert b.end <= blocked[0][0] or b.start >= blocked[0][1]
        assert time(9) <= b.start.time() and b.end.time() <= time(17)

    statuses = {t.title: t.status for t in TaskRepository(conn).list()}
    assert statuses == {"Hard and due": "scheduled", "Easy": "scheduled",
                        "Medium": "scheduled", "Too long": "pending"}


def test_dry_run_does_not_persist(conn):
    _add_tasks(conn, date(2030, 1, 7))
    plan(conn, date(2030, 1, 7), persist=False)
    assert conn.execute("SELECT COUNT(*) FROM schedule_runs").fetchone()[0] == 0


def test_compare_runs_both_strategies_on_same_input(conn):
    _add_tasks(conn, date(2030, 1, 7))
    results = compare(conn, date(2030, 1, 7), days=2)
    assert set(results) == {"fatigue_aware", "edf"}
    assert results["edf"].metrics["breaks"] == 0


def test_log_session_reconstructs_fatigue_even_out_of_order(conn):
    repo = TaskRepository(conn)
    a = repo.add(Task("a", "coding", 5, 3, 60))
    b = repo.add(Task("b", "admin", 1, 3, 30))
    day = datetime(2030, 1, 8)
    # Log the LATER session first, then the earlier one.
    later = log_session(conn, b.id, day + timedelta(hours=11), 30)
    assert later.fatigue_at_start == 0.0  # alone in the day -> fresh
    log_session(conn, a.id, day + timedelta(hours=9), 60)
    refreshed = conn.execute("SELECT fatigue_at_start FROM work_sessions WHERE task_id = ?",
                             (b.id,)).fetchone()[0]
    expected = after_rest(after_work(0.0, 5, 60), 60)
    assert refreshed == pytest.approx(expected, abs=1e-6)
    assert repo.get(a.id).status == "done"


def test_log_session_validates_input(conn):
```

```
    with pytest.raises(ValueError):
        log_session(conn, 999, datetime(2030, 1, 8, 9), 30)
```

`test_cli.py`

File: `tests/test_cli.py`

```
import argparse
from datetime import datetime, time

import pytest

from conftest import requires_engine
from flexsched.cli import main, parse_block, parse_clock, parse_datetime, table


def test_parse_datetime_accepts_common_formats():
    assert parse_datetime("2026-10-07 13:30") == datetime(2026, 10, 7, 13, 30)
    assert parse_datetime("2026-10-07T13:30") == datetime(2026, 10, 7, 13, 30)
    assert parse_datetime("2026-10-07") == datetime(2026, 10, 7)
    with pytest.raises(argparse.ArgumentTypeError):
        parse_datetime("next tuesday")


def test_parse_block_and_clock():
    assert parse_block("2026-10-07 13:00-14:00") == (datetime(2026, 10, 7, 13),
                                                     datetime(2026, 10, 7, 14))
    assert parse_clock("08:45") == time(8, 45)
    with pytest.raises(argparse.ArgumentTypeError):
        parse_block("2026-10-07 14:00-13:00")


def test_table_aligns_columns():
    out = table(["a", "long header"], [[1, "x"], [22, "yy"]]).splitlines()
    assert len({len(line.rstrip()) for line in out[:2]}) == 1  # header and rule align
    assert out[1].startswith("--")


def test_add_list_and_validation(tmp_path, capsys):
    db = str(tmp_path / "t.db")
    assert main(["--db", db, "add", "Write report", "-c", "writing", "-d", "3",
                 "-p", "4", "-e", "45", "--deadline", "2030-01-08 17:00"]) == 0
    assert main(["--db", db, "list"]) == 0
    out = capsys.readouterr().out
    assert "Write report" in out and "2030-01-08 17:00" in out
    with pytest.raises(SystemExit):  # argparse rejects difficulty 9
        main(["--db", db, "add", "x", "-c", "coding", "-d", "9", "-e", "10"])


def test_train_without_data_fails_gracefully(tmp_path, capsys):
    assert main(["--db", str(tmp_path / "t.db"), "train"]) == 1
    assert "need at least" in capsys.readouterr().err


@requires_engine
def test_full_cli_workflow(tmp_path, capsys):
    common = ["--db", str(tmp_path / "t.db"), "--model", str(tmp_path / "m.joblib")]
    assert main(common + ["seed", "--days", "60"]) == 0
    assert main(common + ["train"]) == 0
    assert main(common + ["stats"]) == 0
    assert main(common + ["add", "Big feature", "-c", "coding", "-d", "5", "-p", "5",
                          "-e", "90"]) == 0
    assert main(common + ["plan", "--start", "2030-01-07", "--days", "1",
                          "--block", "2030-01-07 13:00-14:00"]) == 0
    assert main(common + ["runs"]) == 0
    out = capsys.readouterr().out
    assert "chosen" in out and "Big feature" in out and "fatigue_aware" in out
```

4.14 Project configuration files

`pyproject.toml`

File: `pyproject.toml`

```toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "flexsched"
version = "1.0.0"
description = "Fatigue-aware task scheduler with ML duration prediction and a C++ scheduling engine"
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
    "numpy>=1.26",
    "pandas>=2.1",
    "scikit-learn>=1.4",
    "joblib>=1.3",
]

[project.optional-dependencies]
dev = ["pytest>=8"]

[project.scripts]
flexsched = "flexsched.cli:main"

[tool.setuptools]
packages = ["flexsched"]

[tool.setuptools.package-data]
flexsched = ["schema.sql"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = [".", "tests"]
addopts = "-q"
```

Declares the package, its dependencies, the `flexsched` console script (`pip install -e .` makes `flexsched plan ...` work directly) and pytest settings (`pythonpath` so tests can import `flexsched` and `conftest`).

`requirements.txt`

File: `requirements.txt`

```
numpy>=1.26
pandas>=2.1
scikit-learn>=1.4
joblib>=1.3
pytest>=8
```

`.gitignore`

File: `.gitignore`

```
# Python
__pycache__/
*.py[cod]
*.egg-info/
```

```
.venv/
venv/
.pytest_cache/

# C++ build output
build/
cmake-build-*/
out/

# Local runtime data (databases and trained models are generated, not versioned)
data/
models/
*.db
*.joblib

# Editors / OS
.vscode/
.idea/
*.swp
.DS_Store
Thumbs.db
```

Build output, virtual environments, local databases and trained models are **generated artefacts** and are never committed; anyone can regenerate them with the commands in the README.

`.github/workflows/ci.yml`

File: `.github/workflows/ci.yml`

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build-and-test:
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
    runs-on: ${{ matrix.os }}

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install Python dependencies
        run: python -m pip install -r requirements.txt

      - name: Configure C++ engine
        run: cmake -S engine -B build -DCMAKE_BUILD_TYPE=Release

      - name: Build C++ engine
        run: cmake --build build --config Release

      - name: C++ unit tests (Catch2 via CTest)
        run: ctest --test-dir build -C Release --output-on-failure

      - name: Python tests (pytest)
        run: python -m pytest
```

A **matrix** build: the same steps on Ubuntu (GCC) and Windows (MSVC). `fail-fast: false` lets both finish so you see every failure at once.

Part 5 — Algorithms & design deep dive

5.1 The fatigue model, with numbers

Parameters: `work_rate = 0.012`, `recovery_rate = 0.05`, load = difficulty / 5.

Situation	Formula	Result
60 min of difficulty-4 work, fresh	`1 − (1 − 0)·e^(−0.012·0.8·60)` = `1 − e^(−0.576)`	**0.438**
60 min of difficulty-5 work, fresh	`1 − e^(−0.72)`	0.513
60 min of difficulty-1 work, fresh	`1 − e^(−0.144)`	0.134
15-min break at F = 0.693	`0.693·e^(−0.75)`	**0.327**
115-min gap (lunch + meeting) at F = 0.327	`0.327·e^(−5.75)`	**0.001**
Night (16 h) at F = 1.0	`e^(−48)`	≈ 0

Properties worth stating in an interview:

- **Bounded**: F always stays in [0, 1], by construction and by clamping.
- **Monotonic**: more work → more fatigue; more rest → less fatigue.
- **Diminishing**: the same hour adds less fatigue when you're already tired (headroom shrinks), which is a realistic saturation.
- **Two parameters**, both interpretable. They're defaults, not fitted to a real person (see limitations); the roadmap shows how to fit them from logged data.

5.2 Free slots, worked example

Working windows: day 1 09:00–17:00 (540–1020) and day 2 09:00–17:00 (1980–2460). Blocked: lunch 13:00–14:00 (780–840).

```
windows:   [540 ──────────────────────── 1020]        [1980 ────────── 2460]
blocked:                 [780 ── 840]
free:      [540 ──────── 780]  [840 ─── 1020]         [1980 ────────── 2460]
```

5.3 A full scheduler trace (`engine/examples/request.json`)

Tasks (durations shown at fatigue 0):

id	difficulty	priority	deadline	duration @F=0
1	5	5	day 2 17:00	120
2	4	4	day 1 15:00	60
3	1	3	day 1 12:00	20
4	2	4	day 1 17:00	30
5	3	2	none (9999)	45

Decision at 09:00 (F = 0): all five fit before 13:00. Scores (`0.35·P + 0.45·U + 0.20·E`):

id	slack (min)	priority term P	urgency U	energy E	score
1	1800	1.00	0.024	1.00	0.561
2	300	0.75	0.535	0.75	**0.653**
3	160	0.50	0.717	0.00	0.497
4	450	0.75	0.392	0.25	0.489
5	9414	0.25	0.000	0.50	0.188

Task 2 wins. The rollout guard checks "2 first, then EDF (3, 4, 1, 5)" against "pure EDF (3, 2, 4, 1, 5)". Neither causes lateness, so task 2 is accepted.

Resulting timetable (fatigue-aware):

time	block	F start → end	why
09:00–10:00	task 2 (diff 4)	0.000 → 0.438	best score while fresh
10:00–10:22	task 3 (diff 1)	0.438 → 0.467	urgent (due 12:00), easy; duration 22 not 20 because tired
10:22–10:56	task 4 (diff 2)	0.467 → 0.547	
10:56–11:50	task 5 (diff 3)	0.547 → 0.693	the only one left that fits before 13:00
11:50–12:05	**break**	0.693 → 0.327	F ≥ 0.65
12:05–13:00	(free, unused)		task 1 needs ~136 min at F = 0.33: doesn't fit before 13:00
13:00–14:00	(blocked)		rest: F → 0.001
14:00–16:01	task 1 (diff 5)	0.001 → 0.766	the hardest task gets a fresh mind

EDF on the same input does 3, 2, 4, 5, then 1 with no break. The 12:05–13:00 hole shows a real limitation: tasks can't be split (see the roadmap).

5.4 The rollout deadline guard: how it evolved (a good story for interviews)

The first version of the fatigue-aware policy was *score + breaks* only. The benchmark exposed a problem:

version	idea	avg total lateness (min)	EDF reference
v1	score + breaks	143 *(100 scenarios)*	101
v2	one-step guard: don't postpone the single most urgent task	114	91
v3	rollout: "this task first, then EDF", continuous time	112	91
v4	rollout packs tasks into real **free slots** (nights, meetings)	96	91
v5 (final)	rollout over **all remaining tasks**, not just those fitting now	**90.4**	**90.9**

Two lessons:

1. **An ablation found the real cause.** I first suspected breaks (they cost time), so I ran the policy with breaks disabled. Lateness was unchanged, so breaks weren't the culprit; the **ordering decisions** were. That pointed to the lookahead.
2. **The lookahead was blind to nights.** Simulating "back to back" made a large due-tomorrow task look safe at 16:00. Packing into real slots fixed most of the gap, and giving the rollout every remaining task closed it.

Mechanism in one sentence: *pick the highest-scoring task whose "do it first, then EDF" simulation produces no more late tasks and no more late minutes than plain EDF would from the same moment.*

Cost: for k candidates and n pending tasks, each decision costs O(k·(n log n + S)), which is milliseconds for realistic sizes.

5.5 Why greedy? (scheduling theory you can quote)

- With fixed durations on one machine, **EDF (Jackson's rule) minimises the maximum lateness**, so it's the natural baseline.
- Minimising the **number of late jobs** has an exact O(n log n) algorithm (Moore–Hodgson).
- Minimising **total tardiness** on one machine is **NP-hard** (Du & Leung, 1990).
- FLEXSched's durations also depend on **the order** (fatigue carries over), which makes exact optimisation even harder.

So FLEXSched uses a **greedy simulation with a lookahead guard**: fast, deterministic and explainable, protected against the main failure mode (deadlines), and benchmarked against the textbook baseline. The roadmap proposes an exact/metaheuristic optimiser (OR-Tools CP-SAT or local search) as an upgrade.

5.6 ML design decisions (summary)

Decision	Why	Alternative rejected
Predict **ratio** actual/estimate	scale-free; trees can't extrapolate raw minutes	raw minutes as target
Temporal 80/20 split	mirrors real use (past → future), no leakage	random split
`TimeSeriesSplit` CV for model selection	honest choice; hold-out touched once	choosing by hold-out score
Two **baselines**	proves the model adds value	reporting model scores alone
`Pipeline` + `ColumnTransformer`	no preprocessing leakage, one artefact to save	manual preprocessing
Fatigue as a feature → curve	lets the engine read duration at any fatigue	one fixed prediction per task
Monotone curve (running max)	encodes domain knowledge, removes model noise	trusting raw tree output
Refit on all data after evaluation	production uses every sample	shipping the 80% model
Heuristic fallback	the system works from day one (cold start)	refusing to plan

5.7 Design patterns and principles

Pattern / principle	Where
Strategy	`SelectionPolicy` with `EdfPolicy`, `FatigueAwarePolicy`
Factory	`make_policy(name)`
Repository	`TaskRepository`, `SessionRepository`, `ScheduleRepository`
Service layer	`planner.py` coordinates repositories, predictor, engine
Pipeline	scikit-learn preprocessing + model
Adapter / anti-corruption layer	`engine_bridge.py` + `build_request` translate between Python objects and the engine contract
Single Responsibility	one concern per C++ module and per Python module
Open/Closed	new strategies without touching the scheduler loop
Dependency Inversion	`Scheduler` depends on the abstract `SelectionPolicy`
RAII	`std::unique_ptr` ownership; `with conn:` transactions in Python
Fail fast	validation in engine, repository, CLI and DB constraints
Determinism	stable tie-breaks, seeded RNGs, fixed benchmark date

5.8 Complexity summary

Operation	Time
`merge_intervals`	O(m log m)
`free_slots`	O(W log W + B log B)
`effective_duration`	O(1)
one scheduling decision	O(n) durations + O(k·(n log n + S)) guard
whole plan	≈ $O(n^2 \cdot (n \log n + S))$ worst case; ms for n ≤ 100
ML training	dominated by 5 models × 6 fits; ~8 s on 445 rows
curve prediction	one vectorised `predict` for all tasks × 5 levels
engine call overhead	~50 ms (process start-up on Windows)

Part 6 — Results, limitations, running the project

6.1 Reproducing every number in this guide

```
python -m flexsched seed            # 120 days, seed 42 → 445 sessions
python -m flexsched train           # model table + permutation importance
python -m flexsched stats           # estimation bias per category
python -m flexsched benchmark --scenarios 200      # seed 7
ctest --test-dir build              # 39 Catch2 test cases
python -m pytest                    # 49 tests
```

Duration prediction

```
model              CV MAE (min)  hold-out MAE  MAPE   R^2
-----------------  ------------  ------------  -----  -----
trust_estimate     15.55         17.47         21.5%  0.614
mean_ratio         13.39         14.04         20.3%  0.763
ridge              7.20          6.68          10.0%  0.938  <- chosen
random_forest      8.58          7.08          10.3%  0.932
gradient_boosting  7.93          6.91          11.1%  0.943

Permutation importance:
  category +0.098 | difficulty +0.087 | fatigue_at_start +0.040 | hour_of_day +0.020
  priority +0.016 | estimate_min -0.000 | day_of_week -0.003
```

How to read it: error drops from 17.5 to 6.7 minutes (−62%). The importance ranking matches the simulated habits; `day_of_week` ≈ 0 is correct because the generator has no weekday effect, so the model isn't inventing patterns. `estimate_min` ≈ 0 is expected because the target is already *relative* to the estimate.

Estimation bias (`stats`)

```
category  sessions  avg estimate  avg actual  actual/estimate
coding    72        55.6          80.3        1.433
writing   77        53.2          73.7        1.366
design    70        53.0          66.6        1.29
study     71        50.9          62.2        1.23
review    76        50.8          56.6        1.102
admin     79        59.7          57.6        0.964
```

Scheduling benchmark (200 scenarios, paired)

```
metric (mean)         fatigue_aware  edf
total_work_minutes    653.920        664.335
makespan              2306.335       2293.180
tasks_scheduled       10.620         10.680
tasks_late            0.425          0.320
total_lateness        90.440         90.855
breaks                2.415          0.000
peak_fatigue          0.783          0.842
avg_fatigue_at_start  0.265          0.321

fatigue_aware strictly better: work minutes 68% | avg fatigue 80% | peak fatigue 82%
```

Honest reading:

- **Wins:** −17% average fatigue at task start, −7% peak fatigue, −1.6% total work time (tasks done in a better state take less time per the model, which more than pays for ~36 minutes of breaks).
- **Ties:** total lateness (90.4 vs 90.9 minutes).
- **Costs:** slightly more late tasks on average (+0.1 per scenario) and a slightly later makespan (breaks take time). The guard bounds lateness *minutes*, not the *count*, and it's approximate (it ignores future fatigue).

6.2 Limitations and assumptions (say these before you're asked)

1. **Synthetic data.** No real-user study; `synthetic.py` defines the "truth". The pipeline is real and would work unchanged on logged data, but the numbers describe the simulated user.
2. **Fatigue model parameters are assumed**, not fitted. The *form* is plausible (saturating growth, exponential recovery) but not validated on people.
3. **Duration depends on fatigue at the start only**; fatigue changes during a long task aren't fed back into its duration.
4. **Planning hour fixed at 11:00** when predicting curves (the start time isn't known until scheduling). Hour-of-day effects are therefore only approximated at plan time.
5. **No task splitting / pre-emption.** A 2-hour task can't use two 1-hour holes (see the unused 12:05–13:00 in Part 5.3).
6. **Blocked time counts as rest**: meetings are treated like breaks.
7. **Greedy + heuristic lookahead**, not an optimal solver.
8. **Single user, local CLI**: no accounts, sync, concurrency control or UI.
9. **Subprocess bridge** costs ~50 ms per call: fine for planning, wasteful for bulk simulation.

Each limitation maps to a concrete item in `docs/FUTURE_ROADMAP.md`.

6.3 Resume bullets ↔ evidence in the code

Resume claim	Evidence
"fatigue-aware task scheduling system"	`fatigue.hpp/.cpp`, `FatigueAwarePolicy`, breaks in `scheduler.cpp`, duration curves
"using historical task data"	`work_sessions` table, `TRAINING_SQL`, `log_session` + `replay_day`
"machine-learning-based duration prediction"	`predictor.py`: 5 pipelines, TimeSeriesSplit CV, hold-out, permutation importance
"modular scheduling logic"	6 C++ modules with one responsibility each, Strategy + Factory
"task management"	`add` / `list` / `done` /statuses, `TaskRepository`, validation
"data persistence with Python, C++ and SQLite"	`schema.sql`, `db.py`, `repository.py`; C++ engine exchanges state via JSON
"Catch2 unit tests … scheduling modules and edge cases"	6 test files, 39 cases, `[edge]`, `[invariant]`, `[determinism]` tags

Optional stronger, still-honest versions of your bullets:

- *Developed a fatigue-aware task scheduler that predicts task durations from historical work logs with scikit-learn (time-series CV across 5 models), reducing prediction error 62% vs. user estimates on held-out data.*
- *Built a modular C++17 scheduling engine (free-slot computation, fatigue simulation, pluggable EDF / fatigue-aware policies with a rollout deadline guard) integrated with a Python + SQLite planning layer via a JSON contract; lowered simulated fatigue 17% at equal lateness over 200 benchmark scenarios.*
- *Wrote 39 Catch2 test cases (340+ assertions) covering scheduling invariants, edge cases and determinism, plus 49 pytest tests incl. a Python–C++ parity test; CI on Linux and Windows.*

(If you use the numbers, be ready to say "on synthetic data".)

6.4 Setup on a new machine

Windows (what this project was built with): Python 3.13, CMake, and either WinLibs GCC (`winget install BrechtSanders.WinLibs.POSIX.UCRT`) with Ninja, or Visual Studio Build Tools (MSVC).

```
cmake -S engine -B build -G Ninja -DCMAKE_BUILD_TYPE=Release   # GCC + Ninja
cmake -S engine -B build                                        # or MSVC (Visual Studio generator)
cmake --build build --config Release
ctest --test-dir build -C Release
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
python -m pytest
python -m flexsched demo
```

Linux / macOS: same commands; activate with `source .venv/bin/activate` .

6.5 Troubleshooting

Symptom	Cause / fix
`C++ engine not found`	build it (commands above) or set `FLEXSCHED_ENGINE` to the binary path
CMake: `unable to find a build program corresponding to "Ninja"`	Ninja not on PATH; add the compiler's `bin` folder to PATH or pass `-DCMAKE_MAKE_PROGRAM=...`
CMake download errors	first configure needs internet (Catch2, nlohmann/json via FetchContent)
`need at least 40 logged sessions`	run `seed` (demo data) or log more sessions with `done`
integration tests "skipped"	the engine isn't built yet; pytest skips rather than fails
`sqlite3.IntegrityError`	a constraint caught invalid data; the message names the column
model fails to load after upgrading scikit-learn	retrain: `python -m flexsched train`

6.6 Repository history (commit by commit)

The repository was built in reviewable steps: scaffold → persistence → fatigue model and synthetic history → ML predictor → C++ engine → Catch2 tests → Python–engine bridge, planner and CLI → pytest suite and CI

→ this guide → future roadmap. `git log --oneline` shows the sequence.

Part 7 — Interview drill

7.0 How to use this drill

Interviewers rarely ask one question. They take one line of your resume and **drill**: *what → why → what exactly → why not something else → what breaks → how would you scale it*. Each section below follows one resume line through that ladder, from **[Basic]** to **[Expert]**. The model answers are deliberately short. Learn the *reasoning*, not the wording.

Answer shape that works almost everywhere: **claim → reason → evidence → trade-off.**

> "I used a temporal split *(claim)* because the model predicts future sessions from past ones *(reason)*; a random split scored better but leaked future habits *(evidence)*; the cost is a smaller, more recent test set *(trade-off)*."

Your resume lines:

> **FLEXSched – Intelligent Task Scheduler & Planner** · *Python, C++, scikit-learn, SQLite, Catch2* 1. Developed a fatigue-aware task scheduling system using historical task data and machine-learning-based duration prediction. 2. Implemented modular scheduling logic, task management and data persistence with Python, C++ and SQLite. 3. Wrote Catch2 unit tests to validate scheduling modules and edge cases, improving confidence in core logic.

The drill covers four points: **(A)** the headline, **(B)** bullet 1, **(C)** bullet 2, **(D)** bullet 3, then **(E)** tech-stack justification, **(F)** cross-cutting and behavioural questions, **(G)** rapid fire.

A. The headline: "Intelligent Task Scheduler & Planner"

A1 [Basic] What is FLEXSched in one sentence? A planner that learns how long your tasks really take, including how much slower you are when tired, and builds a timetable that schedules hard work when you're fresh, adds breaks, and protects deadlines.

A2 [Basic] Who is it for and what problem does it solve? Students and developers who plan their own days. It solves two problems ordinary calendars ignore: people underestimate tasks (planning fallacy) and slow down as they get tired.

A3 [Basic] What does "intelligent" mean here, concretely? Isn't that a buzzword? Two concrete things: (1) durations are *learned* from your history with a validated ML model instead of trusted blindly; (2) the scheduler *simulates* your fatigue and decides task order and breaks from it. Without those two, it would be an ordinary to-do list.

A4 [Intermediate] Walk me through the architecture. Python CLI → planner (service layer) → SQLite repositories for data, a scikit-learn pipeline for duration curves, and a C++ engine called as a subprocess

with a JSON request on stdin and a JSON schedule on stdout. The engine has six modules: calendar, fatigue, duration, policy, scheduler and json_io.

A5 [Intermediate] What happens end-to-end when the user runs `plan`? Load open tasks → predict each task's duration at 5 fatigue levels in one vectorised call → convert datetimes to minute offsets → engine computes free slots, simulates the day and picks tasks through the policy → Python maps the result back, saves it in one transaction, updates statuses and prints it.

A6 [Intermediate] Why split it into Python and C++ instead of one language? Each part plays to its ecosystem: Python has scikit-learn, pandas and sqlite3; the scheduling core is a tight, deterministic algorithm where C++ gives speed, static types and Catch2. The JSON boundary keeps them independent, so each can be tested alone. (Honest add-on: at today's problem sizes Python would also be fast enough. The engine is built for larger horizons, many simulations, and a clean core that could ship as a library.)

A7 [Intermediate] How do you know it works? Three layers of evidence: 39 Catch2 test cases with invariants and edge cases; 49 pytest tests including end-to-end and a Python-vs-C++ parity test; and two quantitative evaluations: hold-out ML error and a 200-scenario paired benchmark against EDF.

A8 [Advanced] What results did you get, and what's the catch? 62% lower duration error than trusting estimates (17.5 → 6.7 min MAE) and 17% lower average fatigue at equal total lateness versus EDF. The catch: the data is synthetic, and the fatigue model's parameters are assumed. The pipeline would run unchanged on real logs, but the numbers describe a simulated user.

A9 [Advanced] If you had to remove one component, which and why? The C++ engine could be rewritten in Python with minimal loss at current sizes. I'd keep it because it enforces a clean contract and is where the algorithmic depth lives. The ML model is the component I wouldn't remove: it's the main source of accuracy.

A10 [Expert] How would you turn this into a product with real users? Collect real sessions (timer or calendar integration), fit fatigue parameters per user, add a REST API + web UI, move to PostgreSQL with auth, run the engine as a library or service, add task splitting and an optimiser, and monitor prediction error per user with retraining. (Details: `FUTURE_ROADMAP.md`.)

B. Bullet 1: "fatigue-aware scheduling using historical task data and ML-based duration prediction"

B-I. Fatigue (what → why → what → why)

B1 [Basic] What does "fatigue-aware" mean? The scheduler tracks a fatigue number between 0 (fresh) and 1 (exhausted) as it builds the plan. Work raises it, rest lowers it, and it influences task order, durations and breaks.

B2 [Basic] Why model fatigue at all? Because duration and quality depend on it. A plan assuming you're as fast at 4 pm as at 9 am is wrong in a predictable way, and predictable errors can be corrected.

B3 [Intermediate] What exactly is your fatigue model? Work: `F' = 1 − (1 − F)·e^(−rate·(difficulty/5)·minutes)`, a saturating increase. Rest: `F' = F·e^(−recovery·minutes)`, exponential recovery. Rate 0.012/min, recovery 0.05/min.

B4 [Intermediate] Why those formulas and not, say, linear? Linear growth would exceed 1 and needs clamping hacks; the exponential form is bounded, monotonic and saturating, and harder tasks tire faster through the load factor. Recovery is fast at first and slows down, which matches how a short break restores much of your focus. Each formula has one parameter that's easy to explain.

B5 [Intermediate] Where does fatigue change decisions? Three places: (1) **duration**, since the engine reads each task's curve at the current fatigue; (2) **ordering**, since the energy-match term prefers hard tasks when fresh and easy ones when tired; (3) **breaks**, inserted when F ≥ 0.65 (unless the slot is ending anyway).

B6 [Advanced] How do you know the fatigue at the start of past sessions? Users don't log "I'm 0.4 tired". I reconstruct it: for each day, replay the logged sessions through the same model (start fresh, rest during gaps, work during sessions). `log_session` recomputes the whole day on each insert, so out-of-order logging still gives correct values.

B7 [Advanced] Isn't it circular to use an assumed fatigue model to create a feature, then learn from it? It's a modelling assumption, not circular logic: the feature is a deterministic summary of "how much work, how recently". If the assumption were useless, the model would learn that `fatigue_at_start` has ~0 importance. With real data it would be validated the same way. The next step is to *fit* the rates by maximising predictive accuracy (roadmap).

B8 [Expert] How would you fit the fatigue parameters from data? Treat work_rate and recovery_rate as hyper-parameters: grid-search or Bayesian-optimise them, recompute `fatigue_at_start` for each candidate, and choose the pair with the best time-series-CV MAE. Or put the model inside a differentiable pipeline and learn them jointly. Per-user parameters would need hierarchical pooling to avoid overfitting sparse users.

B-II. Historical data

B9 [Basic] What historical data do you use? Work sessions: task id, start, end, actual minutes and reconstructed fatigue at start, joined with task attributes (category, difficulty, priority, estimate).

B10 [Basic] Where does it come from in your demo? A seeded simulator (`synthetic.py`) that generates 120 days of a person with hidden habits. Real usage logs through `flexsched done` into the same tables.

B11 [Intermediate] Why synthetic data? Isn't that cheating? There's no public dataset of personal task durations with fatigue context, and a new user has no history. The simulator gives a controlled ground truth, so I can check the pipeline *recovers* known patterns: the permutation importances line up with the hidden habits, and `day_of_week`, which has no effect in the generator, gets ≈ 0. I always state the numbers are on synthetic data.

B12 [Intermediate] What hidden habits does the simulator have? Category bias (coding ×1.35, admin ×0.9), +8% per difficulty step above 3, −3% per priority step (focus), up to +55% when exhausted, faster mornings, a 10% post-lunch dip, and ~12% log-normal noise.

B-III. ML duration prediction

B13 [Basic] What exactly does the model predict? The ratio actual/estimate; the prediction in minutes is ratio × the user's estimate.

B14 [Intermediate] Why predict a ratio instead of minutes? It's scale-free: "I take 1.4× for coding" holds for 30- and 120-minute tasks. Tree models can't extrapolate beyond targets they've seen, and the ratio sidesteps that. It also makes the baseline natural (ratio = 1 means "trust the estimate").

B15 [Intermediate] Which features, and why each? Category (bias differs by kind of work), estimate (anchor), difficulty (underestimation grows with difficulty), priority (focus), fatigue at start (the core idea), hour of day (circadian/lunch dip), day of week (tested, turned out irrelevant).

B16 [Intermediate] Which models did you try and which won? Two baselines (trust estimate, mean ratio), Ridge, Random Forest and Gradient Boosting. Ridge won on cross-validated MAE (7.2 min), with GBR/RF close. Hold-out MAE was 6.7 vs 17.5 for trusting the estimate.

B17 [Advanced] Why did a linear model beat the ensembles? In ratio space the habits are near-multiplicative factors combining almost linearly, so a regularised linear model captures them with low variance; trees add variance and approximate smooth effects with steps. I let cross-validation decide instead of assuming the most complex model wins. On real, messier data the result might differ, which is why the selection is automatic.

B18 [Advanced] How did you evaluate without fooling yourself? Temporal 80/20 split (past → future); model selection by `TimeSeriesSplit` CV on the training part only; a single evaluation on the hold-out; comparison against two baselines; all preprocessing inside a `Pipeline` so scaler/encoder statistics never see validation data.

B19 [Advanced] What's data leakage and where could it have happened here? Information from the evaluation data influencing training or selection. Here it could have come from: a random split (future habits in training), picking the model by hold-out score, fitting the scaler on all data before CV, or features computed from future sessions. Each is prevented by design.

B20 [Advanced] MAE vs RMSE vs MAPE: why MAE? MAE is in minutes and directly interpretable ("off by 6.7 minutes on average") and is robust to outliers. RMSE punishes big misses more (useful if over-runs are very costly). MAPE is scale-free but explodes for tiny actual values. I report MAE (primary), MAPE and R^2.

B21 [Advanced] How does the model plug into the scheduler? It predicts minutes at fatigue 0, 0.25, 0.5, 0.75, 1 → a duration **curve** per task. The engine linearly interpolates at the simulated fatigue and rounds up. I enforce monotonicity with a running maximum because more fatigue should never make you faster.

B22 [Expert] Your curve assumes 11:00 as start hour. Isn't that wrong for afternoon tasks? Yes, it's an approximation, because the start time is unknown until scheduling. Options: predict a 2-D grid (fatigue × hour) and interpolate both in the engine; or iterate (schedule → re-predict with actual start times → reschedule until stable). The grid is the cleaner fix and is in the roadmap.

B23 [Expert] How would you get uncertainty, not just a point estimate? Quantile regression (e.g. GBR with quantile loss for P50/P80) or conformal prediction intervals on the time-series CV residuals. The scheduler could then book P80 for deadline-critical tasks and P50 otherwise, which is a risk-aware schedule.

B24 [Expert] How would you handle a brand-new user (cold start)? Today: a heuristic curve (+50% at full fatigue). Better: a global model trained on all users as a prior, blended with the user's own data as it accumulates (hierarchical / mixed-effects model, or per-user offsets on a shared model).

C. Bullet 2: "modular scheduling logic, task management and data persistence with Python, C++ and SQLite"

C-I. Modular scheduling logic

C1 [Basic] What does "modular" mean in your engine? Six modules with one job each: calendar (free slots), fatigue (dynamics), duration (curve interpolation), policy (decision rules), scheduler (the loop), json_io (contract). Each has its own header, source and test file.

C2 [Basic] What does the scheduler do, step by step? Walk through free slots; rest during gaps; take a break if too tired; find tasks that fit at the current fatigue; ask the policy which to do; place it; update fatigue and clock; repeat; explain leftovers.

C3 [Intermediate] What design pattern is the policy? Strategy: an abstract `SelectionPolicy` with `EdfPolicy` and `FatigueAwarePolicy`, created by a factory from a string. The loop never changes when I add a strategy (Open/Closed principle).

C4 [Intermediate] Explain the fatigue-aware score. `0.35·priority + 0.45·urgency + 0.20·energy-match`. Priority is normalised 0..1; urgency is 1 with no slack and decays as `e^(−slack/480)`; energy match is `1 − |difficulty − energy|`, highest for hard-when-fresh or easy-when-tired.

C5 [Intermediate] Why EDF as the baseline? It's the textbook rule: with fixed durations on one machine, EDF minimises the maximum lateness (Jackson's rule). It's simple, strong and fatigue-blind, which is exactly the comparison I want.

C6 [Advanced] Your first version had worse lateness than EDF. What did you do? An ablation showed breaks weren't the cause; ordering was. I added a lookahead: take the best-scored task only if "this task first, then EDF" produces no more late tasks or minutes than pure EDF. Two fixes followed: pack the simulation into real slots (it ignored nights), and include all remaining tasks. Lateness went 143 → 90 minutes (EDF: 91) while keeping the fatigue gains.

C7 [Advanced] Is your scheduler optimal? Why greedy? No. Minimising total tardiness on one machine is NP-hard even with fixed durations, and my durations depend on order through fatigue. Greedy with a lookahead is fast, deterministic and explainable; I benchmark it against EDF to show where it wins and where it costs. An optimiser such as CP-SAT or local search is the upgrade path.

C8 [Advanced] Complexity of planning? Free slots: O(W log W + B log B). Per decision: O(n) durations plus up to k rollouts of O(n log n + S). Worst case is roughly $O(n^2 \cdot (n \log n + S))$ per plan, which is milliseconds for $n \le 100$.

C9 [Expert] How did you guarantee the loop terminates? Every iteration does at least one of: advance time to a later slot, consume a slot, take a positive-length break, or schedule a task. Slots and tasks are finite, and `break_minutes > 0` and `0 < threshold ≤ 1` are validated in the constructor.

C10 [Expert] Any subtle C++ correctness issue you hit? Two. (1) My ranking comparator first used an epsilon "equal", which violates strict weak ordering, and that's undefined behaviour in `std::sort`; I switched to exact comparisons with deterministic tie-breaks. (2) Duration rounding: the single-point curve path skipped the floating-point epsilon, caught by a Catch2 test (`61 == 60`).

C-II. Task management

C11 [Basic] What can a user do with tasks? Add (title, category, difficulty, priority, estimate, deadline), list, plan, log work (`done`), view stats, past runs and benchmarks, all through the CLI.

C12 [Intermediate] How is input validated? Three layers: argparse types (`int_range(1,5)`, date parsers), `validate_task` in the repository (friendly messages), and SQL `CHECK` / `NOT NULL` /FK constraints (last line of defence, tested by bypassing Python).

C13 [Intermediate] What's the task lifecycle? `pending` → `scheduled` (placed by a saved plan) → `done` (work logged); unscheduled tasks go back to `pending`; `cancelled` is allowed by the schema.

C-III. Data persistence

C14 [Basic] What's stored and where? SQLite file `data/flexsched.db`: `tasks`, `work_sessions`, `schedule_runs`, `schedule_items`. The trained model is a joblib file in `models/`.

C15 [Intermediate] Walk me through the schema decisions. Normalised tables with foreign keys (`ON DELETE CASCADE` for sessions, `SET NULL` for schedule items), `CHECK` constraints for ranges and enums, indexes matching queries (status, started_at, task_id, run_id), ISO-8601 text dates, metrics as JSON because they're read whole.

C16 [Intermediate] How do you prevent SQL injection? Only parameterised queries (`?` placeholders), even for dynamic `IN (...)` lists, where only the placeholder count is generated.

C17 [Intermediate] How do you keep data consistent when saving a plan? A transaction (`with conn:`): the run row and all item rows commit together or roll back together. A test injects an invalid item and asserts no run row remains.

C18 [Advanced] Where does C++ fit in "persistence with Python, C++ and SQLite"? Precisely: SQLite is accessed from Python; the C++ engine is stateless and exchanges all state through a versionable JSON contract. Python persists both its inputs and outputs (runs and items), so every plan is reproducible and comparable later. (Be precise here; don't claim C++ talks to SQLite.)

C19 [Advanced] Why the Repository pattern? It isolates all SQL in one module: the planner works with `Task` objects, tests can use `:memory:`, and moving to PostgreSQL changes one layer.

C20 [Advanced] Why subprocess + JSON instead of pybind11? Decoupling and simplicity: no ABI coupling to a specific Python/compiler (painful on Windows), the engine is testable from the command line, and a crash can't take the Python process down. The cost (~50 ms per call) is irrelevant for planning; for heavy simulation I'd switch to pybind11 or batch requests.

C21 [Expert] What would change with many concurrent users? SQLite allows one writer at a time, which is fine for a desktop app. For a multi-user service: PostgreSQL with connection pooling, row-level ownership (user_id), migrations (Alembic), and the engine as an in-process library or a stateless service behind a job queue.

C22 [Expert] How do you evolve the schema safely? `PRAGMA user_version` holds the version; on start-up apply ordered migrations (`if version < 2: ALTER TABLE ...`) inside a transaction. With PostgreSQL I'd use a migration tool and make changes backward compatible (expand → migrate → contract).

D. Bullet 3: "Catch2 unit tests to validate scheduling modules and edge cases"

D1 [Basic] What is Catch2? A C++ unit-testing framework: `TEST_CASE`s with tags, `SECTION`s, natural assertions (`CHECK(a == b)`) that print both sides on failure, and matchers for floating point.

D2 [Basic] What do your tests cover? All six engine modules: 39 test cases, ~340 assertions. Formulas, interpolation, interval algebra, scoring, the deadline guard, the scheduling loop, JSON parsing and errors.

D3 [Basic] Give three edge cases you test. A task longer than every free slot (reported with a reason); a blocked interval spanning two working days; an empty or NaN duration curve (rejected). Others: no tasks, no free time, duplicate ids, out-of-range values, no break squeezed in at slot end.

D4 [Intermediate] CHECK vs REQUIRE? `CHECK` records a failure and continues; `REQUIRE` aborts the test case. I use REQUIRE before indexing (`REQUIRE(blocks.size()==1)` then `blocks[0]`) so a failure can't turn into a crash.

D5 [Intermediate] What's an invariant test? Give yours. A property that must hold for *any* valid input: blocks never overlap, always lie inside free slots, fatigue stays in [0,1], and every task is either scheduled exactly once or reported unscheduled. I check these for both strategies on a varied 25-task, 3-day workload with meetings.

D6 [Intermediate] How do you test floating-point code? `CHECK_THAT(x, WithinAbs(expected, 1e-12))` against closed-form values (e.g. `1 - e^(-0.72)`), plus relational properties (harder > easier, more rest < less rest).

D7 [Intermediate] Did the tests actually catch bugs? Yes: the rounding epsilon bug in single-point curves (`61 == 60`). Writing the "no break at slot end" test also forced me to reproduce a bug seen in the demo (3-minute breaks at 16:57) before fixing it.

D8 [Advanced] How did you make tests deterministic? The engine has no randomness and breaks ties by deadline then id; there's a test that reversed input order yields an identical schedule. Python randomness is seeded.

D9 [Advanced] How do the C++ tests relate to the Python tests? Catch2 = unit level for the engine; pytest = Python units, integration through the real binary, CLI workflow, and a **cross-language parity test** that replays the engine's fatigue values in Python to 1e-9. CI runs both on Linux and Windows.

D10 [Advanced] Why build the engine logic as a library? `flexsched_core` is linked into both the executable and the test runner, so the tests exercise exactly the code that ships; `main.cpp` stays a thin I/O shell.

D11 [Expert] What would you add to the test strategy? Property-based testing with random generators (RapidCheck / Hypothesis); fuzzing the JSON parser (libFuzzer); sanitizers (ASan/UBSan) in CI; benchmark regression tests (Catch2 `BENCHMARK`); coverage reports (gcov/lcov) with a threshold; mutation testing to measure test strength.

D12 [Expert] "Improving confidence in core logic": how would you quantify that? Line/branch coverage of `engine/src` , the number of invariants checked, mutation score, and defect escape rate (bugs found after

merge). Today I can point to the invariant suite, 39 cases across all modules, a parity test and CI on two OS/compiler combos.

E. Tech-stack justification (what → why → why not → advanced)

Python

- **What for?** Data handling, ML, orchestration, CLI.
- **Why?** scikit-learn, pandas, NumPy and sqlite3 are mature and built-in or one pip away; fast to iterate.
- **Why not only Python?** It would work at today's sizes; the C++ core gives a fast, strictly typed, separately testable engine with a clean contract.
- **Advanced:** the GIL limits CPU parallelism in threads; heavy work happens in NumPy/scikit-learn (C code) or in the engine process.

C++ (17)

- **What for?** The scheduling engine.
- **Why?** Performance for long horizons and many simulations, deterministic behaviour, static typing, RAII, and Catch2 for testing; also a showcase of systems skills.
- **Why C++17?** `std::clamp`, digit separators and guaranteed copy elision (the tests return a move-only `Scheduler` by value); universally supported by GCC, Clang and MSVC.
- **Why not Rust/Go/Java?** All viable; C++ matches the resume target roles and has mature tooling (CMake, Catch2, nlohmann/json).
- **Advanced:** memory safety via `unique_ptr`, no raw `new/delete`; zero warnings with `-Wall -Wextra -Wpedantic`; strict weak ordering in comparators.

scikit-learn

- **What for?** Duration prediction.
- **Why?** Tabular data, a few hundred rows: classical models are the right tool; `Pipeline`, `ColumnTransformer`, `TimeSeriesSplit` and `permutation_importance` cover the whole workflow.
- **Why not deep learning (PyTorch)?** Too little data, less interpretable, slower to train, and no accuracy benefit on small tabular data.
- **Why not XGBoost/LightGBM?** Possible upgrades; scikit-learn's GBR was within 0.2 min of the best model, so an extra dependency wasn't justified.

SQLite

- **What for?** Tasks, history, plans.
- **Why?** Zero setup, single file, ACID, full SQL (joins, constraints, indexes), driver built into Python; ideal for a single-user local app.
- **Why not PostgreSQL/MySQL?** They need a server; their strengths (concurrent writers, roles, replication) aren't needed yet. The repository layer makes a switch easy.
- **Why not CSV/JSON files?** No constraints, no transactions, no indexed queries, easy to corrupt.
- **Advanced:** foreign keys must be enabled per connection; one writer at a time (WAL mode improves concurrency); ISO-8601 text dates.

Catch2

- **What for?** C++ unit tests.
- **Why?** Natural syntax, expression decomposition in failure messages, sections, tags, matchers, CTest integration, easy FetchContent install.
- **Why not GoogleTest?** Equally valid (more popular in big companies, built-in mocking); Catch2 is lighter for a small project. I can explain both.

Supporting choices

- **CMake + FetchContent:** cross-platform builds, dependencies pinned by version.
- **nlohmann/json:** de-facto standard, readable API.
- **pytest:** fixtures, parametrisation, skips. **GitHub Actions:** CI on Ubuntu + Windows.

F. Cross-cutting and behavioural questions

F1. What was the hardest problem? Making fatigue-aware scheduling not hurt deadlines. Benchmarking showed 42% more lateness than EDF; an ablation ruled out breaks; a rollout guard, made slot-aware and then given all remaining tasks, brought lateness level with EDF while keeping 17% lower fatigue.

F2. What would you do differently? Start with the benchmark harness first. It drove every important design decision. And design the duration prediction as a fatigue × hour grid from the beginning.

F3. How did you decide the score weights? Hand-set to favour urgency (0.45) over priority (0.35) and energy (0.20), then checked in the benchmark. A proper next step is tuning them by search over the benchmark objective (e.g. minimise fatigue subject to lateness ≤ EDF).

F4. How would you scale to 1 million users? Stateless API servers; PostgreSQL partitioned by user; the engine as a library inside workers fed by a queue; nightly or event-driven retraining with a shared global model plus per-user calibration; caching of curves; monitoring of per-user MAE drift.

F5. How would you deploy it? Package Python with `pyproject.toml` (already there), build the engine in CI per OS, ship wheels or a Docker image containing both; for a service, a FastAPI container calling the engine library.

F6. Is there any ethical or privacy concern? Work logs reveal habits. Keep data local by default, encrypt at rest if synced, make collection opt-in and explainable, allow deletion, and avoid using fatigue data for employee monitoring.

F7. How do you know your benchmark isn't biased toward your policy? Paired design (same scenarios for both), fixed seeds, the same duration model for both, and I report metrics where my policy loses (late-task count, makespan).

F8. How long does it take to run? Planning: well under a second (engine ≈ 50 ms incl. process start). Training: ~8 s on 445 rows. Benchmark of 200 scenarios: ~25 s (dominated by 400 process starts).

G. Rapid-fire (one-liners)

Question	Answer
Half-open interval?	`[start, end)` : touching intervals don't overlap
Why minutes as int64?	exact arithmetic, no time zones in the engine
Why `unique_ptr` ?	single ownership, automatic cleanup
Virtual destructor why?	correct destruction through base pointer
`const&` parameter?	no copy, no modification
What's RAII?	resources released by destructors automatically
Exit code 2?	invalid input to the engine
One-hot why?	categories have no numeric order
Why scale for Ridge?	L2 penalty assumes comparable feature scales
`TimeSeriesSplit` ?	CV folds that train on the past, validate on the future
Permutation importance?	error increase when a feature is shuffled
R^2 = 0.94?	model explains 94% of duration variance (hold-out)
`PRAGMA foreign_keys` ?	SQLite's FK enforcement is off by default
Transaction?	all-or-nothing group of statements
Index?	B-tree making lookups/range scans O(log n)
Strict weak ordering?	the comparator contract `std::sort` relies on
EDF optimal for?	maximum lateness, fixed durations, one machine
Total tardiness complexity?	NP-hard (1‖ΣTj)
Parity test?	Python and C++ fatigue models give identical numbers
Cold start?	heuristic curve until 40 sessions exist